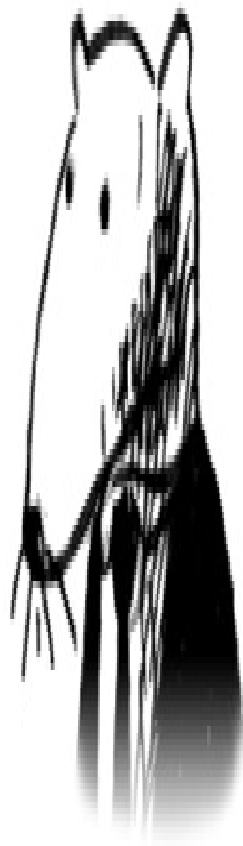


1

Introdução a algoritmos



Neste capítulo

- Você terá acesso ao fundamental para compreender o restante do livro.
- Escreverá seu primeiro algoritmo de busca (pesquisa binária).
- Aprenderá como falar sobre o tempo de execução de um algoritmo (na notação Big O).

- Será apresentado a uma prática comum para projetar algoritmos (recursão).

Introdução

Um *algoritmo* é um conjunto de instruções que realizam uma tarefa. Cada trecho de código poderia ser chamado de um algoritmo, mas este livro trata dos trechos mais interessantes. Escolhi os algoritmos apresentados neste livro porque eles são rápidos ou porque resolvem problemas interessantes, ou por ambos os motivos. A seguir estão descritos alguns pontos importantes que serão demonstrados.

- O Capítulo 1 aborda pesquisa binária e mostra como um algoritmo pode acelerar o seu código. Em um dos exemplos, o número de etapas necessárias passa de 4 bilhões para 32 etapas!
- Um dispositivo de GPS utiliza algoritmos de grafos (que você aprenderá nos Capítulos 6, 7 e 8) para calcular a rota mais curta até o seu destino.
- Você pode usar a programação dinâmica (ver Capítulo 9) para escrever um algoritmo de IA (inteligência artificial) que joga damas.

Em cada caso, descreverei o algoritmo e apresentarei um exemplo. Em seguida, falarei sobre o tempo de execução do algoritmo em notação Big O. Por fim, serão explorados os demais tipos de problemas que poderiam ser solucionados com o mesmo algoritmo.

O que você aprenderá sobre desempenho

Tenho uma boa notícia: uma implementação de cada algoritmo apresentado neste livro provavelmente estará disponível em sua linguagem favorita, portanto você não terá que escrever cada algoritmo! Porém essas implementações serão inúteis caso você não entenda o desempenho dos algoritmos. Neste livro, você aprenderá como comparar o desempenho de diferentes algoritmos: Você deve utilizar merge sort (ordenação por mistura) ou quicksort (ordenação rápida)? Você deve utilizar um array ou uma lista? A escolha da estrutura de dados pode fazer uma grande diferença.

O que você aprenderá sobre a solução de problemas

Você aprenderá técnicas para resolução de problemas que poderiam estar fora da sua gama de habilidades até agora, como por exemplo:

- Se você gosta de criar jogos, poderá desenvolver um sistema de inteligência artificial (IA) que segue o usuário utilizando algoritmos gráficos.
- Você aprenderá a criar um sistema de recomendações utilizando os K-vizinhos mais próximos.
- Alguns problemas não podem ser resolvidos em um tempo hábil! A parte deste livro que trata de problemas NP-completos demonstra como identificar estes problemas e como criar um algoritmo que forneça uma resposta aproximada.

Ao terminar de ler este livro, provavelmente, você conhecerá alguns dos algoritmos de maior aplicabilidade. Assim, poderá usar os seus conhecimentos para aprender sobre algoritmos mais específicos para IA, bancos de dados etc. Além disso, poderá encarar grandes desafios em seu trabalho.

O que você precisa saber

Você deverá conhecer álgebra básica antes de iniciar a leitura deste livro. Em particular, partindo da função $f(x) = x \times 2$, qual será o valor de $f(5)$? Se respondeu 10, você está pronto.

Além de tudo, este capítulo (e este livro) será compreendido mais facilmente se você conhecer alguma linguagem de programação. Todos os exemplos deste livro foram escritos em Python, assim, caso você não conheça nenhuma linguagem de programação e queira aprender uma, escolha Python, pois ela é ótima para iniciantes. Se você conhece alguma outra linguagem, como a Ruby, se sairá bem.

Pesquisa binária



Vamos supor que você esteja procurando o nome de uma pessoa em uma agenda telefônica (que frase antiquada!). O nome começa com *K*. Você pode começar na primeira página da agenda e ir folheando até chegar aos *Ks*. Porém você provavelmente vai começar pela metade, pois sabe que os *Ks* estarão mais perto dali.

Ou suponha que esteja procurando uma palavra que começa com *O* em um dicionário. Novamente, você começa a busca pelo meio.

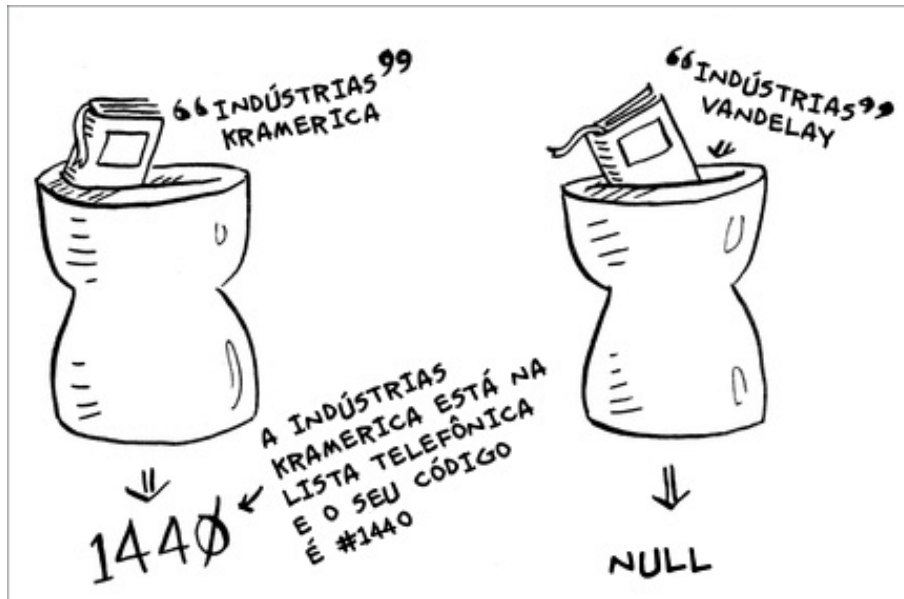
Agora, imagine que você entre no Facebook. Quando faz isso, o Facebook precisa verificar que você tem uma conta no site. Logo, ele procura seu nome de usuário em um banco de dados. Digamos que seu usuário seja karlmageddon. O Facebook poderia começar pelos *As* e procurar seu nome – mas faz mais sentido que ele comece a busca pelo meio.



Isto é um problema de busca. E todos estes casos usam um algoritmo para resolvê-lo: *pesquisa binária*.

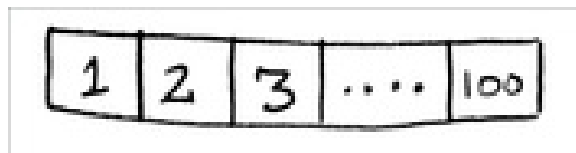
A pesquisa binária é um algoritmo. Sua entrada é uma lista ordenada de elementos (explicarei mais tarde por que motivo a lista precisa ser ordenada). Se o elemento que você está buscando está na lista, a pesquisa binária retorna a sua localização. Caso contrário, a pesquisa binária retorna **None**.

Por exemplo:



Procurando empresas em uma agenda com a pesquisa binária.

Eis um exemplo de como a pesquisa binária funciona. Estou pensando em um número entre 1 e 100.



Você deve procurar adivinhar o meu número com o menor número de tentativas possível. A cada tentativa, digo se você chutou muito para cima, muito para baixo ou corretamente.

Digamos que começou tentando assim: 1, 2, 3, 4... Veja como ficaria.



Uma tentativa ruim de acertar o número.

Isso se chama *pesquisa simples* (talvez *pesquisa estúpida* seja um termo melhor). A cada tentativa, você está eliminando apenas um número. Se o meu número fosse o 99, você precisaria de 99 chances para acertá-lo!

Uma maneira melhor de buscar

Aqui está uma técnica melhor. Comece com 50.



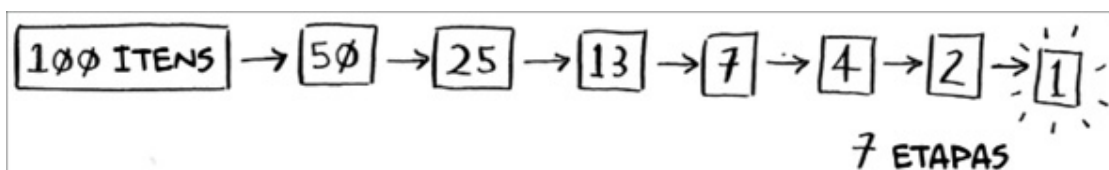
Muito baixo, mas você eliminou *metade* dos números! Agora, você sabe que os números de 1 a 50 são muito baixos. Próximo chute: 75.



Muito alto, mas novamente você pode cortar metade dos números restantes! Com a *pesquisa binária*, você chuta um número intermediário e elimina a metade dos números restantes a cada vez. O próximo número é o 63 (entre 50 e 75).



Isso é a pesquisa binária. Você acaba de aprender um algoritmo! Aqui está a quantidade de números que você pode eliminar a cada tentativa.



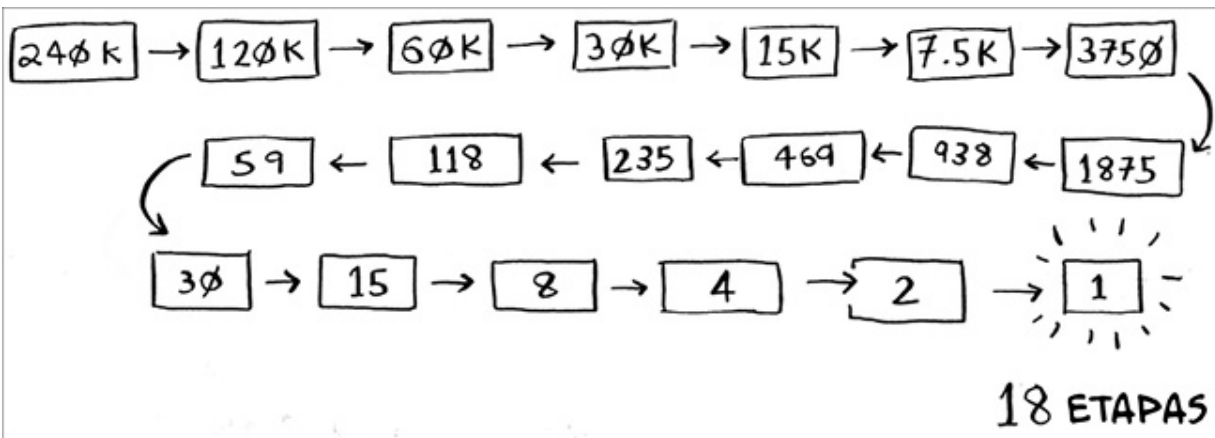
Elimine metade dos números a cada tentativa com a pesquisa binária.

Seja qual for o número que eu estiver pensando, você pode adivinhá-lo em um máximo de sete tentativas – porque a pesquisa binária elimina muitas possibilidades!

Suponha que você esteja procurando uma palavra em um dicionário. O dicionário tem 240.000 palavras. Na *pior das hipóteses*, de quantas etapas você acha que a pesquisa precisaria?

PESQUISA SIMPLES: _____ ETAPAS
 PESQUISA BINÁRIA: _____ ETAPAS

A pesquisa simples poderia levar 240.000 etapas se a palavra que você estivesse procurando fosse a última do dicionário. A cada etapa da pesquisa binária, você elimina o número de palavras pela metade até que só reste uma palavra.



Logo, a pesquisa binária levaria apenas 18 etapas – uma grande diferença! De maneira geral, para uma lista de n números, a pesquisa binária precisa de $\log_2 n$ para retornar o valor correto, enquanto a pesquisa simples precisa de n etapas.

Logaritmos

Você pode não se lembrar de logaritmos, mas provavelmente lembra-se de como calcular exponenciais. A expressão $\log_{10} 100$ basicamente diz: “Quantos 10s conseguimos multiplicar para chegar a 100?”. A resposta é 2: 10×10 . Então, $\log_{10} 100 = 2$. Logaritmos são o oposto de exponenciais.

$10^2 = 100$	$\leftrightarrow$	$\log_{10} 100 = 2$
$10^3 = 1000$	$\leftrightarrow$	$\log_{10} 1000 = 3$
$2^3 = 8$	$\leftrightarrow$	$\log_2 8 = 3$
$2^4 = 16$	$\leftrightarrow$	$\log_2 16 = 4$
$2^5 = 32$	$\leftrightarrow$	$\log_2 32 = 5$

Logaritmos são o oposto de exponenciais.

Neste livro, quando falamos sobre a notação Big O (explicada daqui a pouco), levamos em conta que log sempre significa $\log_2$. Quando você procura um elemento usando a pesquisa

simples, no pior dos casos, terá de analisar elemento por elemento, passando por todos. Se for uma lista de oito elementos, precisaria checar no máximo oito números. Na pesquisa binária, precisa verificar $\log n$ elementos para o pior dos casos. Para uma lista de oito elementos, $\log 8 == 3$, porque $2^3 == 8$. Então, para uma lista de oito números, precisaria passar por, no máximo, três tentativas. Para uma lista de 1.024 elementos, $\log 1.024 == 10$, porque $2^{10} == 1.024$. Logo, para uma lista de 1.024 números, precisaria verificar no máximo dez deles.

Nota

Falarei muito sobre logaritmos neste livro. Portanto você deve entender o conceito. Se não entender, a Khan Academy (khanacademy.org) tem um vídeo legal que esclarece muita coisa.

Nota

A pesquisa binária só funciona quando a sua lista está ordenada. Por exemplo, os nomes em uma agenda telefônica estão em ordem alfabética, então você pode utilizar a pesquisa binária para procurar um nome. O que aconteceria se a lista não estivesse ordenada?

Vamos ver como escrever a pesquisa binária em Python. O exemplo de código que utilizamos aqui usa arrays. Se não sabe como eles funcionam, não se preocupe; abordaremos isso no próximo capítulo. Você só precisa saber que pode armazenar uma sequência de elementos em uma linha de buckets consecutivos que se chama array. Os buckets são numerados a partir do 0: o primeiro bucket está na posição #0; o segundo, em #1; o terceiro, em #2, e assim por diante.

A função `pesquisa_binaria` pega um array ordenado e um item. Se o item está no array, a função retorna a sua posição. Dessa maneira, você é capaz de saber por qual ponto do array deve continuar procurando. No começo, o código do array segue assim:

```
baixo = 0
alto = len(lista) - 1
```



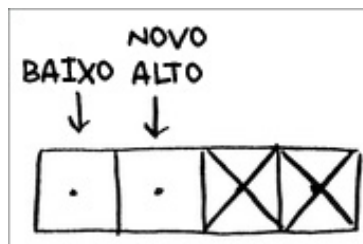
A cada tentativa, você testa para o elemento central.

```
meio = (baixo + alto) / 2 ❶
chute = lista[meio]
```

❶ meio será arredondado para baixo automaticamente pelo Python se (baixo + alto) não for um número par.

Se o chute for muito baixo, você atualizará a variável **baixo** proporcionalmente:

```
if chute < item:
    baixo = meio + 1
```



E se o chute for muito alto, você atualizará a variável **alto**. Aqui está o código completo:

```
def pesquisa_binaria(lista, item):
    baixo = 0 ❶
    alto = len(lista) - 1 ❷
    while baixo <= alto: ❸
        meio = (baixo + alto) / 2 ❹
        chute = lista[meio]
        if chute == item: ❺
            return meio
        if chute > item: ❻
```

```

        alto = meio - 1
    else: ❹
        baixo = meio + 1
    return None ❺

minha_lista = [1, 3, 5, 7, 9] ❻
print pesquisa_binaria(minha_lista, 3) # => 1 ❼
print pesquisa_binaria(minha_lista, -1) # => None ❶

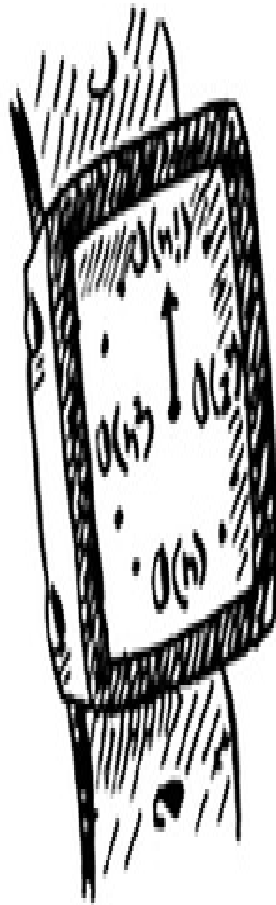
```

- ❶ baixo e alto acompanham a parte da lista que você está procurando.
- ❷ Enquanto ainda não conseguiu chegar a um único elemento...
- ❸ ... verifica o elemento central.
- ❹ Acha o item.
- ❺ O chute foi muito alto.
- ❻ O chute foi muito baixo.
- ❼ O item não existe.
- ❶ Vamos testá-lo!
- ❼ Lembre-se, as listas começam no 0. O próximo endereço tem índice 1.
- ❶ "None" significa nulo em Python. Ele indica que o item não foi encontrado.

EXERCÍCIOS

- 1.1 Suponha que você tenha uma lista com 128 nomes e esteja fazendo uma pesquisa binária. Qual seria o número máximo de etapas que você levaria para encontrar o nome desejado?
- 1.2 Suponha que você duplique o tamanho da lista. Qual seria o número máximo de etapas agora?

Tempo de execução



Sempre que falo sobre um algoritmo, falo sobre o seu tempo de execução. Geralmente, você escolhe o algoritmo mais eficiente – caso esteja tentando otimizar tempo e espaço.

Voltando à pesquisa simples, quanto tempo se otimiza utilizando-a? Bem, a primeira abordagem seria verificar número por número. Se fosse uma lista de 100 números, precisaríamos de 100 tentativas. Se fosse uma lista de 4 bilhões de números, precisaríamos de 4 bilhões de tentativas. Logo, o número máximo de tentativas é igual ao tamanho da lista. Isso é chamado de *tempo linear*.

A pesquisa binária é diferente. Se a lista tem 100 itens, precisa-se de, no máximo, sete tentativas. Se tem 4 bilhões, precisa-se de, no máximo, 32 tentativas. Poderoso, não? A pesquisa binária é executada com *tempo logarítmico*. A tabela a seguir resume as nossas descobertas até agora.

PESQUISA SIMPLES	PESQUISA BINÁRIA
100 ITENS ↓ 100 PALPITES	100 ITENS ↓ 7 PALPITES
4.000.000.000 ITENS ↓ 4.000.000.000 PALPITES	4.000.000.000 ITENS ↓ 32 PALPITES
$O(n)$ ↑ TEMPO DE EXECUÇÃO LINEAR	$O(\log n)$ ↑ TEMPO DE EXECUÇÃO LOGARÍTMICO

GRANDE ECONOMIA!

GRANDE ECONOMIA!

Tempo de execução para algoritmos de pesquisa.

Notação Big O

A notação *Big O* é uma notação especial que diz o quão rápido é um algoritmo. Mas quem liga para isso? Bem, acontece que você frequentemente utilizará o algoritmo que outra pessoa fez – e quando faz isso, é bom entender o quão rápido ou lento o algoritmo é. Nesta seção, explicarei como a notação Big O funciona e fornecerei uma lista com os tempos de execução mais comuns para os algoritmos.



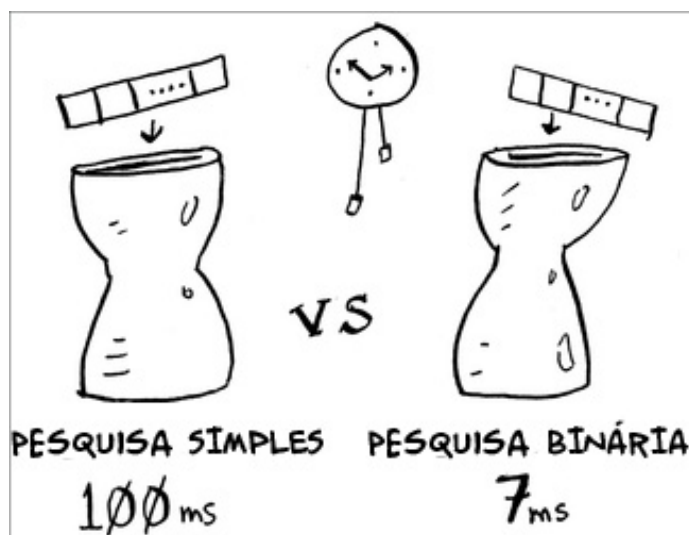
Tempo de execução dos algoritmos cresce a taxas diferentes

Bob está escrevendo um algoritmo para a NASA. O algoritmo dele entrará em ação quando o foguete estiver prestes a pousar na lua, e ele o ajudará a calcular o local de pouso.

Este é um exemplo de como o tempo de execução de dois algoritmos pode crescer a taxas diferentes. Bob está tentando decidir entre a pesquisa simples e a pesquisa binária. O algoritmo precisa ser tão rápido quanto correto. Por um lado, a pesquisa binária é mais rápida, o que é bom, pois Bob tem apenas *10 segundos* para descobrir onde pousar, ou o foguete sairá de seu curso. Por outro lado, é mais fácil escrever a pesquisa simples, o que gera um risco menor de erros. Bob não quer *mesmo* erros no seu código! Para ser ainda mais cuidadoso, Bob decide cronometrar ambos os algoritmos com uma lista de 100 elementos.

Vamos presumir que leva-se 1 milissegundo para verificar um elemento.

Com a pesquisa simples, Bob precisa verificar 100 elementos, então a busca leva 10 ms para rodar. Em contrapartida, ele precisa verificar apenas sete elementos na pesquisa binária ($\log_2 100$ é aproximadamente 7), logo, a pesquisa binária leva 7 ms para ser executada. Porém, realisticamente falando, a lista provavelmente terá em torno de 1 bilhão de elementos. Se a lista tiver esse número, quanto tempo a pesquisa simples levará para ser executada? E a pesquisa binária? Tenha certeza de que sabe a resposta para essa pergunta antes de continuar lendo.



Tempo de execução para pesquisa simples vs. pesquisa binária para uma lista de 100 elementos.

Bob executa a pesquisa binária com 1 bilhão de elementos e leva 30 ms ($\log_2 1.000.000.000$ é aproximadamente 30). “30 ms!” – ele pensa. “A pesquisa binária é quase 15 vezes mais rápida do que a pesquisa simples, porque a pesquisa simples levou 100 ms para uma lista de 100 elementos e a pesquisa binária levou só 7 ms. Logo, a pesquisa simples levará $30 \times 15 = 450$ ms, certo? Bem abaixo do meu limite de 10 segundos.” Bob decide utilizar a pesquisa simples. Ele fez a escolha certa?

Não. Bob está errado. Muito errado. O tempo de execução para a pesquisa simples para 1 bilhão de itens é 1 bilhão ms, ou seja, 11 dias! O problema é que o tempo de execução da pesquisa simples e da pesquisa binária *crece com taxas diferentes*.

	PESQUISA SIMPLES	PESQUISA BINÁRIA
100 ELEMENTOS	100ms	7ms
10000 ELEMENTOS	10 segundos	14 ms
1,000,000,000 ELEMENTOS	11 dias	32 ms

Tempos de execução crescem com velocidades diferentes!

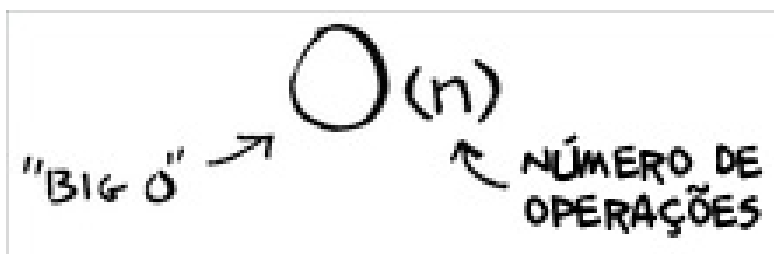
Sendo assim, conforme o número de itens cresce, a pesquisa binária aumenta só um pouco o seu tempo de execução. Já a pesquisa simples leva *muito* tempo a mais. Logo, conforme a lista de números cresce, a pesquisa binária se torna *muito* mais rápida do que a pesquisa simples. Bob pensou que a pesquisa binária fosse 15 vezes mais rápida que a pesquisa simples, mas isso está incorreto. Se a lista tem 1 bilhão de itens, o tempo de execução é aproximadamente 33 milhões de vezes mais rápido. Por isso, não basta saber quanto tempo um algoritmo leva para ser executado – você precisa saber se o tempo de execução aumenta conforme a lista aumenta. É aí que a notação Big O entra.

A notação Big O informa o quão rápido é um algoritmo. Por exemplo, imagine que você tem uma lista de tamanho n . O tempo de execução na notação Big O é $O(n)$. Onde estão os segundos? Eles não existem – a notação Big O não fornece o tempo em segundos. *A notação Big O permite que você compare o número de operações.* Ela informa o quão rapidamente um algoritmo cresce.



Temos outro exemplo disso. A pesquisa binária precisa de $\log n$ operações

para verificar uma lista de tamanho n . Qual é o tempo de execução na notação Big O? É $O(\log n)$. De maneira geral, a notação Big O é escrita da seguinte forma:



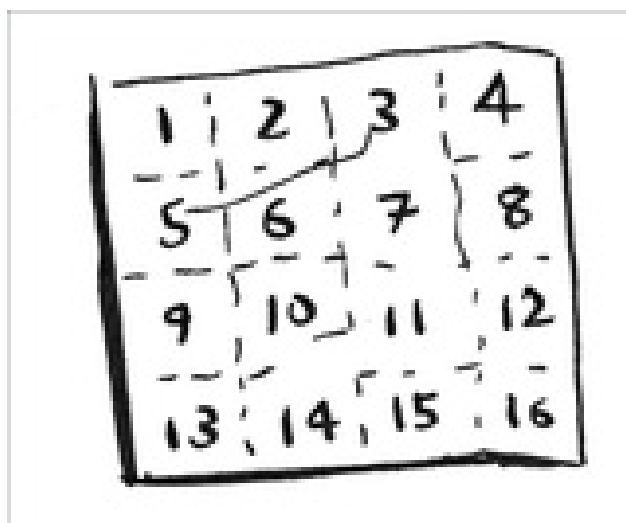
O formato da notação Big O.

Isso fornece o número de operações que um algoritmo realiza. É chamado de notação Big O porque coloca-se um “grande O” na frente do número de operações (parece piada, mas é verdade!).

Agora, vamos ver alguns exemplos. Veja se consegue descobrir o tempo de execução para esses algoritmos.

Vendo diferentes tempos de execução Big O

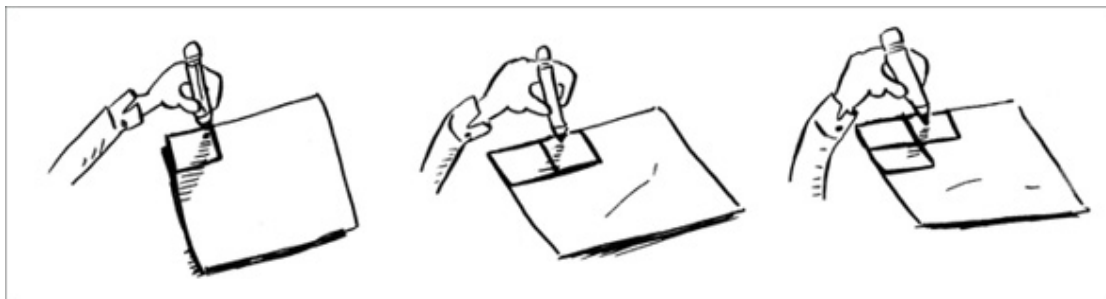
Aqui segue um exemplo prático que você pode reproduzir em casa com um pedaço de papel e um lápis. Suponha que você tenha que desenhar uma grade com 16 divisões.



Qual é um bom algoritmo para desenhar essa grade?

Algoritmo 1

Uma forma de desenhar essa grade de 16 divisões é desenhar uma divisão de cada vez. Lembre-se, a notação Big O conta o número de operações. Nesse exemplo, desenhar uma divisão é uma operação. Você precisa desenhar 16 divisões. Quantas operações você terá de fazer se desenhar uma divisão por vez?

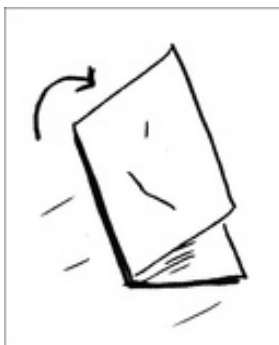


Desenhando a grade executando uma divisão por vez.

É necessário passar por 16 etapas para desenhar 16 divisões. Qual é o tempo de execução desse algoritmo?

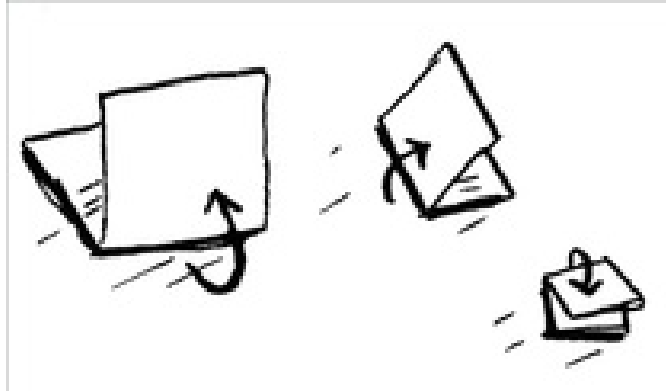
Algoritmo 2

Tente agora este algoritmo. Dobre o papel.

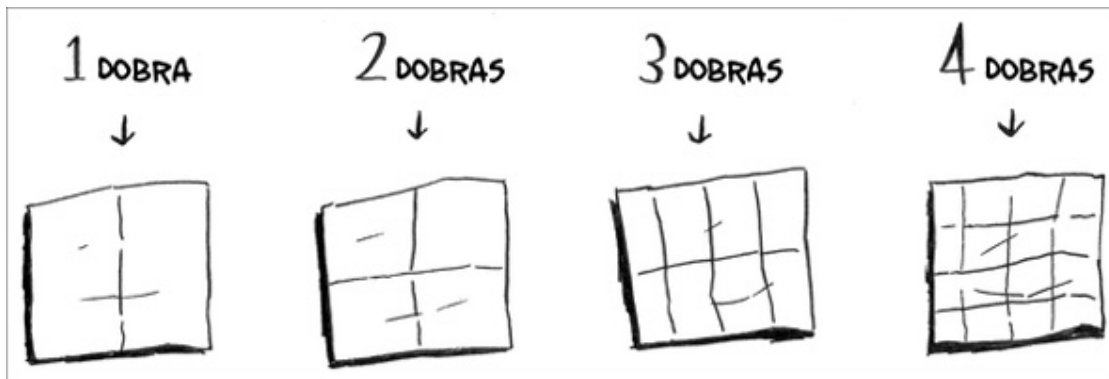


Neste exemplo, dobrar o papel uma vez é uma operação. Você fez duas divisões com essa operação!

Dobre o papel de novo, de novo e de novo.



Desdobre depois de quatro dobras e você terá uma bela grade! A cada dobra, o número de divisões duplica. Você fez 16 divisões com quatro operações!



Desenhando uma grade com quatro dobras.

Você pode “desenhar” duas vezes mais divisões a cada dobra, logo, você pode desenhar 16 divisões em quatro etapas. Qual é o tempo de execução para esse algoritmo? Encontre o tempo de execução dos dois algoritmos antes de seguir adiante.

Respostas: O Algoritmo 1 tem tempo de execução $O(n)$ e o algoritmo 2 tem tempo de execução $O(\log n)$.

A notação Big O estabelece o tempo de execução para a pior hipótese

Suponha que você utiliza uma pesquisa simples para procurar o nome de uma pessoa em uma agenda telefônica. Você sabe que a pesquisa simples tem tempo de execução $O(n)$, o que significa que na pior das hipóteses terá verificado cada nome da agenda telefônica. Nesse caso, você está procurando uma pessoa chamada Adit. Essa pessoa é a primeira de sua lista. Logo, não teve de passar por todos os nomes – você a encontrou na primeira

tentativa. Esse algoritmo levou o tempo de execução $O(n)$? Ou levou $O(1)$ porque encontrou o que queria na primeira tentativa?

A pesquisa simples ainda assim tem tempo de execução $O(n)$. Nesse caso, você encontrou o que queria instantaneamente. Essa é a melhor das hipóteses. A notação Big O leva em conta a *pior das hipóteses*. Então pode-se dizer que, para o *pior caso*, você analisou cada item da lista. Esse é o tempo $O(n)$. É uma garantia – você sabe, com certeza, que a pesquisa simples nunca terá tempo de execução mais lento do que $O(n)$.

Nota

Além do tempo de execução para o pior dos casos, é importante analisar o tempo de execução para o “caso médio”. O pior caso e o caso médio serão discutidos no Capítulo 4.

Alguns exemplos comuns de tempo de execução Big O

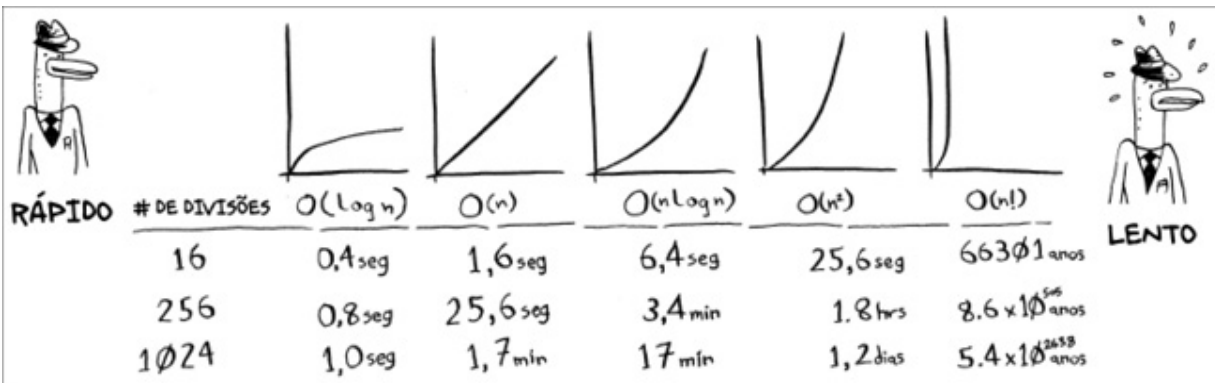
Aqui temos cinco tempos de execução Big O que você encontrará bastante, ordenados do mais rápido para o mais lento.

- $O(\log n)$, também conhecido como *tempo logarítmico*. Exemplo: pesquisa binária.
- $O(n)$, conhecido como *tempo linear*. Exemplo: pesquisa simples.
- $O(n * \log n)$. Exemplo: um algoritmo rápido de ordenação, como a ordenação quicksort (explicada no Capítulo 4).
- $O(n^2)$. Exemplo: um algoritmo lento de ordenação, como a ordenação por seleção (explicada no Capítulo 2).
- $O(n!)$. Exemplo: um algoritmo bastante lento, como o do caixeiro-viajante (explicado a seguir!).

Suponha que você esteja desenhando novamente a grade de 16 divisões e você possa escolher cinco algoritmos diferentes para fazer isso. Se escolher o primeiro algoritmo, levará um tempo de execução de $O(\log n)$ para desenhar a grade. Você pode fazer dez operações por segundo. Com o tempo de execução $O(\log n)$, você levará quatro operações para desenhar uma grade com 16 divisões ($\log 16$ é 4). Logo, levará 0,4 segundos para desenhar a grade. E se tiver que desenhar 1.024 divisões? Levará $1.024 = 10$ operações, ou um segundo para desenhar uma grade de 1.024 divisões. Estes números são para o primeiro algoritmo.

O segundo algoritmo é mais lento: ele tem tempo de execução $O(n)$. Levará 16 operações para desenhar 16 divisões e levará 1.024 operações para desenhar 1.024 divisões. Quanto tempo isso leva em segundos?

Aqui está quanto tempo levaria para desenhar a grade com os algoritmos restantes, do mais rápido ao mais lento:



Existem outros tempos de execução, mas esses são os cinco mais comuns.

Isso é uma simplificação. Na realidade, você não pode converter um tempo de execução na notação Big O para um número de operações, mas a aproximação é boa o suficiente por enquanto. Voltaremos a falar da notação Big O no Capítulo 4, depois de ter aprendido um pouco mais sobre algoritmos. Por enquanto, os principais pontos são os seguintes:

- A rapidez de um algoritmo não é medida em segundos, mas pelo crescimento do número de operações.
- Em vez disso, discutimos sobre o quão rapidamente o tempo de execução de um algoritmo aumenta conforme o número de elementos aumenta.
- O tempo de execução em algoritmos é expresso na notação Big O.
- $O(\log n)$ é mais rápido do que $O(n)$, e $O(\log n)$ fica ainda mais rápido conforme a lista aumenta.

EXERCÍCIOS

Forneça o tempo de execução para cada um dos casos a seguir em termos da notação Big O.

1.3 Você tem um nome e deseja encontrar o número de telefone para esse nome em uma agenda telefônica.

1.4 Você tem um número de telefone e deseja encontrar o dono dele em uma

agenda telefônica. (Dica: Deve procurar pela agenda inteira!)

1.5 Você quer ler o número de cada pessoa da agenda telefônica.

1.6 Você quer ler os números apenas dos nomes que começam com A. (Isso é complicado! Esse algoritmo envolve conceitos que são abordados mais profundamente no Capítulo 4. Leia a resposta – você ficará surpreso!)

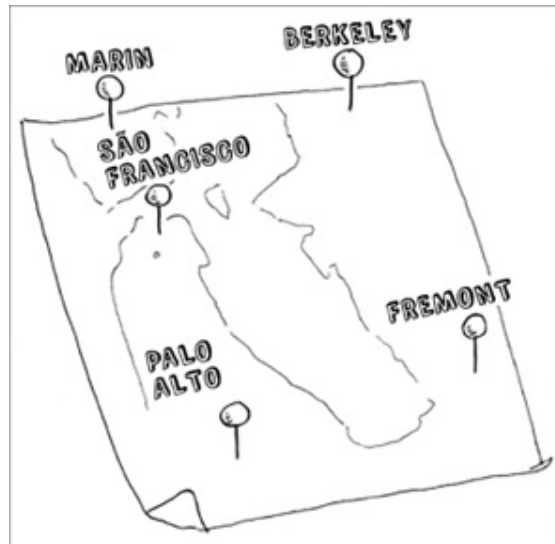
O caixeiro-viajante

Você pode ter lido a última seção e pensado: “De maneira alguma vou executar um algoritmo que tem tempo de execução $O(n!)$.” Bem, deixe-me tentar provar o contrário! Aqui está um exemplo de um algoritmo com um tempo de execução muito ruim. Ele é um problema famoso da ciência da computação, pois seu crescimento é apavorante e algumas pessoas muito inteligentes acreditam que ele possa ser melhorado. Esse algoritmo é chamado de “o problema do *caixeiro-viajante*”.

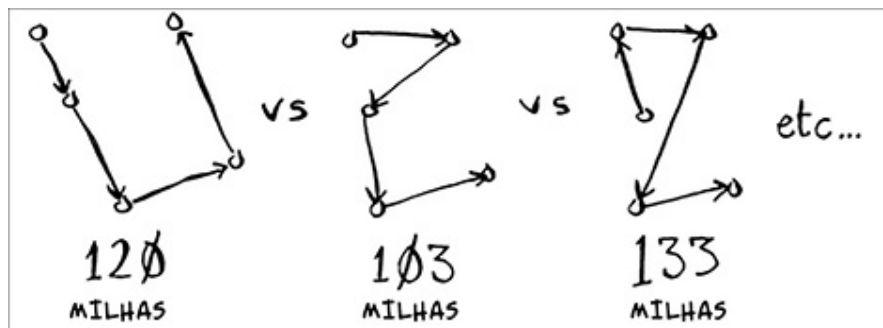


Você tem um caixeiro-viajante.

O caixeiro precisa ir a cinco cidades.



O caixeiro, o qual chamarei de Opus, quer passar por todas as cidades percorrendo uma distância mínima. Podemos enxergar o problema da seguinte forma: analisar cada ordem possível de cidades para visitar.



Ele soma a distância total e escolhe o caminho de menor distância. Existem 120 permutações para cinco cidades, logo, precisa-se de 120 operações para resolver o problema de cinco cidades. Para seis cidades, precisa-se de 720 operações (ou 720 permutações). Para sete cidades são necessárias 5.050 operações!

CIDADES	OPERAÇÕES
6	720
7	5040
8	40320
...	...
15	1,307,674,368,000
...	...
30	2,652,528,598,121,910,586,330,848,000,000

O número de operações aumenta drasticamente.

De maneira geral, para n itens, é necessário $n!$ (fatorial de n) operações para chegar a um resultado. Então, este é o tempo de execução $O(n!)$ ou o *tempo fatorial*. Esse algoritmo consome muitas operações, exceto para casos envolvendo números pequenos. No entanto, uma vez que lidamos com mais de 100 cidades, é impossível calcular a resposta em função do tempo – o sol entrará em colapso antes.

Esse é um algoritmo terrível! O *pus* deveria usar outro, não? Mas ele não pode. Esse é um problema sem solução. Não existe um algoritmo mais rápido para esse problema, e as pessoas mais inteligentes acreditam ser *impossível* melhorá-lo. O melhor que se pode fazer é chegar a uma solução aproximada. Veja o Capítulo 10 para saber mais sobre isso.

Uma observação final: se você é um leitor avançado, leia sobre árvores binárias de busca. No capítulo seguinte, há uma breve descrição do assunto.

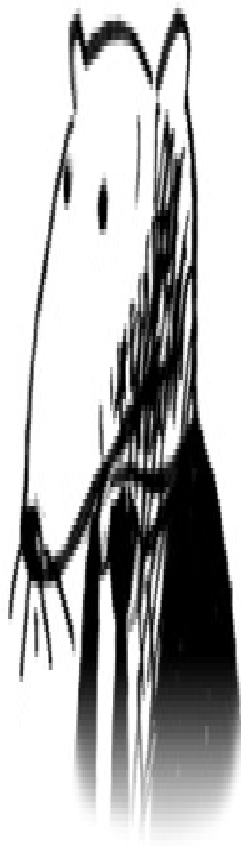
Recapitulando

- A pesquisa binária é muito mais rápida do que a pesquisa simples.
- $O(\log n)$ é mais rápido do que $O(n)$, e $O(\log n)$ fica ainda mais rápido conforme os elementos da lista aumentam.
- A rapidez de um algoritmo não é medida em segundos.
- O tempo de execução de um algoritmo é medido por meio de seu *crescimento*.

- O tempo de execução dos algoritmos é expresso na notação Big O.

2

Ordenação por seleção



Neste capítulo

- Você conhecerá arrays e listas encadeadas – dois tipos de estrutura básica. Eles estão por todo lugar. Você já teve acesso aos arrays no capítulo 1 e continuará se utilizando deles por praticamente todos os capítulos. Arrays são fundamentais, então preste atenção! Porém algumas vezes é melhor usar a lista encadeada em vez do array. Este

capítulo explana os prós e os contras de ambas as estruturas para que possa decidir qual é a ideal para o seu algoritmo.

- Você aprenderá a fazer o seu primeiro algoritmo de ordenação. Muitos algoritmos só funcionam se os dados estiverem ordenados. Lembra-se da pesquisa binária? Você só pode executá-la se os elementos de sua lista estiverem ordenados. Este capítulo lhe apresentará a ordenação por seleção. A maioria das linguagens de programação contém nativamente os algoritmos de seleção, então raramente terá de escrever a sua própria versão a partir do zero. No entanto a ordenação por seleção é um trampolim para o quicksort, que abordarei no próximo capítulo. O quicksort é um algoritmo importante e será compreendido mais facilmente se você já conhecer algum tipo de algoritmo de ordenação.

O que você precisa saber

Para entender a análise de desempenho deste capítulo, você precisa conhecer a notação Big O e logaritmos. Se não conhece, sugiro que leia o Capítulo 1. A notação Big O é utilizada em todo este livro.

Como funciona a memória

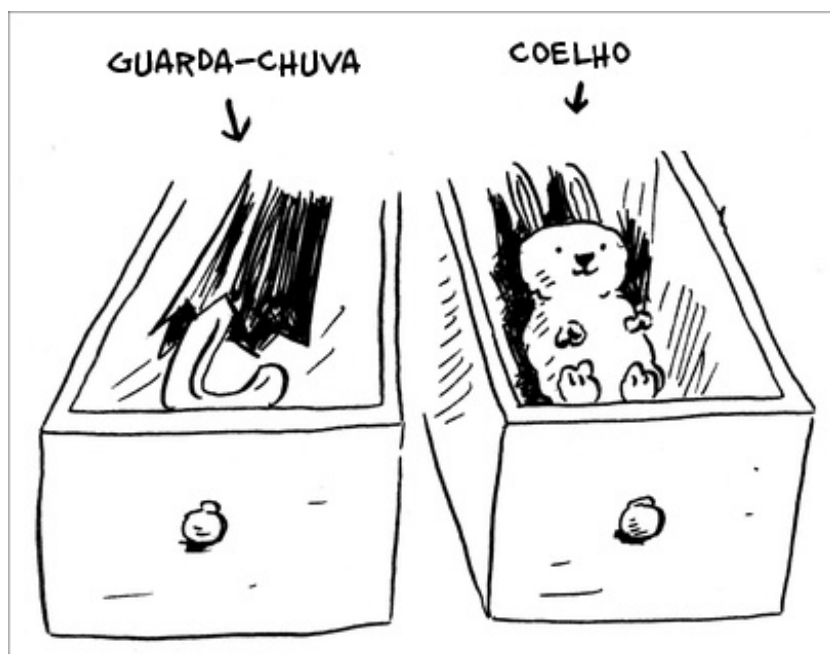
Imagine que você vai a um show e precisa guardar as suas coisas na chapelaria. Algumas gavetas estão disponíveis.



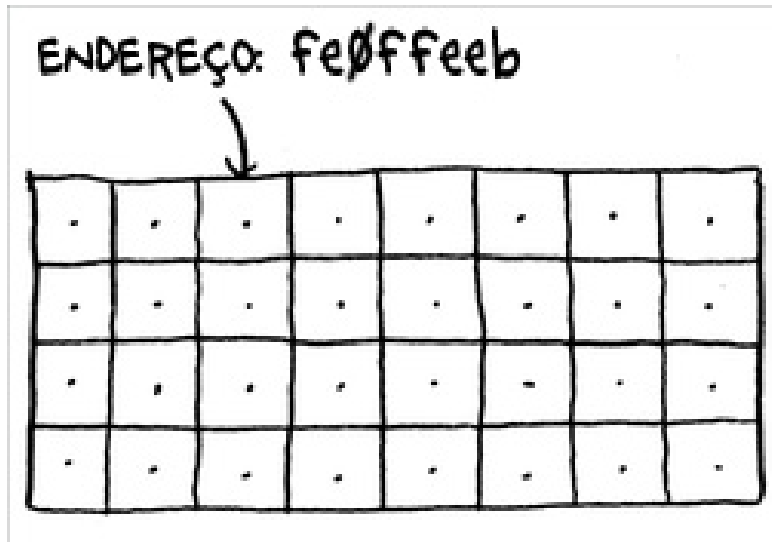
Cada gaveta pode guardar um elemento. Você deseja guardar duas coisas, então pede duas gavetas.



Você guardou as suas duas coisas aqui.



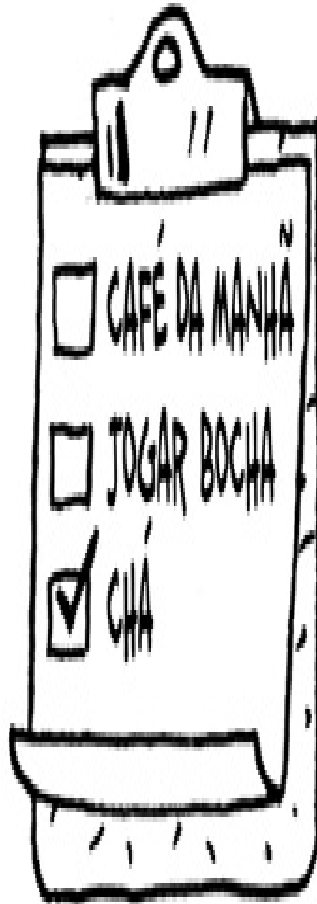
Você está pronto para o show! É mais ou menos assim que a memória do seu computador funciona. O computador se parece com um grande conjunto de gavetas, e cada gaveta tem seu endereço.



fe0ffeeb é o endereço de um slot na memória.

Cada vez que quer armazenar um item na memória, você pede ao computador um pouco de espaço e ele te dá um endereço no qual você pode armazenar o seu item. Se quiser armazenar múltiplos itens, existem duas maneiras para fazer isso: arrays e listas. Falarei sobre arrays e listas depois, bem como sobre os prós e contras de cada um. Não existe apenas uma maneira correta para armazenar itens em cada um dos casos, então é importante saber as diferenças.

Arrays e listas encadeadas



Algumas vezes, você precisa armazenar uma lista de elementos na memória. Suponha que você esteja escrevendo um aplicativo para gerenciar os seus afazeres. É necessário armazenar os seus afazeres como uma lista na memória.

Você deve usar um array ou uma lista encadeada? Vamos armazenar os afazeres primeiro em um array, pois assim a compreensão fica mais fácil. Usar um array significa que todas as suas tarefas estão armazenadas contiguamente (uma ao lado da outra) na memória.



Agora, suponha que você queira adicionar mais uma tarefa. No entanto a próxima gaveta está ocupada por coisas de outra pessoa!



É como se você estivesse indo ao cinema com os seus amigos e encontrasse um lugar para sentar, mas outro amigo se juntasse a vocês e não houvesse lugar para ele. Vocês todos precisariam se mover e encontrar um lugar onde todos coubessem. Neste caso, você precisaria solicitar ao computador uma área de memória em que coubessem todas as suas tarefas. Então você as moveria para lá.

Se outro amigo aparecesse, vocês ficariam sem lugar novamente – e todos precisariam se mover uma segunda vez! Que incômodo. Da mesma forma, adicionar novos itens a um array será muito lento. Uma maneira fácil de resolver isso é “reservando lugares”: mesmo que você tenha três itens na sua lista de tarefas, você pode solicitar ao computador dez espaços, só por via das dúvidas. Então, você pode adicionar dez itens a sua lista sem precisar mover nada. Isto é uma boa maneira de contornar o problema, mas você

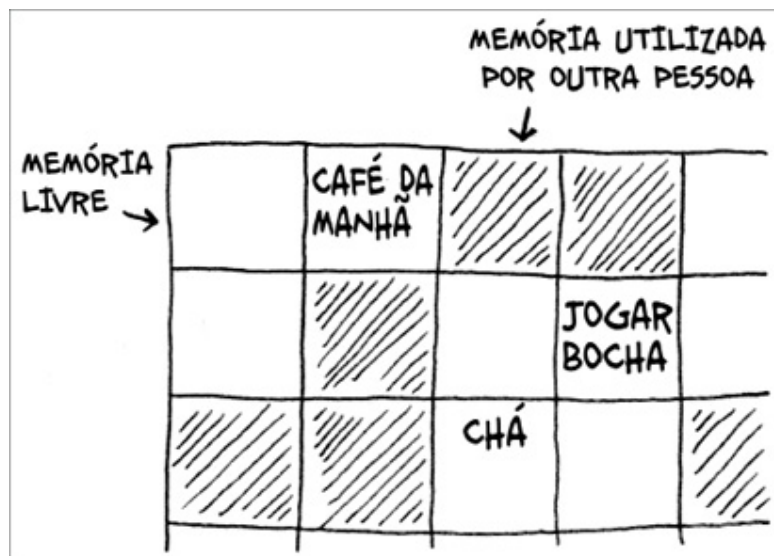
precisa ficar atento às desvantagens:

- Você pode não precisar dos espaços extras que reservou; então a memória será desperdiçada. Você não está utilizando a memória, mas ninguém mais pode usá-la também.
- Você pode precisar adicionar mais de dez itens a sua lista de tarefas, então você terá de mover seus itens de qualquer maneira.

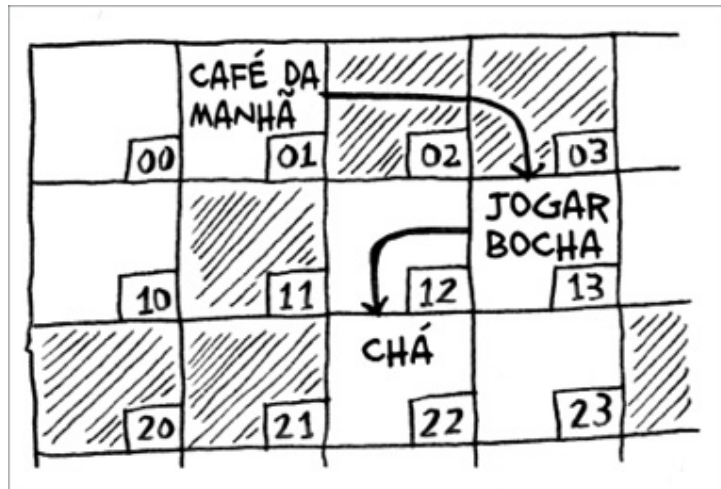
Embora seja uma boa forma de contornar o problema, não é uma solução perfeita. Listas encadeadas resolvem este problema de adição de itens.

Listas encadeadas

Com as listas encadeadas, seus itens podem estar em qualquer lugar da memória.



Cada item armazena o endereço do próximo item da lista. Um monte de endereços aleatórios de memória estão ligados.



Endereços de memória ligados.

É como uma caça ao tesouro. Você vai ao primeiro endereço e ele diz “o próximo item pode ser encontrado no endereço 123”. Então vai ao endereço 123 e ele diz “O próximo item pode ser encontrado no endereço 847”, e assim por diante. Adicionar um item a uma lista encadeada é fácil: você o coloca em qualquer lugar da memória e armazena o endereço do item anterior.

Com as listas encadeadas você nunca precisa mover os seus itens; também evita outro problema. Digamos que você vá a um cinema famoso com os seus amigos. Vocês seis estão tentando procurar um lugar para sentar, mas o cinema está cheio. Não há seis lugares juntos. Bem, algumas vezes isso acontece com arrays. Imagine que está tentando encontrar 10.000 slots para um array. Sua memória tem 10.000 slots, mas eles não estão juntos. Você não consegue arrumar um lugar para o seu array! Usar uma lista encadeada seria como dizer “vamos nos dividir e assistir ao filme”. Se existir espaço na memória, você terá espaço para a sua lista encadeada.

Se as listas encadeadas são muito melhores para inserções, para que servem os arrays?

Arrays

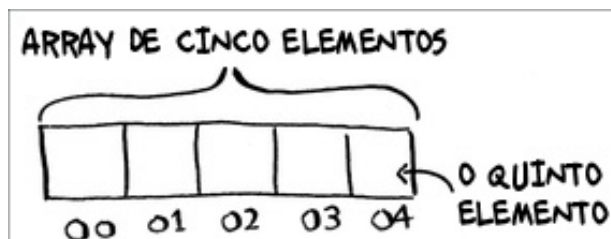


Os websites que apresentam listas “top 10” usam uma tática trapaceira para conseguir mais visualizações. Em vez de mostrarem a lista em uma única página, eles colocam um item em cada página e fazem você clicar em “próximo” para ler o item seguinte. Por exemplo, “Os 10 melhores vilões da TV” não estarão listados em uma única página, em vez disso, você começará pelo #10 (Newman) e seguirá clicando em “próximo” até chegar em #1 (Gustavo Fring). Esta técnica fornece aos sites dez páginas inteiras para incluir anúncios, mas fica chato ficar clicando em “próximo” nove vezes até chegar ao número 1. Seria muito melhor se a lista estivesse em uma única página e você pudesse clicar no nome de cada vilão para saber mais.

Listas encadeadas têm um problema similar. Suponha que você queira ler o último item de uma lista encadeada. Você não pode fazer isso porque não sabe o endereço dele. Em vez disso, precisa ir ao item #1 para pegar o endereço do item #2. Então, é necessário ir ao item #2 para encontrar o endereço do item #3, e assim por diante, até conseguir o endereço do último

item. Listas encadeadas são ótimas se você quiser ler todos os itens, um de cada vez: você pode ler um item, seguir para o endereço do próximo item e fazer isso até o fim da lista. Mas se você quiser pular de um item para outro, as listas encadeadas são terríveis.

Com arrays é diferente. Você sabe o endereço de cada item. Por exemplo, suponha que seu array tenha cinco itens e que você saiba que o primeiro está no endereço 00. Qual é o endereço do item #5?



A matemática lhe dá a resposta: está no endereço 04. Arrays são ótimos se você deseja ler elementos aleatórios, pois pode encontrar qualquer elemento instantaneamente em um array. Na lista encadeada, os elementos não estão próximos uns dos outros, então você não pode calcular instantaneamente a posição de um elemento na memória – precisa ir ao primeiro elemento para encontrar o endereço do segundo, então ir ao segundo elemento para encontrar o endereço do terceiro e seguir fazendo isso até chegar ao elemento que deseja.

Terminologia

Os elementos em um array são numerados. Essa numeração começa no 0, não no 1. Neste array, por exemplo, o número 20 está na posição 1.

10	20	30	40
0	1	2	3

O número 10 está na posição 0. Isso geralmente confunde novos programadores. Começar no 0 simplifica todos os tipos de array na programação, logo, os programadores não podem fugir disso. Quase todas as linguagens de programação começarão os arrays numerando o primeiro elemento como 0. Logo você se acostuma!

A posição de um elemento é chamada de *índice*. Portanto, em vez de dizer “o

número 20 está na *posição* 1”, a terminologia correta seria dizer “o número 20 está no *índice* 1”. Usarei índice para falar de *posição* neste livro.

Aqui está o tempo de execução para operações comuns de arrays e listas.

	ARRAYS	LISTAS
LEITURA	$O(1)$	$O(n)$
INSERÇÃO	$O(n)$	$O(1)$

$O(n)$ = TEMPO DE EXECUÇÃO LINEAR
 $O(1)$ = TEMPO DE EXECUÇÃO CONSTANTE

Pergunta: Por que é necessário tempo de execução $O(n)$ para inserir um elemento em um array? Suponha que você queira inserir um elemento no começo de um array. Como faria isso? Quanto tempo levaria? Encontre as respostas no final desta seção!

EXERCÍCIOS

2.1 Suponha que você esteja criando um aplicativo para acompanhar as suas finanças.

1. COMPRAS
2. CINEMA
3. MENSALIDADE
DO SFBC

Todos os dias você anotará tudo o que gastou e onde gastou. No final do mês, você deverá revisar os seus gastos e resumir o quanto gastou. Logo, você terá um monte de inserções e poucas leituras. Você deverá usar um array ou uma lista para implementar este aplicativo?

Inserindo algo no meio da lista

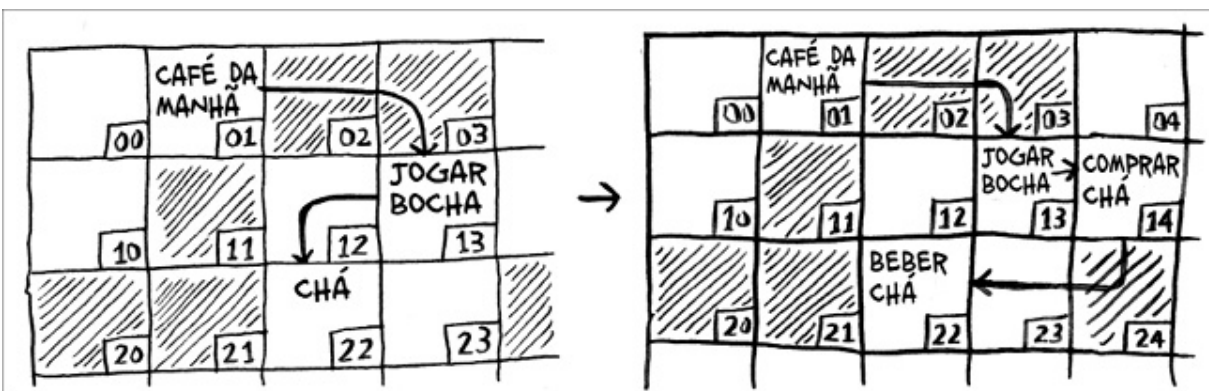
Imagine que você queira que a sua lista de tarefas se pareça mais com um calendário. Antes, você adicionava os itens ao final da lista. Agora, quer

adicionar suas tarefas na ordem em que elas devem ser realizadas.

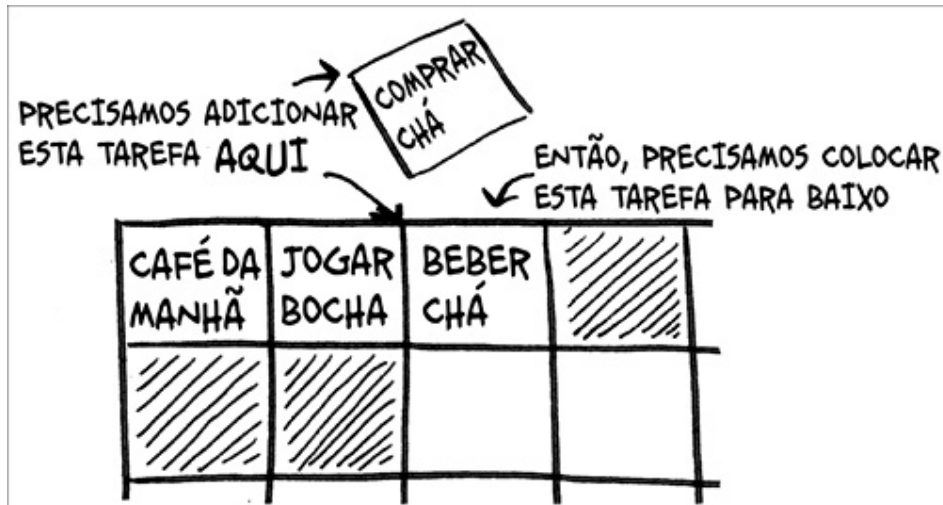


Lista desordenada.

O que seria melhor se você quisesse inserir elementos no meio de uma lista: arrays ou listas encadeadas? Usando listas encadeadas, basta mudar o endereço para o qual o elemento anterior está apontando.



Já para arrays, você deve mover todos os itens que estão abaixo do endereço de inserção.



Se não houver espaço, pode ser necessário mover tudo para um novo local! Por isso, listas encadeadas são melhores caso você queira inserir um elemento no meio de uma lista.

Deleções

E se você quiser deletar um elemento? Novamente, é mais fácil fazer isso usando listas encadeadas, pois é necessário mudar apenas o endereço para o qual o elemento anterior está apontando. Com arrays, tudo precisa ser movido quando um elemento é eliminado.

Ao contrário do que ocorre com as inserções, a eliminação de elementos sempre funcionará. A inserção poderá falhar quando não houver espaço suficiente na memória.

Aqui estão os tempos de execução para as operações mais comuns em arrays e listas encadeadas.

	ARRAYS	LISTAS
LEITURA	$O(1)$	$O(n)$
INSERÇÃO	$O(n)$	$O(1)$
ELIMINAÇÃO	$O(n)$	$O(1)$

Vale a pena mencionar que inserções e eliminações terão tempo de execução $O(1)$ somente se você puder acessar instantaneamente o elemento a ser

deletado. É uma prática comum acompanhar o primeiro e o último item de uma lista encadeada para que o tempo de execução para deletá-los seja $O(1)$.

O que é mais usado: arrays ou listas? Obviamente, isso depende do caso em que se aplicam. Entretanto os arrays são mais comuns porque permitem acesso aleatório. Existem dois tipos de acesso: o *aleatório* e o *sequencial*. O sequencial significa ler os elementos, um por um, começando pelo primeiro. Listas encadeadas só podem lidar com acesso sequencial. Se você quiser ler o décimo elemento de uma lista encadeada, primeiro precisará ler os nove elementos anteriores para chegar ao endereço do décimo elemento. O aleatório permite que você pule direto para o décimo elemento. Muitos casos requerem o acesso aleatório, o que faz os arrays serem mais utilizados. Arrays e listas são usados para implementar outras estruturas de dados (isso será explicado mais adiante).

EXERCÍCIOS

2.2 Suponha que você esteja criando um aplicativo para anotar os pedidos dos clientes em um restaurante. Seu aplicativo precisa de uma lista de pedidos. Os garçons adicionam os pedidos a essa lista e os chefes retiram os pedidos da lista. Funciona como uma fila. Os garçons colocam os pedidos no final da fila e os chefes retiram os pedidos do começo dela para cozinhá-los.

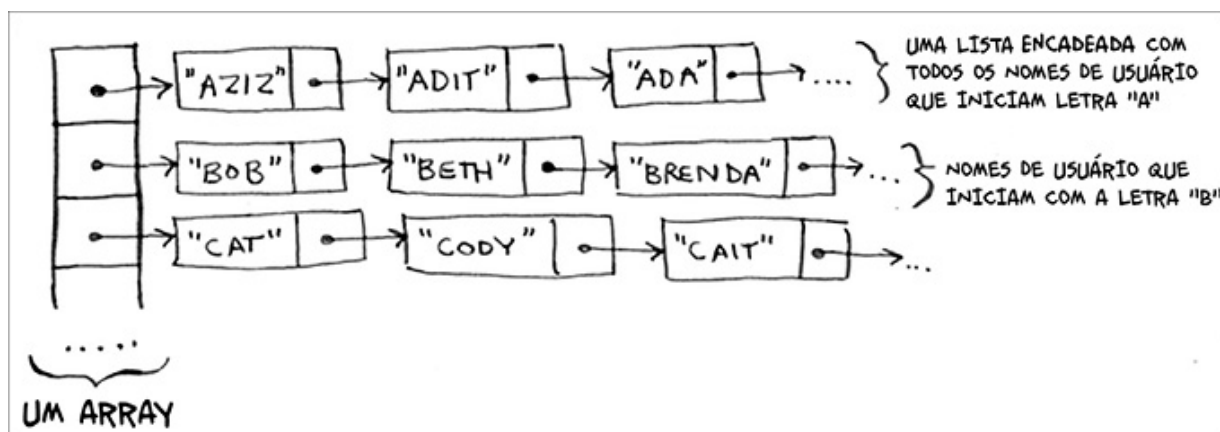


Você usaria um array ou uma lista encadeada para implementar essa lista? (Dica: listas encadeadas são boas para inserções/eliminações e arrays são bons para acesso aleatório. O que fazer neste caso?)

2.3 Vamos analisar um experimento. Imagine que o Facebook guarda uma

lista de usuários. Quando alguém tenta acessar o Facebook, uma busca é feita pelo nome de usuário. Se o nome da pessoa está na lista, ela pode continuar o acesso. As pessoas acessam o Facebook com muita frequência, então existem muitas buscas nessa lista. Presuma que o Facebook usa a pesquisa binária para procurar um nome na lista. A pesquisa binária requer acesso aleatório – você precisa ser capaz de acessar o meio da lista de nomes instantaneamente. Sabendo disso, você implementaria essa lista como um array ou uma lista encadeada?

- 2.4** As pessoas se inscrevem no Facebook com muita frequência também. Suponha que você decida usar um array para armazenar a lista de usuários. Quais as desvantagens de um array em relação às inserções? Em particular, imagine que você está usando a pesquisa binária para buscar os logins. O que acontece quando você adiciona novos usuários em um array?
- 2.5** Na verdade, o Facebook não usa nem arrays nem listas encadeadas para armazenar informações. Vamos considerar uma estrutura de dados híbrida: um array de listas encadeadas. Você tem um array com 26 slots. Cada slot aponta para uma lista encadeada. Por exemplo, o primeiro slot do array aponta para uma lista encadeada que contém os usuários que começam com a letra A. O segundo slot aponta para a lista encadeada que contém os usuários que começam com a letra B, e assim por diante.

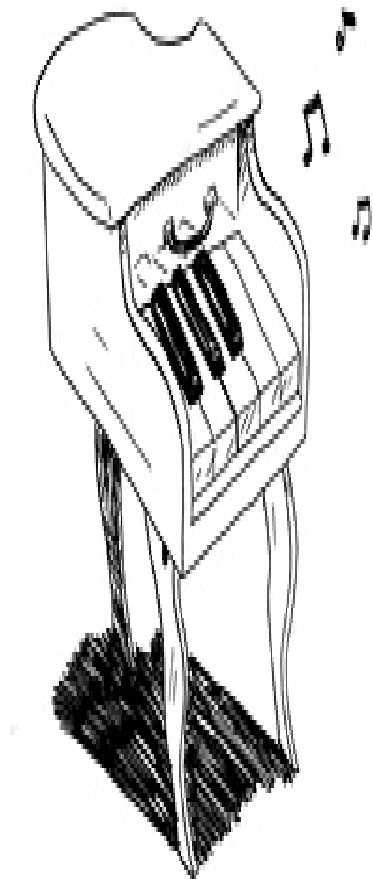


Suponha que o Adit B se inscreva no Facebook e você queira adicioná-lo à lista. Você vai ao slot 1 do array, a seguir para a lista encadeada do slot 1, e adiciona Adit B no final. Agora, suponha que você queira procurar o Zakhir H. Você vai ao slot 26, que aponta para a lista encadeada de todos

os nomes começados em Z. Então, procura pela lista até encontrar o Zakhir H.

Compare esta estrutura híbrida com arrays e listas encadeadas. É mais lento ou mais rápido fazer inserções e eliminações nesse caso? Você não precisa responder dando o tempo de execução Big(O), apenas diga se a nova estrutura de dados é mais rápida ou mais lenta do que os arrays e as listas encadeadas.

Ordenação por seleção



Vamos juntar tudo aprendido até aqui para você conhecer o seu segundo algoritmo: a ordenação por seleção. Para seguir nesta seção, você precisa ter compreendido arrays e listas, bem com a notação Big O.

Suponha que você tenha um monte de músicas no seu computador. Para cada artista, você tem um contador de plays.

~ ♪ ~	CONTADOR DE PLAYS
RADIOHEAD	156
KISHORE KUMAR	141
THE BLACK KEYS	35
NEUTRAL MILK HOTEL	94
BECK	88
THE STROKES	61
WILCO	111

Você quer ordenar uma lista de artistas, do artista mais tocado para o menos tocado, para que possa categorizar os seus artistas favoritos. Como pode fazer isso? Uma maneira seria pegar o artista mais tocado da lista de músicas e adicioná-lo a uma nova lista.

~ ♪ ~	CONTADOR DE PLAYS		♫ SORTED ♪	CONTADOR DE PLAYS
RADIOHEAD	156		RADIOHEAD	156
KISHORE KUMAR	141			
THE BLACK KEYS	35			
NEUTRAL MILK HOTEL	94			
BECK	88			
THE STROKES	61			
WILCO	111			

1. RADIOHEAD É O ARTISTA MAIS TOCADO...

2. ADICIONE-O EM UMA NOVA LISTA

Faça isso de novo para encontrar o próximo artista mais tocado.

~♪~	CONTADOR DE PLAYS		♪SORTED♪	CONTADOR DE PLAYS
			RADIOHEAD	156
KISHORE KUMAR	141		KISHORE KUMAR	141
THE BLACK KEYS	35	→		
NEUTRAL MILK HOTEL	94			
BECK	88			
THE STROKES	61			
WILCO	111			

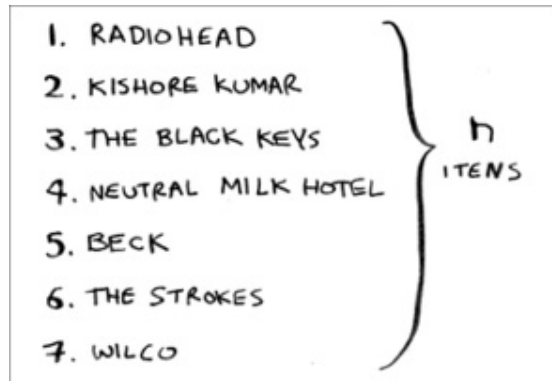
1. KISHORE KUMAR É O PRÓXIMO ARTISTA MAIS TOCADO

2. PORTANTO, ELE É O PRÓXIMO ARTISTA ADICIONADO À NOVA LISTA

Continue fazendo isso e então você terminará com uma lista ordenada.

~♪~	CONTADOR DE PLAYS
RADIOHEAD	156
KISHORE KUMAR	141
WILCO	111
NEUTRAL MILK HOTEL	94
BECK	88
THE STROKES	61
THE BLACK KEYS	35

Vamos pensar como engenheiros da computação e avaliar quanto tempo isso demoraria a ser executado. Lembre-se de que o tempo de execução $O(n)$ significa que você precisa passar por todos os elementos da lista uma vez. Por exemplo, executar uma pesquisa simples na lista de artistas significa olhar para cada artista uma vez.



Para encontrar o artista com o maior número de plays você precisa verificar cada item da lista. Isso tem tempo de execução $O(n)$, como você acabou de ver. Então você tem uma operação com tempo de execução $O(n)$ e precisa repetir essa operação n vezes:



Isso tem tempo de execução $O(n \times n)$ ou $O(n^2)$.

Algoritmos de ordenação são muito úteis. Agora você pode ordenar:

- nomes em uma agenda telefônica
- datas de viagem
- emails (do mais novo ao mais antigo)

Verificando menos elementos a cada vez

Talvez você esteja pensando: conforme passa pelas operações, o número de elementos que precisa analisar diminui. Eventualmente, você acaba tendo de checar apenas um elemento.

Então como o tempo de execução permanece sendo $O(n^2)$? Isto é uma boa pergunta, e a resposta tem a ver com a notação Big O. Falarei mais sobre isso no capítulo 4, mas aqui vai o

ponto principal.

Você estava certo sobre não precisar verificar n elementos a cada vez.

Você verifica n elementos, então $n - 1, n - 2 \dots 2, 1$. Na média, você verifica uma lista que tem $\frac{1}{2} \times n$ elementos. O tempo de execução é $O(n \times \frac{1}{2} \times n)$. Mas constantes como $\frac{1}{2}$ são ignoradas na notação Big O (novamente, leia o Capítulo 4 para ter acesso à discussão completa), então você escreve apenas $O(n \times n)$ ou $O(n^2)$.

A ordenação por seleção é um algoritmo bom, mas não é muito rápido. O Quicksort é um algoritmo de ordenação mais rápido, que tem tempo de execução de apenas $O(n \log n)$. Falarei dele no capítulo 4!

EXEMPLO DE CÓDIGO

Nós não lhe mostramos o código para ordenar a lista de músicas, mas a seguir estão alguns códigos que farão algo bem similar: ordenar um array do menor para o maior. Vamos escrever uma função para encontrar o menor elemento em um array:

```
def buscaMenor(arr):  
    menor = arr[0] ❶  
    menor_indice = 0 ❷  
    for i in range(1, len(arr)):  
        if arr[i] < menor:  
            menor = arr[i]  
            menor_indice = i  
    return menor_indice
```

❶ Armazena o menor valor.

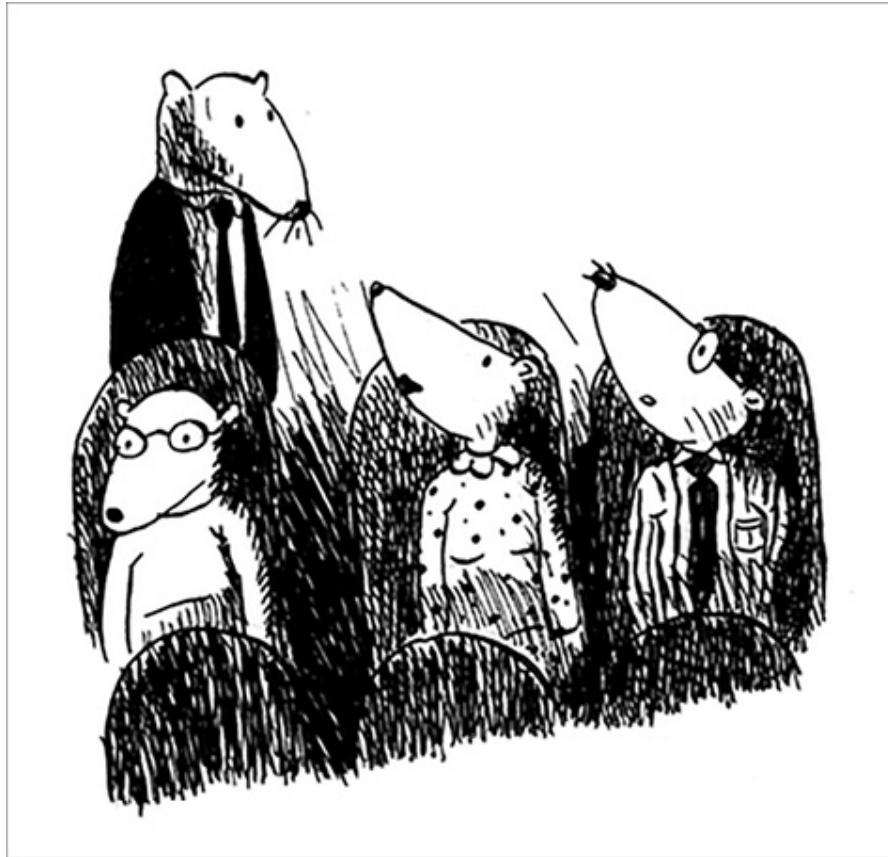
❷ Armazena o índice do menor valor.

Agora, você pode usar esta função para escrever a ordenação por seleção:

```
def ordenacaoPorSelecao(arr): ❶  
    novoArr = []  
    for i in range(len(arr)):  
        menor = buscaMenor(arr) ❷  
        novoArr.append(arr.pop(menor))  
    return novoArr  
  
print ordenacaoPorSelecao([5, 3, 6, 2, 10])
```

❶ Ordenações em um array.

❷ Encontra o menor elemento do array e adiciona ao novo array.

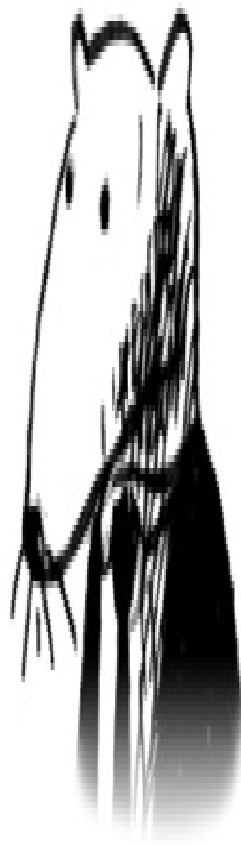


Recapitulando

- A memória do seu computador é como um conjunto gigante de gavetas.
- Quando se quer armazenar múltiplos elementos, usa-se um array ou uma lista.
- No array, todos os elementos são armazenados um ao lado do outro.
- Na lista, os elementos estão espalhados e um elemento armazena o endereço do próximo elemento.
- Arrays permitem leituras rápidas.
- Listas encadeadas permitem rápidas inserções e eliminações.
- Todos os elementos de um array devem ser do mesmo tipo (todos ints, todos doubles, e assim por diante).

3

Recursão



Neste capítulo

- Você aprenderá recursão. A recursão é uma técnica de programação utilizada em muitos algoritmos. É um assunto importante para a compreensão dos capítulos seguintes.
- Você aprenderá como separar um problema em caso-base e caso recursivo. A estratégia dividir para conquistar (Capítulo 4) usa este

conceito simples para resolver problemas complicados.

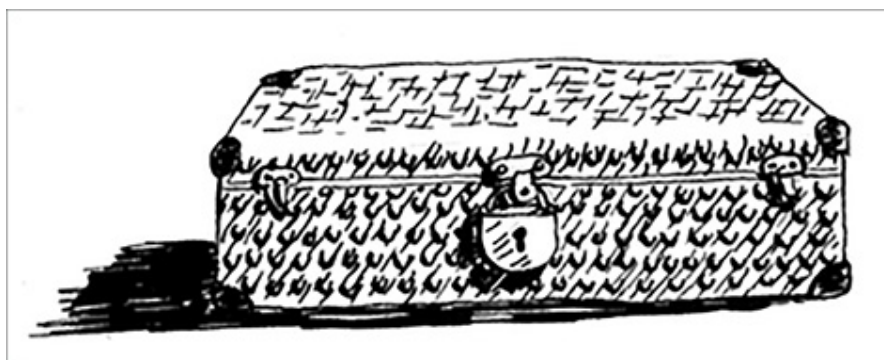
Estou animado com este capítulo porque trata de *recursão*, uma maneira elegante de solucionar problemas. A recursão é um dos meus tópicos favoritos, mas ela é polêmica. As pessoas ou a amam ou a odeiam, ou elas a odeiam até que aprendam a amá-la alguns anos depois. Eu estava nessa terceira situação. Para facilitar as coisas, tenho um conselho:

- Este capítulo apresenta vários exemplos de códigos. Execute-os para ver como eles funcionam.
- Falarei sobre funções recursivas. Pelo menos uma vez, analise uma função recursiva com um papel e uma caneta, algo do tipo “vamos ver, passo o número 5 para a função `fatorial`, e então retorno cinco vezes passando o número 4 para `fatorial`, o que me dá...”, e assim por diante. Analisar uma função dessa forma lhe ajudará a entender como funcionam as funções recursivas.

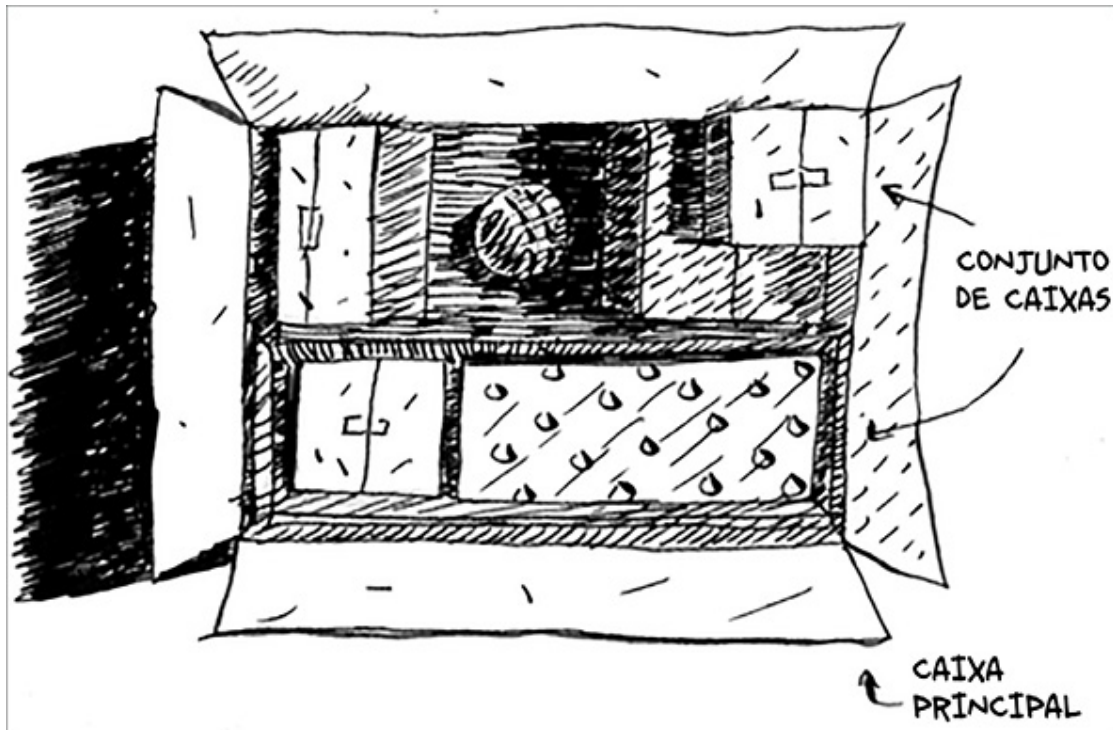
Este capítulo inclui muitos pseudocódigos. *Pseudocódigos* são uma descrição de alto nível de um problema em formato de código. É escrito como um código, mas utiliza linguagem mais próxima da humana.

Recursão

Suponha que você esteja vasculhando o porão de sua avó e encontre uma misteriosa mala trancada.



A sua avó diz que a chave para a mala provavelmente está em uma caixa.



Esta caixa contém mais caixas com mais caixas dentro delas. A chave está em alguma destas caixas. Qual é o seu algoritmo para procurá-la? Pense nisso antes de continuar a leitura.

Aqui está uma abordagem.



1. Monte uma pilha com as caixas que serão analisadas.
 2. Pegue uma caixa e olhe o que tem dentro dela.
 3. Se você encontrar outra caixa dentro dela, adicione-a a um novo monte para ser verificada mais tarde.
 4. Se você encontrar uma chave, terminou!
 5. Repita.
- Aqui está outra abordagem.



1. Olhe o que tem dentro da caixa.

2. Se encontrar outra caixa, volte ao passo 1.

3. Se encontrar a chave, terminou!

Qual abordagem lhe parece mais fácil? A primeira abordagem utiliza um loop `while` (enquanto, em português). Enquanto o monte existir, pegue uma caixa e olhe o que tem dentro dela:

```
def procure_pela_chave(caixa_principal):
    pilha = main_box.crie_uma_pilha_para_busca()
    while pilha is not vazia:
        caixa = pilha.pegue_caixa()
        for item in caixa:
            if item.e_uma_caixa():
                pilha.append(item)
            elif item.e_uma_chave():
                print "achei a chave!"
```

A segunda maneira utiliza a recursão. *Recursão* é quando uma função chama a si mesma. Veja o pseudocódigo de como isso funciona.

```
def procure_pela_chave(caixa):
    for item in caixa:
        if item.e_uma_caixa():
            procure_pela_chave(item) ❶
        elif item.e_uma_chave():
            print "achei a chave!"
```

❶ Recursão!

Ambas as abordagens cumprem com a mesma proposta, mas a segunda me parece mais objetiva. A recursão é usada para tornar a resposta mais clara. Não há nenhum benefício quanto ao desempenho ao utilizar a recursão. Na verdade, os loops algumas vezes são melhor para o desempenho de um programa. Gosto desta frase de Leigh Caldwell, do Stack Overflow: “Os loops podem melhorar o desempenho do seu programa. A recursão melhora o desempenho do seu programador. Escolha o que for mais importante para a sua situação.”¹

Muitos algoritmos importantes usam a recursão, então é fundamental entender este conceito.

Caso-base e caso recursivo



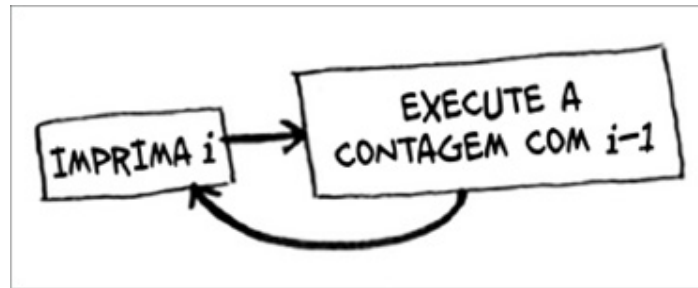
Devido ao fato de a função recursiva chamar a si mesma, é mais fácil escrevê-la erroneamente e acabar em um loop infinito. Por exemplo, suponha que você escreva uma função que imprima uma contagem regressiva, como esta:

```
> 3...2...1
```

Você pode escrever isso de maneira recursiva fazendo o seguinte:

```
def regressiva(i):  
    print i  
    regressiva(i-1)
```

Escreva este código e execute-o. Você perceberá um problema: essa função ficará executando para sempre!



Loop infinito.

> 3...2...1...0...-1...-2...

(Pressione Ctrl-C para interromper o seu script.)

Quando você escreve uma função recursiva, deve informar quando a recursão deve parar. É por isso que *toda função recursiva tem duas partes: o caso-base e o caso recursivo*. O caso recursivo é quando a função chama a si mesma. O caso-base é quando a função não chama a si mesma novamente, de forma que o programa não se torna um loop infinito.

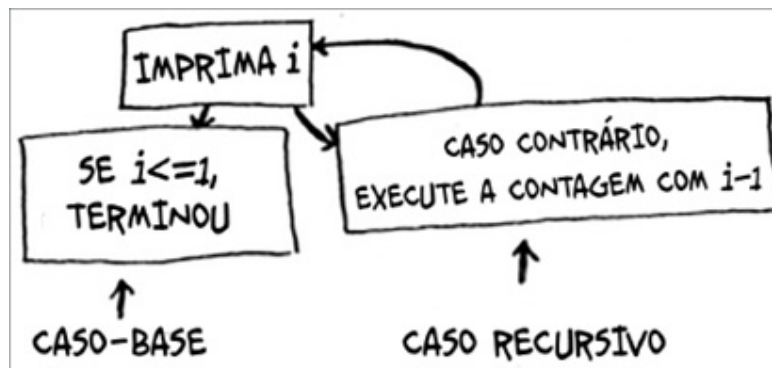
Vamos adicionar o caso-base à função de contagem regressiva:

```
def regressiva(i):
    print i
    if i <= 1: ❶
        return
    else: ❷
        regressiva(i-1)
```

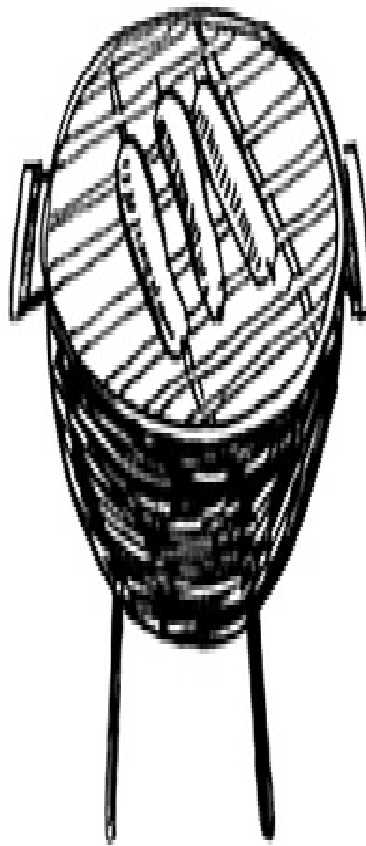
❶ Caso-base.

❷ Caso recursivo.

Agora, a função funciona como esperado. Ela fica mais ou menos assim:



A pilha



Esta seção aborda a *pilha de chamada* (call stack). Isto é um conceito importante em programação e indispensável para entender a recursão. Suponha que você esteja fazendo um churrasco para os seus amigos. Você tem uma lista de afazeres em forma de uma pilha de notas adesivas.



Você se lembra de que, quando falamos de arrays e listas, também havia uma lista de afazeres? Podia adicionar itens em qualquer lugar da lista ou remover itens aleatórios. A pilha de notas adesivas é bem mais simples. Quando você insere um item, ele é colocado no topo da pilha. Quando você lê um item, lê apenas o item do topo da pilha e ele é retirado da lista. Logo, sua lista de afazeres contém apenas duas ações: *push* (inserir) e *pop* (remover e ler).



Vamos ver como isso funciona na prática.



Esta estrutura de dados é chamada de *pilha*. A pilha é uma estrutura de dados simples. Você a tem usado esse tempo todo sem perceber!

A pilha de chamada

Seu computador usa uma pilha interna denominada *pilha de chamada*. Vamos ver isto na prática. Aqui está um exemplo simples:

```
def sauda(nome):
    print "Olá, " + nome + "!"
    sauda2(nome)
    print "preparando para dizer tchau..."
    tchau()
```

Esta função te cumprimenta e chama outras duas funções:

```
def sauda2(nome):
    print "Como vai " + nome + "?"

def tchau():
    print "ok, tchau!"
```

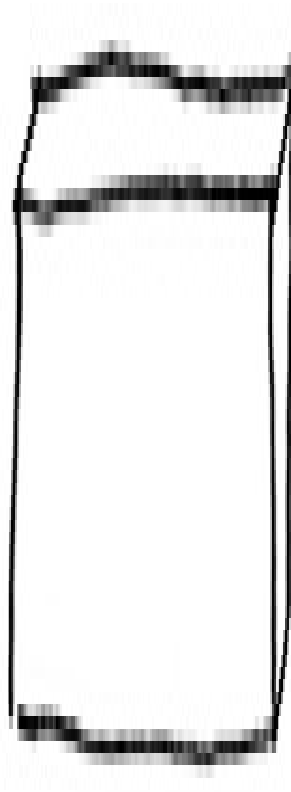
Vamos analisar o que acontece quando você chama uma função.

Nota

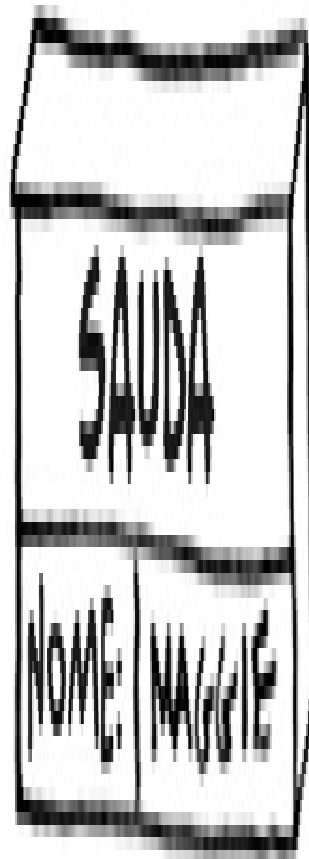
`print` é uma função em Python, mas, para facilitar as coisas, vamos fingir que não é. Entre na brincadeira.

N.R.T.: tecnicamente `print` não é uma função em Python 2, mas uma instrução ou statement.

Suponha que você chame `sauda("maggie")`. Primeiro, seu computador aloca uma caixa de memória para essa chamada.



Agora, vamos usar a memória. A variável `nome` é setada para “maggie”. Isso precisa ser salvo.

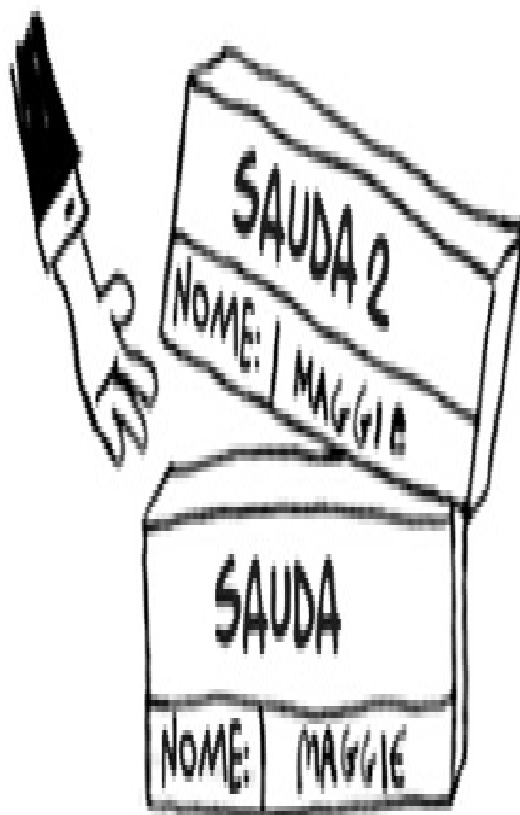


Cada vez que você faz uma chamada de função, seu computador salva na memória os valores para todas as variáveis. Depois disso, imprime `olá, maggie!`. Então, chama `sauda2("maggie")`.

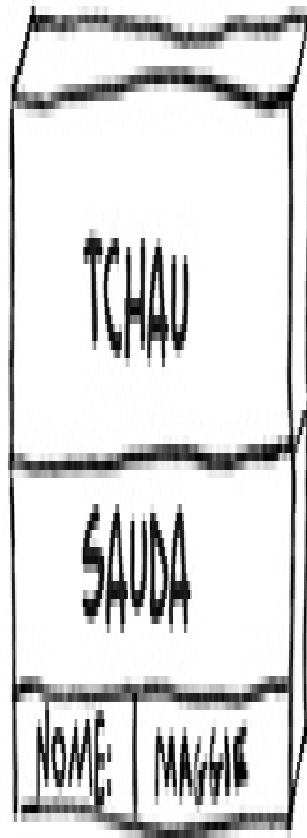


Novamente, seu computador aloca uma caixa de memória para essa chamada de função.

Seu computador está usando uma pilha para estas caixas. A segunda caixa é adicionada em cima da primeira. Você imprime "como vai maggie?". Então, retorna da chamada de função. Quando isso acontece, a caixa do topo da pilha é retirada.



Agora, a caixa do topo da pilha aloca os valores da função `sauda`, o que significa que você retornou à função `sauda`. Quando você chamou a função `sauda2`, a função `sauda` ficou *parcialmente completa*. Esta é a grande ideia por trás desta seção: *quando você chama uma função a partir de outra, a chamada de função fica pausada em um estado parcialmente completo*. Todos os valores das variáveis para aquela função ainda estão armazenados na memória. Agora que você já utilizou a função `sauda2`, você está de volta na função `sauda` e pode continuar de onde parou. Primeiro, imprime "preparando para dizer tchau..." e então chama a função `tchau`.



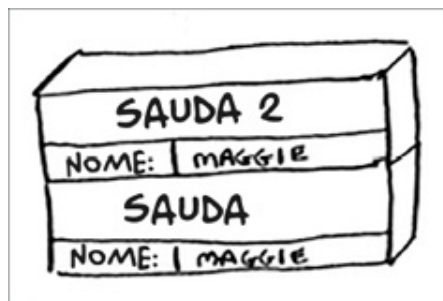
Uma caixa para esta função é adicionada ao topo da pilha. Quando você imprimir ok, tchau!, retornará da chamada de função.



Agora, você está de volta à função `sauda`. Não há nada mais a ser feito, e você pode sair da função `sauda` também. Essa pilha usada para guardar as variáveis de múltiplas funções é denominada pilha de chamada.

EXERCÍCIOS

3.1 Suponha que eu forneça uma pilha de chamada como esta:



Quais informações você pode retirar baseando-se apenas nesta pilha de chamada?

Agora, vamos ver esta pilha de chamada sendo executada com uma função recursiva.

A pilha de chamada com recursão

As funções recursivas também utilizam a pilha de chamada! Vamos analisar isto na prática com a função `fat` (fatorial). `fat(5)` é escrita como $5!$ e é definida da seguinte forma: $5! = 5 * 4 * 3 * 2 * 1$. De forma semelhante, `fat(3)` é $3 * 2 * 1$. Aqui está uma função recursiva para calcular a fatorial de um número:

```
def fat(x):  
    if x == 1:  
        return 1  
    else:  
        return x * fat(x-1)
```

Agora, você chama a função `fat(3)`. Vamos analisar esta pilha de chamada linha por linha e ver como ela se altera. Lembre-se, a caixa mais próxima ao topo lhe diz em qual chamada a função `fat` se encontra atualmente.

CÓDIGO

PILHA DE CHAMADA

fat(3)

FAT
X 3

PRIMEIRA EXECUÇÃO
DE fat: X É 3.

if x == 1:

FAT
X 3

else:

FAT
X 3

UMA CHAMADA
RECURSIVA!

return x * fat(x-1)

FAT
X 2
FAT
X 3

AGORA, ESTA É A
SEGUNDA EXECUÇÃO
DE fat: X É 2.

if x == 1:

FAT
X 2
FAT
X 3

A CHAMADA DE FUNÇÃO
MAIS AO TOPO É
A CHAMADA QUE
ESTAMOS ATUALMENTE.

else:

FAT
X 2
FAT
X 3

PERCEBA QUE AMBAS AS
CHAMADAS DE FUNÇÕES
POSSUEM UMA VARIÁVEL
X, MAS O VALOR DA
VARIÁVEL X É DIFERENTE
EM CADA UMA.

return x * fat(x-1)

FAT
X 1
FAT
X 2
FAT
X 3

VOCÊ NÃO CONSEGUE
ACESSAR ESTA VARIÁVEL
X A PARTIR DESTA
CHAMADA DE FUNÇÃO
E VICE E VERSA.

if x == 1:

FAT
X 1
FAT
X 2
FAT
X 3

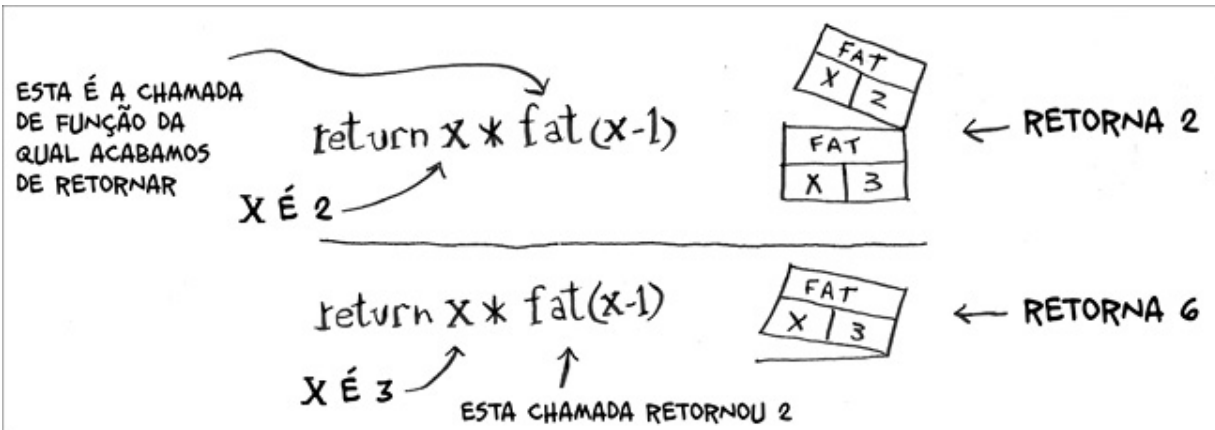
NOSSA, NÓS FIZEMOS
TRÊS CHAMADAS À
FUNÇÃO fat, MAS
NÓS NÃO HAVÍAMOS
FINALIZADO NENHUMA
CHAMADA ATÉ AGORA!

return 1

FAT
X 1
FAT
X 2
FAT
X 3

ESTE É O PRIMEIRO ITEM A
SER RETIRADO DA PILHA,
O QUE SIGNIFICA QUE
ESTA É A PRIMEIRA
CHAMADA DA QUAL
NÓS RETORNAMOS.

RETORNA 1



Repare que cada chamada para a função `fat` tem seu próprio valor de `x`.

Você não consegue acessar a mesma função com outro valor de `x`.

A pilha tem um papel importante na recursão. No primeiro exemplo, mostrei duas abordagens para encontrar a chave. Aqui está a primeira.



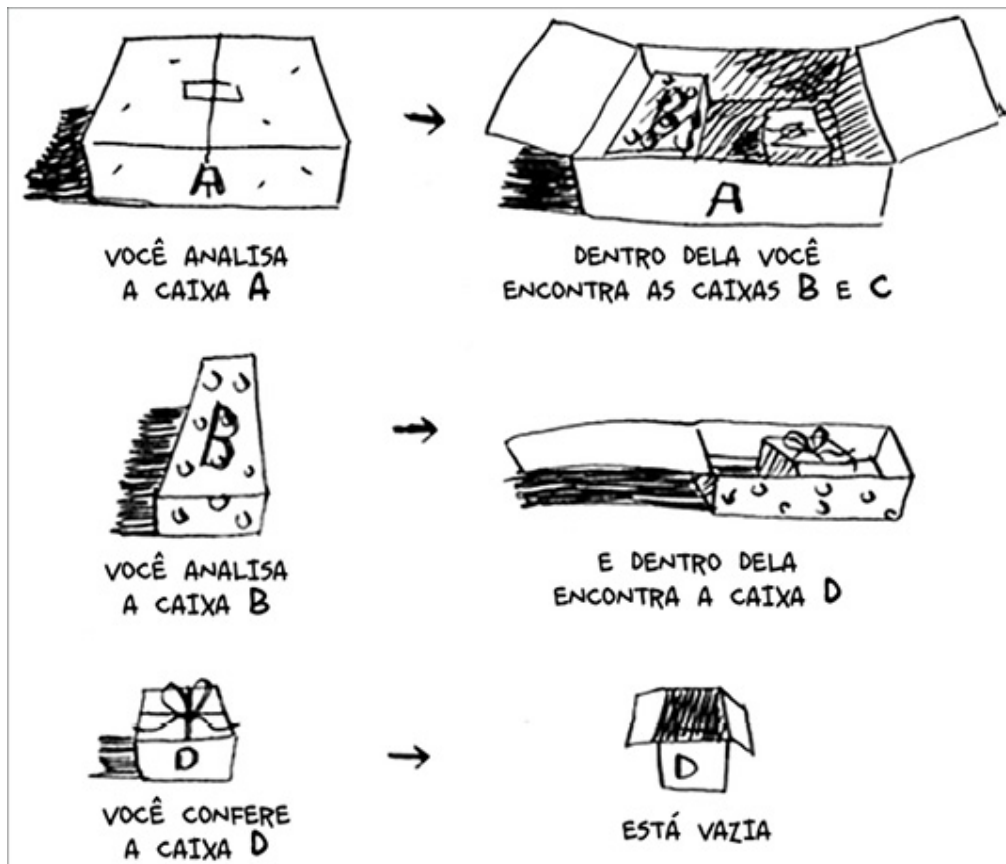
Desta forma, você fez um monte com caixas para analisar, então sabe quais caixas você ainda precisa abrir.



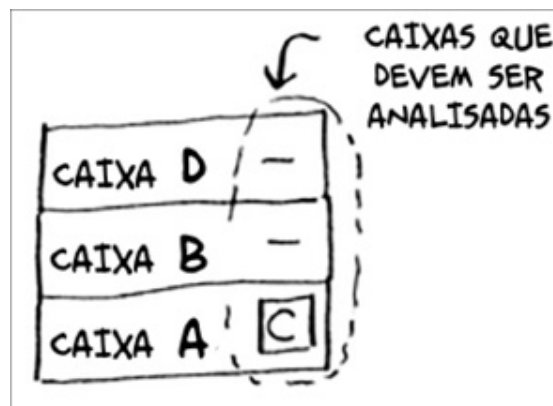
Mas na abordagem recursiva não existem montes.



Se não existem montes, como um algoritmo reconhece quais caixas ele deve procurar? Aqui está um exemplo.



Neste ponto, a pilha de chamada se parece com isto:



O “monte de caixas” é salvo na pilha! Esta é uma pilha com as funções de chamada completadas até a metade, cada uma com a sua lista de caixas, também completadas até a metade, para ser analisadas. Utilizar a pilha é conveniente porque você não precisa acompanhar o monte de caixas – a pilha faz isso para você. Usar a pilha é bom, porém, existe um custo: salvar toda essa informação pode ocupar muita memória. Cada uma destas funções de chamada ocupa um pouco de memória, e quando a sua pilha está

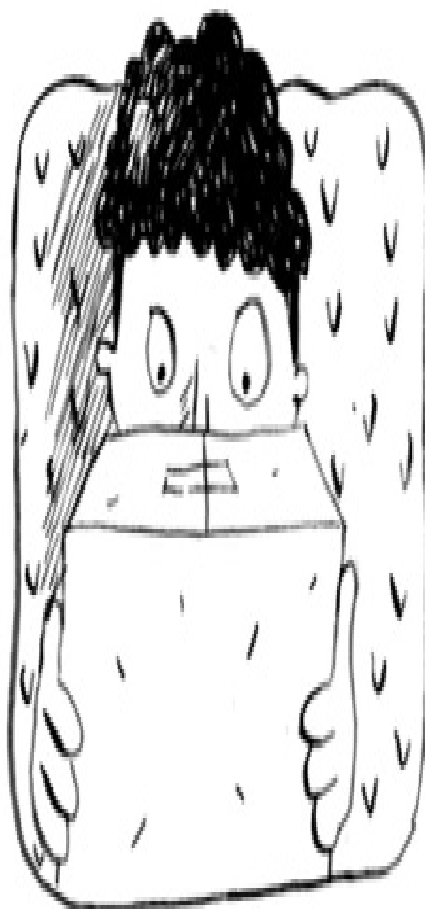
muito cheia é sinal de que seu computador está salvando informação para muitas chamadas de funções. Para esta situação, você tem duas opções:

- Reescrever seu código utilizando loops.
- Utilizar o que chamamos de tail recursion (*recursão de cauda*). Isto é um tópico avançado em recursão e está fora do escopo deste livro. Esta técnica também não é suportada por todas as linguagens de programação.

EXERCÍCIO

3.2 Suponha que você acidentalmente escreva uma função recursiva que fique executando infinitamente. Como você viu, seu computador aloca memória na pilha para cada chamada de função. O que acontece com a pilha quando a função recursiva fica executando infinitamente?

Recapitulando



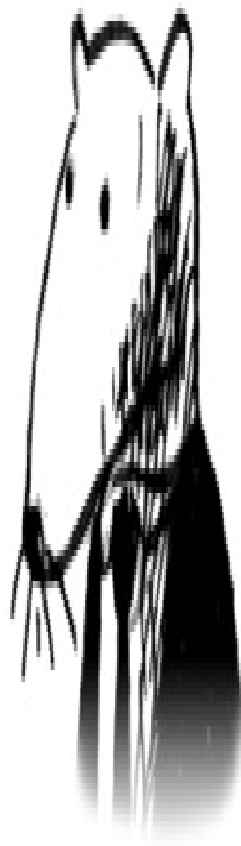
- Recursão é quando uma função chama a si mesma.

- Toda função recursiva tem dois casos: o caso-base e o caso recursivo.
- Uma pilha tem duas operações: push e pop.
- Todas as chamadas de função vão para a pilha de chamada.
- A pilha de chamada pode ficar muito grande e ocupar muita memória.

1 <http://stackoverflow.com/a/72694/139117>.

4

Quicksort



Neste capítulo

- Você irá se deparar, ocasionalmente, com problemas que não podem ser resolvidos com algum algoritmo de seu conhecimento. Porém quando um bom desenvolvedor de algoritmos encontra um destes problemas, ele não desiste. Pelo contrário, ele utiliza uma ampla gama de técnicas para encontrar uma solução, sendo a

técnica de dividir para conquistar a primeira que você aprenderá.

- Você conhecerá também o quicksort, um algoritmo de ordenação elegante que é utilizado com frequência. Este algoritmo utiliza a técnica de dividir para conquistar.

No último capítulo você aprendeu tudo sobre recursão. Este capítulo focará na utilização destas suas novas habilidades aplicadas na resolução de problemas. Para isto, vamos explorar a técnica *dividir para conquistar* (DC), uma técnica recursiva muito conhecida para resolução de problemas.

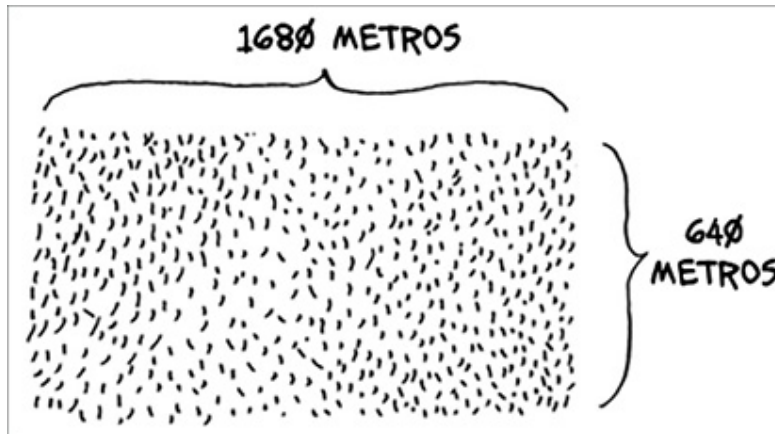
Este capítulo trata do ponto principal dos algoritmos, pois um algoritmo que consegue resolver apenas um tipo de problema não é muito útil. Assim, a técnica DC oferece uma nova maneira de pensar sobre a resolução de problemas, tornando-se mais uma alternativa em sua caixa de ferramentas. Quando você se deparar com um problema novo, não terá motivos para ficar desorientado. Em vez disso, poderá se perguntar “Será que posso resolver este problema usando a técnica de dividir para conquistar?”.

Ao final deste capítulo você terá aprendido o seu primeiro algoritmo que utiliza a técnica DC: o *quicksort*. O algoritmo quicksort é um algoritmo de ordenação muito mais rápido do que o algoritmo de ordenação por seleção (que você aprendeu no Capítulo 2), e é também um bom exemplo de programação elegante.



Dividir para conquistar

A técnica DC pode levar algum tempo para ser compreendida. Por isso, veremos três exemplos. Primeiro, mostrarei um exemplo visual. Depois, mostrarei um código de exemplo simples, mas não tão elegante. Por fim, nos aprofundaremos no quicksort, um algoritmo de ordenação que utiliza DC. Suponha que você seja um fazendeiro que tenha uma área de terra.



Você quer dividir sua fazenda em porções *quadradas* iguais, sendo que estas porções devem ter o maior tamanho possível. Assim, nenhuma destas alternativas funcionará.

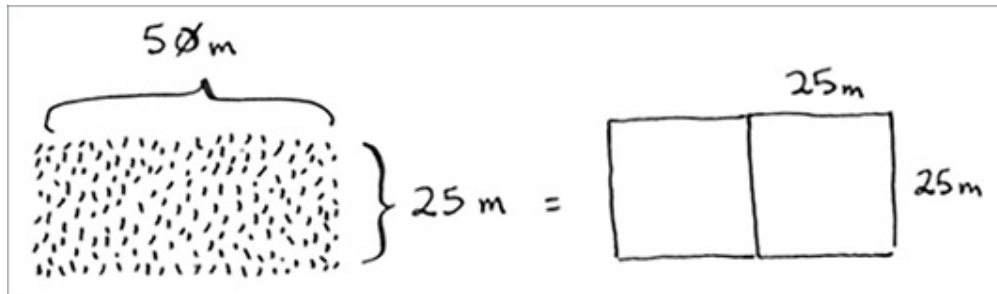


Como encontrará o maior tamanho possível para estes quadrados? Usando a estratégia DC! Os algoritmos DC são recursivos. Assim, para resolver um problema utilizando DC, você deve seguir dois passos:

1. Descubra o caso-base, que deve ser o caso mais simples possível.
2. Divida ou diminua o seu problema até que ele se torne o caso-base.

Vamos usar DC para encontrar a solução deste problema. Qual é a maior largura que você pode usar?

Primeiro, descubra o caso-base. Seria mais fácil solucionar este problema se um dos lados fosse múltiplo do outro.

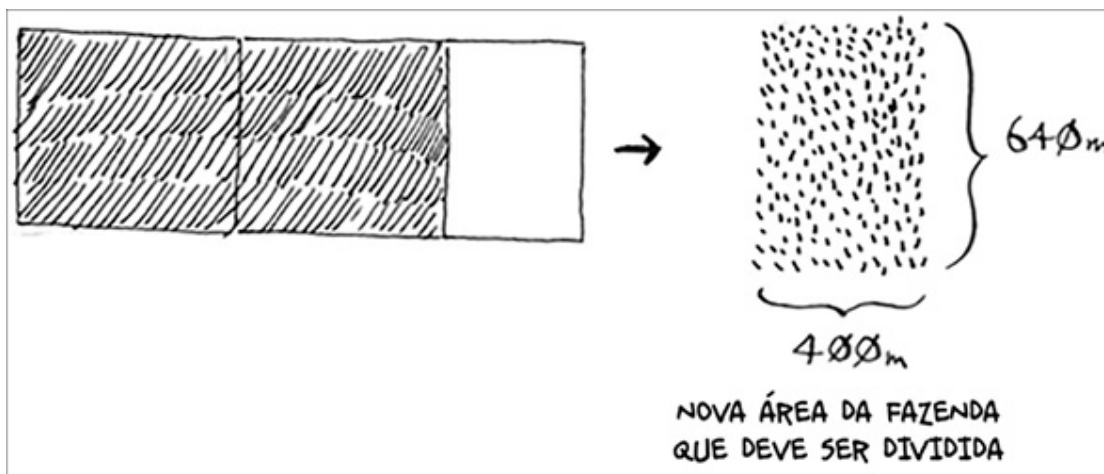


Suponha que um dos lados tenha 25 metros (m) e o outro tenha 50. Assim, o maior quadrado que você pode ter mede 25 m x 25 m. Você precisa de dois destes quadrados para dividir a porção de terra.

Agora você precisa descobrir o caso recursivo, e é aqui que a estratégia DC entra em ação. Seguindo a estratégia DC, a cada recursão você deve reduzir o seu problema. Então, como reduzir este problema? Vamos começar identificando os maiores quadrados que você pode utilizar.



Você pode posicionar dois quadrados de 640×640 na fazenda e ainda continuará com uma porção de terra para ser dividida. Este é o momento “Aha!”. Você ainda tem um segmento da fazenda que deve ser dividido. *Por que não aplica este mesmo algoritmo neste segmento?*



Você iniciou com uma porção de terra medindo 1.680×640 que deveria ser dividida. Porém agora você precisa dividir um segmento menor, que mede 640×400 . *Caso encontre o maior quadrado que divide este segmento, ele será o maior quadrado que dividirá toda a fazenda.* Você acabou de reduzir um problema de divisão de uma fazenda medindo 1.680×640 para um problema de divisão de uma área medindo 640×400 !

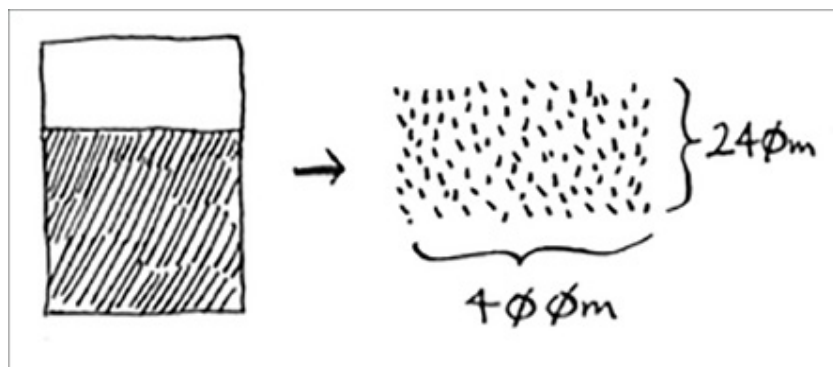
Algoritmo de Euclides

“Caso você encontre o maior quadrado que divide este segmento, ele será o maior quadrado que irá dividir toda a fazenda.” Se não parece óbvio o motivo de esta afirmação ser verdadeira, não se preocupe, ela realmente não é trivial. Infelizmente, a prova desta afirmação é um pouco longa para ser incluída neste livro, então você terá de confiar em mim. Caso você queira entender a prova, procure o Algoritmo de Euclides. A Khan Academy deu uma boa explicação, disponível aqui: <https://www.khanacademy.org/computing/computer-science/cryptography/modarithmetic/a/the-euclidean-algorithm>.

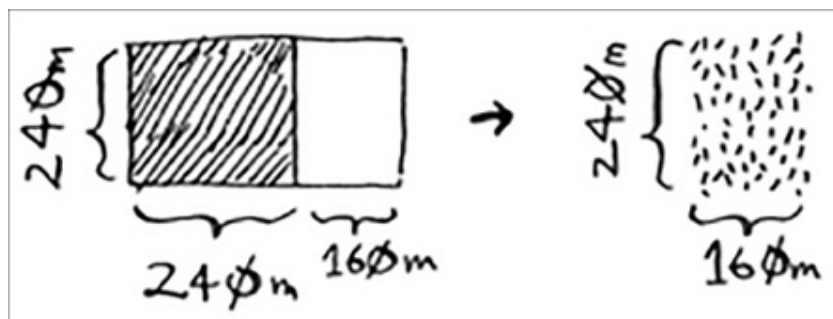


Vamos aplicar o mesmo algoritmo novamente. Começando com uma fazenda medindo 640×400 m, o maior quadrado que você pode ter mede 400×400 m.

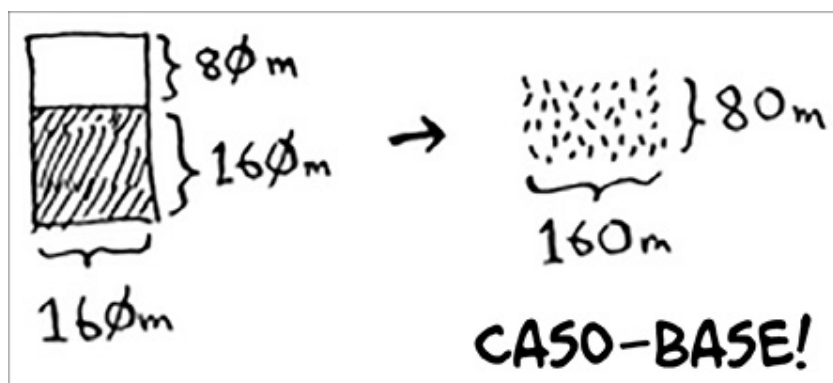
E isso deixa você com um segmento menor do que 400×240 m.



Você pode desenhar um quadrado neste segmento que lhe deixa com um segmento ainda menor, de 240×160 m.



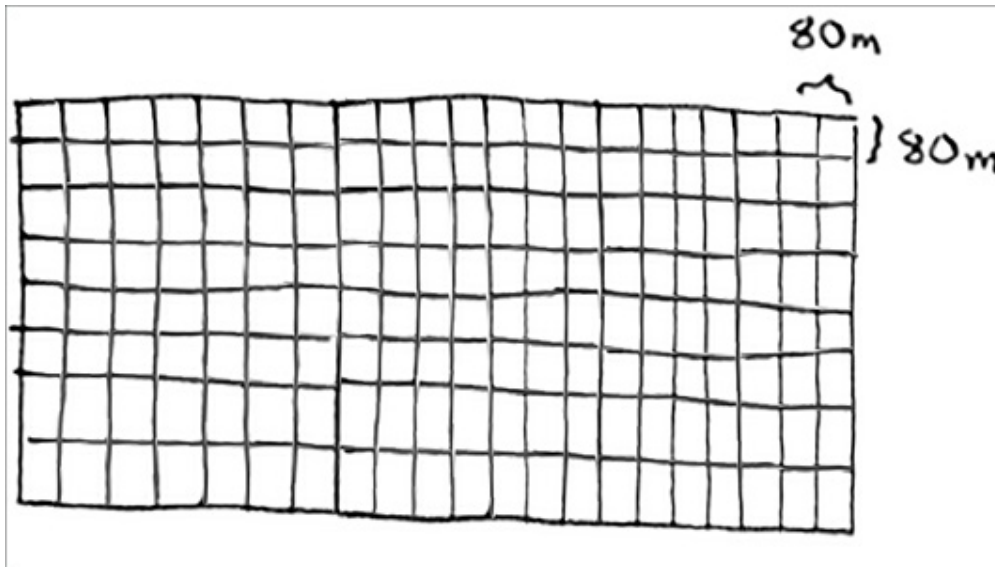
Então, você desenha um quadrado neste segmento para ter um segmento ainda menor.



Ei, você acabou de descobrir o caso-base, pois 80 é um múltiplo de 160. Se dividir este segmento em quadrados, não haverá segmentos sobrando!



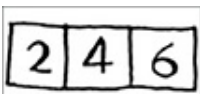
Assim, para a fazenda original, o maior quadrado que você pode utilizar é 80×80 m.



Para recapitular, estes são os passos para aplicação da estratégia DC:

1. Descubra o caso-base, que deve ser o caso mais simples possível.
2. Descubra como reduzir o seu problema para que ele se torne o caso-base.

O algoritmo DC não é um simples algoritmo que você aplica em um problema, mas sim uma maneira de pensar sobre o problema. Vamos ver mais um exemplo.



Você tem um array de números.

Você deve somar todos os números e retornar o valor total. Isto é simples de ser feito com um loop:

```
def soma(lista):  
    total = 0  
    for x in lista:  
        total += x  
    return total
```

```
print soma([1, 2, 3, 4])
```

Mas como isso poderia ser feito com uma função recursiva?

Passo 1: Descubra o caso-base. Qual é o array mais simples que você pode obter? Pense sobre o caso mais simples: se você tiver um array com 0 ou com 1 elemento, será muito simples calcular a soma.

$$\text{CASO-BASE} \left\{ \begin{array}{l} [] \text{ } \emptyset \text{ ELEMENTOS} = \text{SOMA É } \emptyset \\ [7] \text{ } 1 \text{ ELEMENTOS} = \text{SOMA É } 7 \end{array} \right.$$

Logo, esse é o caso-base.

Passo 2: Você deve chegar mais perto de um array vazio a cada recursão. Como pode reduzir o tamanho do seu problema? Esta é uma alternativa:

$$\text{SOMA} ([2, 4, 6]) = 12$$

A soma deste array é igual a isto:

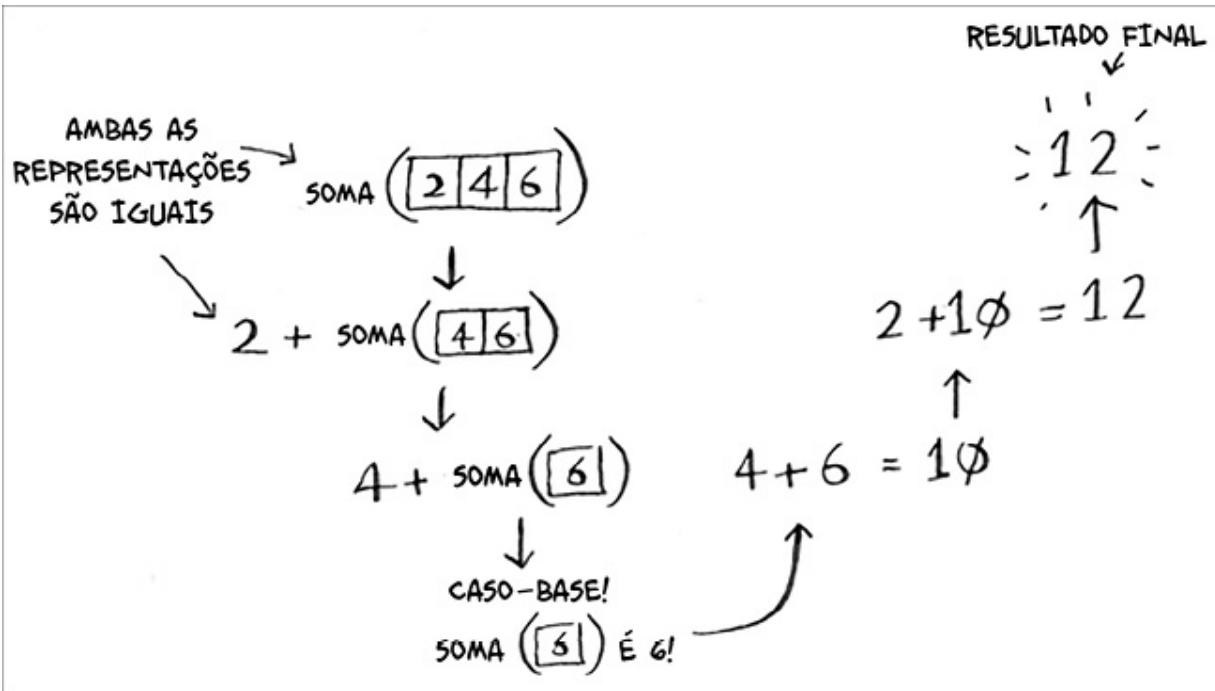
$$2 + \text{SOMA} ([4, 6]) = 2 + 10 = 12$$

Em ambos os casos o resultado é 12. Porém, na segunda versão, você está usando um array menor na função soma. Ou seja, *you are decreasing the size of the problem!*

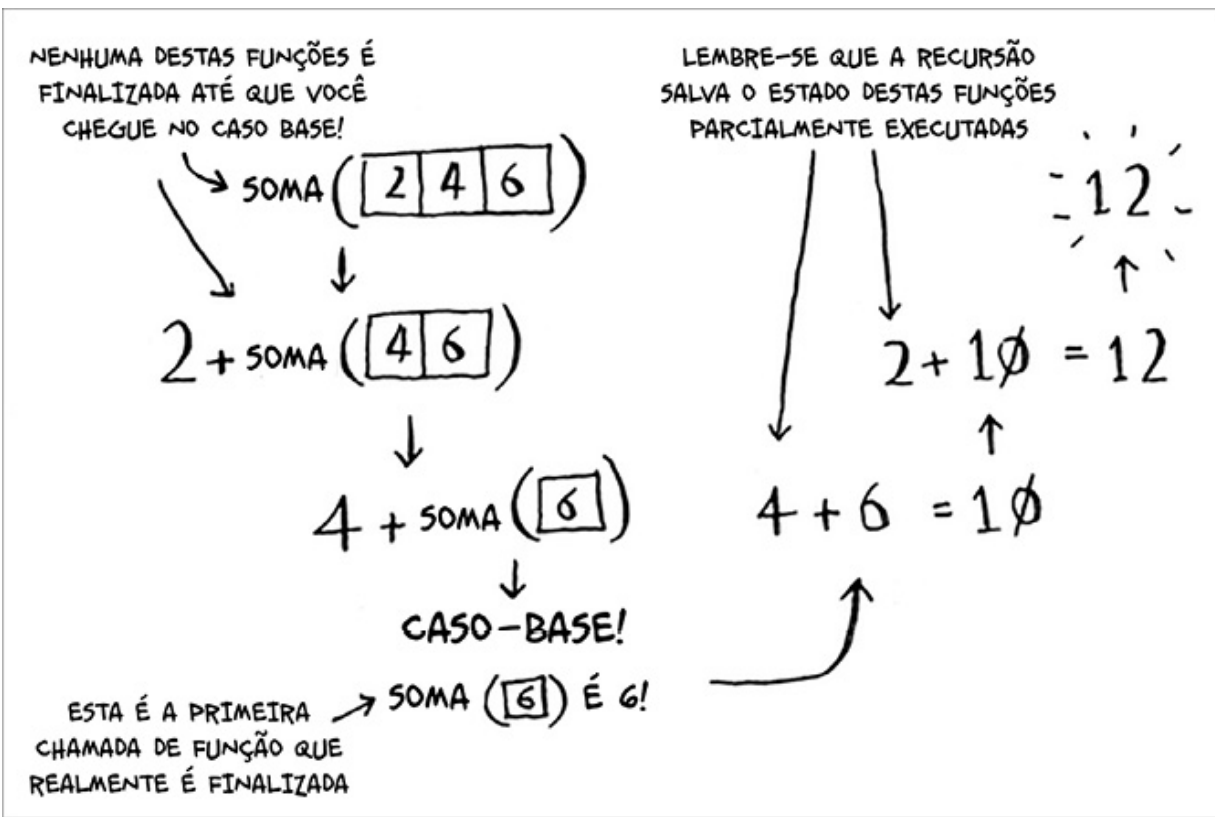
A sua função soma poderia funcionar assim:



Aqui está um exemplo da função na prática:



Lembre-se de que a recursão tem memória dos estados anteriores.



Dica

Quando estiver escrevendo uma função de recursão que envolva um array, o

caso-base será, muitas vezes, um array vazio ou um array com apenas um elemento. Se estiver com problemas, use este caso como base.

Uma espiada em programação funcional

“Por que eu faria isto recursivamente quando é mais simples fazer através de um loop?” é o que você pode estar pensando. Bem, estamos dando uma espiada em programação funcional! Linguagens de programação funcional, como Haskell, não contêm loops, e isso faz com que você tenha de usar funções como essa. Se você compreende bem o que é recursão, linguagens funcionais serão simples de entender. Por exemplo, você escreveria uma função somatória em Haskell assim:

```
soma [] = 0 ❶
```

```
soma (x:xs) = x + (soma xs) ❷
```

❶ Caso-base.

❷ Caso recursivo.

Perceba que parece que você tem duas definições para a função. A primeira definição é executada quando você alcança o caso-base, e a segunda é executada no caso recursivo. Você também pode escrever essa função em Haskell usando um operador condicional if (se, em português):

```
soma arr = if arr == []  
           then 0  
           else (head arr) + (soma (tail arr))
```

Porém a primeira definição é mais simples de ler. Como Haskell se baseia fortemente em recursão, essa linguagem inclui vários detalhes como este para tornar a recursão mais simples. Se você gosta de recursão ou caso você esteja interessado em aprender uma nova linguagem de programação, dê uma olhada em Haskell.

EXERCÍCIOS



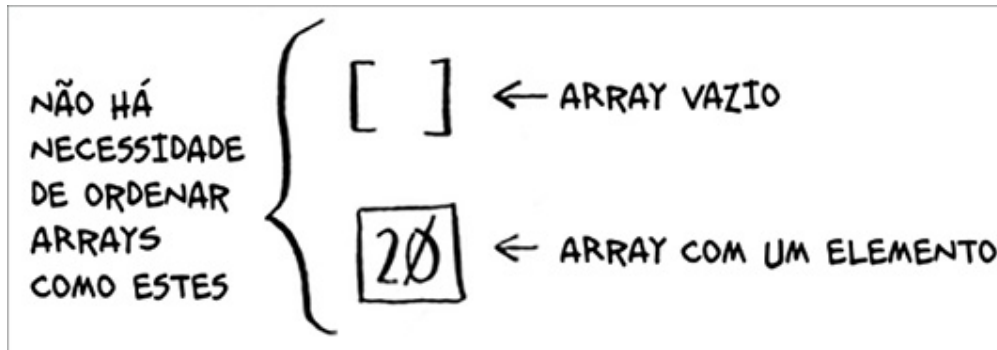
- 4.1** Escreva o código para a função `soma`, vista anteriormente.
- 4.2** Escreva uma função recursiva que conte o número de itens em uma lista.
- 4.3** Encontre o valor mais alto em uma lista.
- 4.4** Você se lembra da pesquisa binária do Capítulo 1? Ela também é um algoritmo do tipo dividir para conquistar. Você consegue determinar o caso-base e o caso recursivo para a pesquisa binária?

Quicksort



O quicksort é um algoritmo de ordenação. Este algoritmo é muito mais rápido do que a ordenação por seleção e é muito utilizado na prática. Por exemplo, a biblioteca-padrão da linguagem C tem uma função chamada `qsort`, que é uma implementação do quicksort. O algoritmo quicksort também utiliza a estratégia DC.

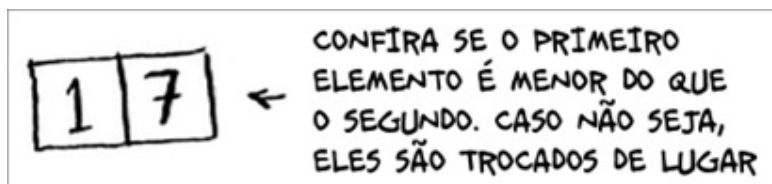
Vamos usar o quicksort para ordenar um array. Qual é o array mais simples que um algoritmo de ordenação pode ordenar (lembre-se da minha dica na seção anterior)? Bem, alguns arrays não precisam nem ser ordenados.



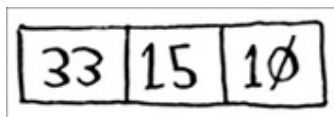
Arrays vazios ou arrays com apenas um elemento serão o caso-base. Você pode apenas retornar esses arrays como eles estão, visto que não há nada para ordenar:

```
def quicksort(array):  
    if len(array) < 2:  
        return array
```

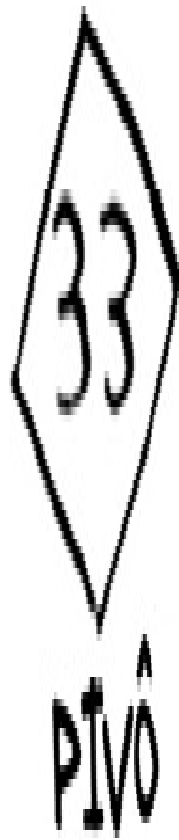
Vamos dar uma olhada em arrays maiores. Um array com dois elementos também é muito simples de ordenar.



E um array com três elementos?



Lembre-se, você está usando DC. Sendo assim, quer quebrar este array até que você chegue ao caso-base. Portanto o funcionamento do quicksort segue esta lógica: primeiro, escolha um elemento do array. Esse elemento será chamado de *pivô*.



Falaremos sobre como escolher um bom pivô mais tarde. Neste momento, vamos utilizar o primeiro item do array como pivô.

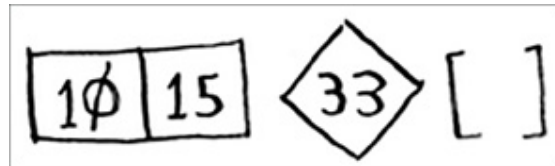
Assim, encontre os elementos que são menores do que o pivô e também os elementos que são maiores.



Isso é chamado de *particionamento*. Desse modo, você tem:

- Um subarray contendo todos os números menores do que o pivô
- O pivô
- Um subarray contendo todos os números maiores do que o pivô

Os dois subarrays não estão ordenados, apenas particionados. Porém, se eles estivessem ordenados, a ordenação do array contendo todos os elementos seria simples.



Caso os subarrays estejam ordenados, poderá combiná-los desta forma: array esquerdo + pivô + array direito. Consequentemente, terá um array ordenado. Neste caso, temos $[10, 15] + [33] + [] = [10, 15, 33]$, que é um array ordenado.

Como você pode ordenar os subarrays? Bem, o caso-base do quicksort consegue ordenar arrays de dois elementos (o subarray esquerdo) e também arrays vazios (o subarray direito). Assim, se utilizar o quicksort em ambos os subarrays e então combinar os resultados, terá um array ordenado!

```
quicksort([15, 10]) + [33] + quicksort([])
> [10, 15, 33] ❶
```

❶ Um array ordenado

Isto funcionará com qualquer pivô. Suponha que você tenha escolhido o número 15 como pivô.

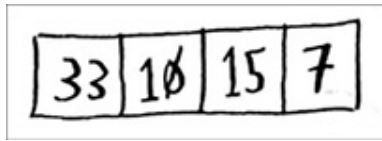


Ambos os subarrays contêm apenas um elemento, e você já sabe como ordenar este tipo de array. Logo, já sabe como ordenar um array de três elementos. Estes são os passos:

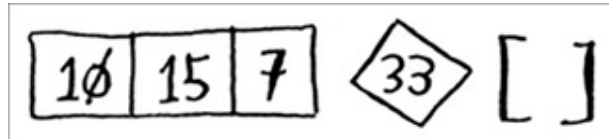
1. Escolha um pivô.
2. Particione o array em dois subarrays, separando-os entre elementos menores do que o pivô e elementos maiores do que o pivô.

3. Execute o quicksort recursivamente em ambos os subarrays.

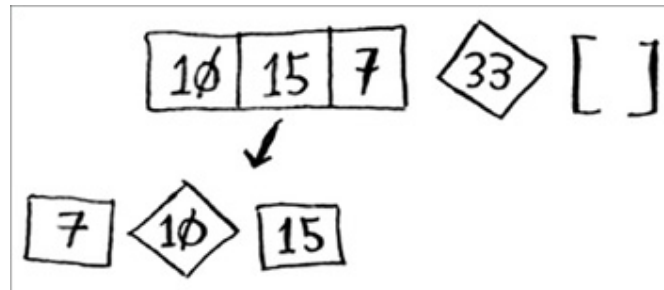
E quanto a um array de quatro elementos?



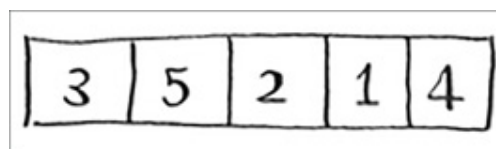
Suponha que, desta vez, você escolheu o número 33 como pivô.



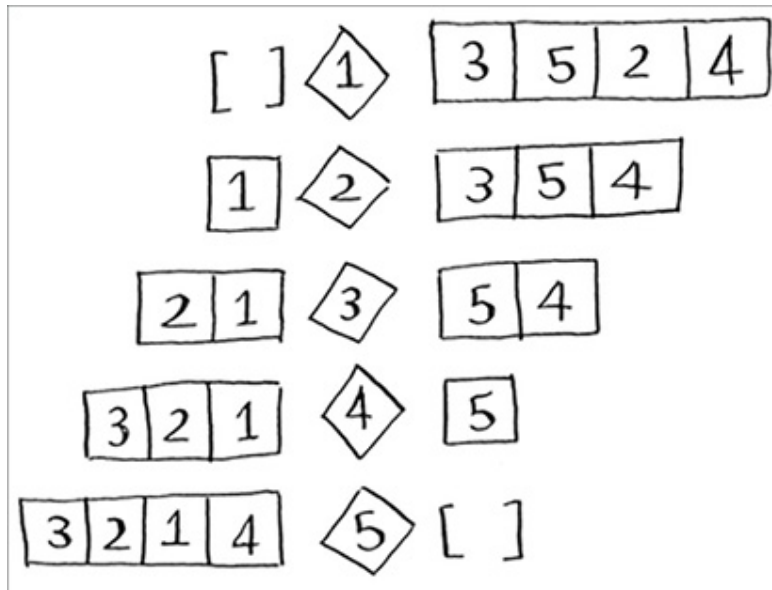
O array da esquerda contém três elementos, e você já sabe como ordenar arrays de três elementos: executando o quicksort recursivamente.



Agora pode ordenar arrays de quatro elementos. Sabendo ordenar arrays com quatro elementos, você consegue ordenar arrays com cinco elementos. Como? Suponha que tenha um array com cinco elementos.

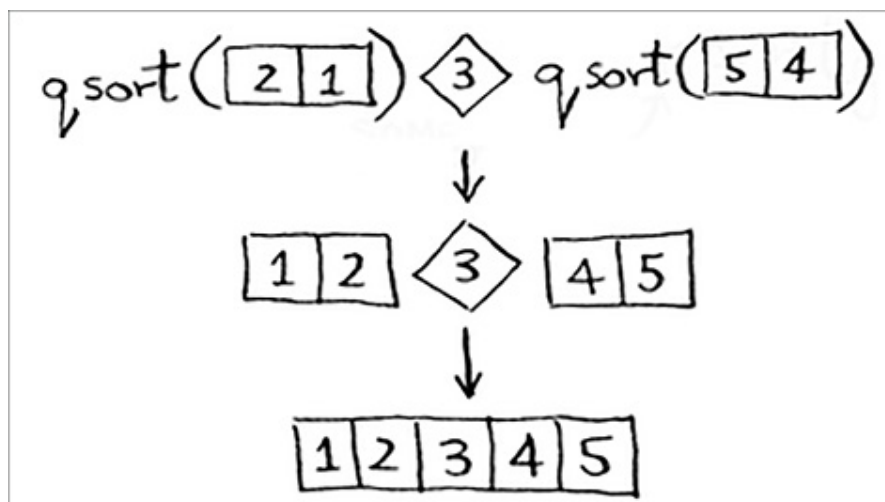


Estas são todas as maneiras pelas quais você pode particionar este array, dependendo do pivô que escolher.

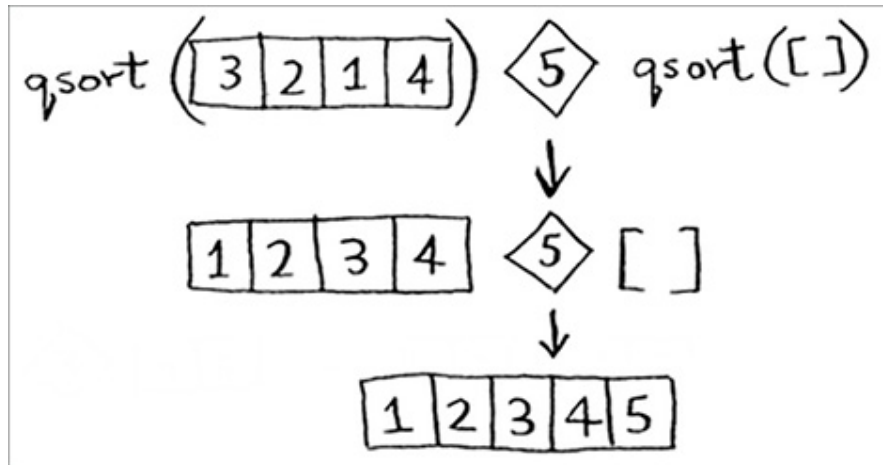


Perceba que todos estes subarrays têm entre 0 a 4 elementos, e você já sabe como ordenar arrays de 0 a 4 elementos usando o quicksort! Logo, não importa o pivô que você escolher, pois você poderá executar o quicksort recursivamente em ambos os subarrays.

Por exemplo, imagine que você escolheu o número 3 como pivô. Você executa o quicksort nos subarrays.



Os subarrays são ordenados e então você os combina, obtendo um array ordenado. Isto funcionará mesmo que escolha o número 5 como pivô.



Isso funcionará, na verdade, com qualquer elemento como pivô. Agora você já consegue ordenar um array de cinco elementos. Usando a mesma lógica, conseguirá ordenar um array de seis elementos ou mais.

Provas por indução

Você acabou de observar alguns exemplos de *provas por indução*. Estas provas representam uma maneira de mostrar que o seu algoritmo funciona. Cada prova por indução segue dois passos: o caso-base e o caso indutivo. Isso não soa familiar? Imagine que eu queira provar que sou capaz de subir até o topo de uma escada. No caso indutivo, se minhas pernas estiverem em um degrau, poderei colocá-las no próximo degrau. Assim, se estiver no degrau 2, poderei subir para o degrau 3. Este é o caso indutivo. Já para o caso-base, falarei que minhas pernas estão no degrau 1 e que, portanto, sou capaz de subir a escada inteira, um degrau de cada vez.

Você usa uma lógica semelhante para o quicksort. No caso-base, mostrei que o algoritmo funciona para o caso-base: arrays de tamanho 0 e 1. No caso indutivo, mostrei que, da mesma forma que o quicksort funciona para um array de tamanho 1, ele também funcionará para arrays de tamanho 2. Assim como ele funciona para arrays de dois elementos, também funcionará para arrays de três elementos, e assim por diante. Dessa forma, podemos dizer que o quicksort funciona para todos os tamanhos de array. Não me aprofundarei em provas por indução, mas elas são divertidas e andam lado a lado com a estratégia DC.



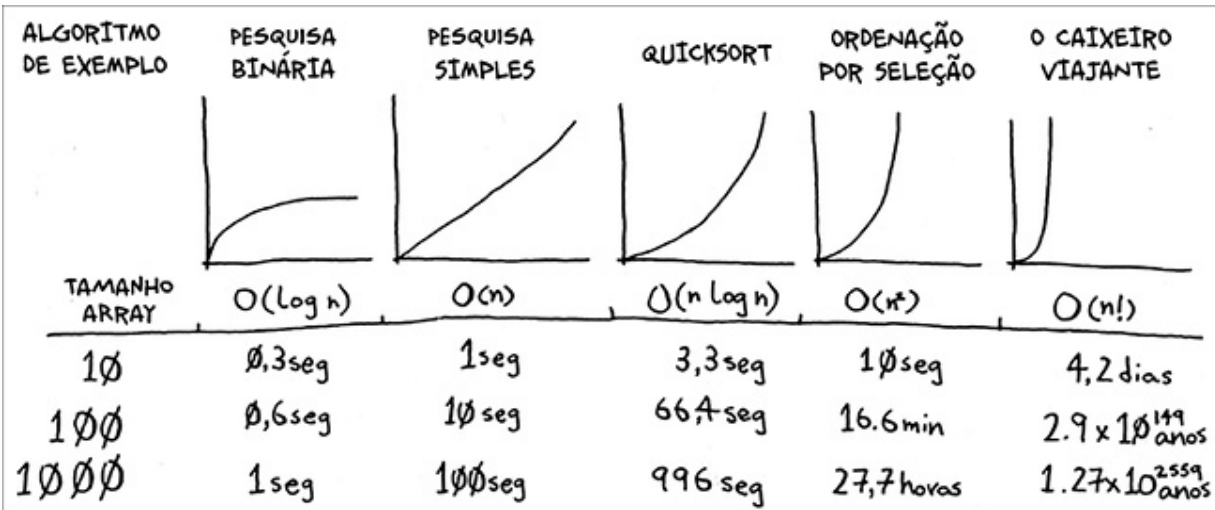
Aqui está o código para o quicksort:

```
def quicksort(array):  
    if len(array) < 2:  
        return array ❶  
    else:  
        pivo = array[0] ❷  
        menores = [i for i in array[1:] if i <= pivo] ❸  
        maiores = [i for i in array[1:] if i > pivo] ❹  
        return quicksort(menores) + [pivo] + quicksort(maiores)  
  
print quicksort([10, 5, 2, 3])
```

- ❶ Base: arrays com 0 ou 1 elemento já estão “ordenados”.
- ❷ Caso recursivo.
- ❸ Subarray de todos os elementos menores do que o pivô.
- ❹ Subarray de todos os elementos maiores do que o pivô.

Notação Big O revisada

O algoritmo quicksort é único, pois sua velocidade depende do pivô escolhido. Antes de falarmos sobre quicksort, vamos analisar novamente os tempos de execução Big O mais comuns.



Estimativas baseadas em um computador lento que realiza dez operações por segundo.

Os exemplos de tempos de execução contidos nestes gráficos são estimativas para um caso em que você executa dez operações por segundo. Estes gráficos não são precisos, mas servem apenas para fornecer um exemplo do quão diferente são os tempos de execução. Na realidade, o seu computador é capaz de executar muito mais do que dez operações por segundo.

Cada tempo de execução contém um algoritmo de exemplo anexo. Dê uma olhada no algoritmo de ordenação por seleção, que aprendeu no Capítulo 2. O seu tempo de execução é $O(n^2)$; é bastante lento.

Há outro algoritmo de ordenação chamado *merge sort*, que tem tempo de execução $O(n \log n)$, o que é muito mais rápido! O algoritmo quicksort é um caso complicado. Na pior situação, o quicksort tem tempo de execução $O(n^2)$.

Ele é tão lento quanto a ordenação por seleção! Porém este é o pior caso possível. No caso médio, o quicksort tem tempo de execução $O(n \log n)$. E agora você pode estar se perguntando:

- O que significa *pior caso* e *caso médio*?
- Se o quicksort tem tempo de execução médio $O(n \log n)$, e o merge sort tem tempo de execução $O(n \log n)$ sempre, por que não utilizar o merge sort? Não seria mais rápido?

Merge sort versus quicksort

Suponha que você tenha esta simples função que imprime na tela todos os itens de uma lista:

```
def imprime_itens(lista):  
    for item in lista:  
        print item
```

Esta função analisa cada item da lista e o imprime. Como esta função passa por toda a lista uma vez, ela tem tempo de execução $O(n)$. Agora, imagine que você modificou esta função para que ela aguarde um segundo antes de imprimir um item:

```
from time import sleep  
def imprime_itens2(lista):  
    for item in lista:  
        sleep(1)  
        print item
```

Antes de imprimir um item, ela espera por um segundo. Suponha que você imprima uma lista contendo cinco itens utilizando ambas as funções.



Ambas as funções passam por toda a lista uma vez, portanto elas têm tempo de execução $O(n)$. Qual das duas você acha que será mais rápida na prática? Acho que a função `imprime_itens` será muito mais rápida, visto que ela não aguarda um segundo antes de imprimir cada item. Assim, mesmo que ambas as funções tenham o mesmo tempo de execução na notação Big O, a função `imprime_itens` acaba sendo mais rápida na prática. Quando você escreve algo na notação Big O, como $O(n)$, por exemplo, está querendo dizer isso.

$$c * n$$

↑
ALGUMA QUANTIDADE
DETERMINADA DE TEMPO

A letra c é uma quantidade determinada de tempo que o seu algoritmo leva para ser executado. Ela é chamada de *constante*. Pode ser, por exemplo, 10 milissegundos * n para a função `imprime_itens` contra 1 segundo * n para a função `imprime_itens2`.

Normalmente você ignora a constante, pois, caso dois algoritmos tenham tempos de execução Big O diferentes, a constante não importará. Vamos usar a pesquisa binária e a pesquisa simples como exemplos deste fato. Imagine que ambos os algoritmos contenham estas constantes.

$10_{ms} * n$	$1_{seg} * \log n$
PESQUISA SIMPLES	PESQUISA BINÁRIA

Você pode pensar “Nossa! A pesquisa simples contém uma constante de 10 milissegundos, enquanto a pesquisa binária contém uma constante de um segundo. A pesquisa simples é muito mais rápida!”. Agora, suponha que esteja realizando uma busca em uma lista com 4 bilhões de elementos. A seguir, pode visualizar os tempos de execução desta busca.

PESQUISA SIMPLES	$10_{ms} * 4 \text{ BILHÕES} = 463 \text{ dias}$
PESQUISA BINÁRIA	$1_{seg} * 32 = 32 \text{ segundos}$

Como você pode ver, a pesquisa binária continua sendo muito mais rápida. A constante não causou diferença alguma no final das contas.

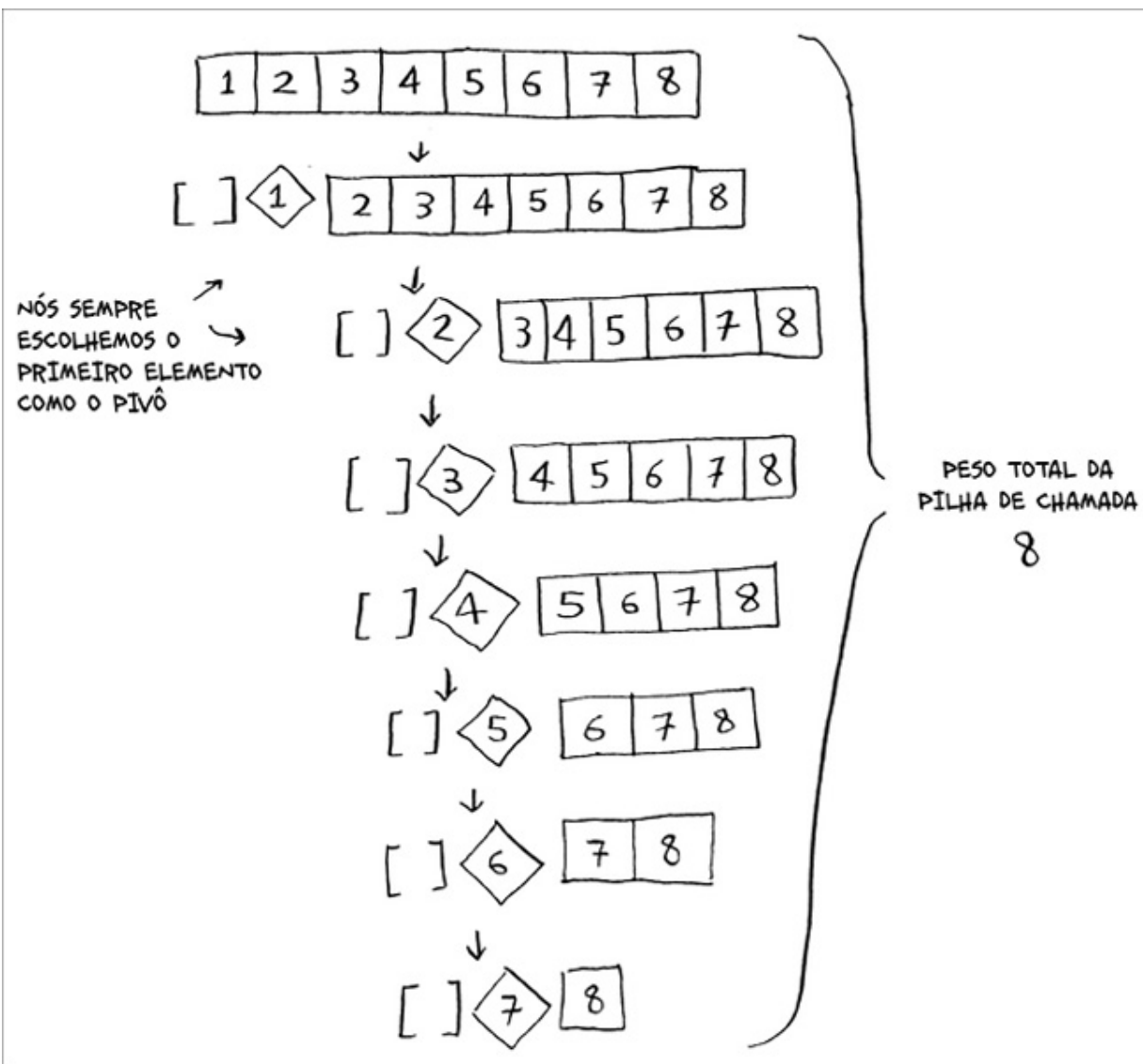
Porém, às vezes, a constante *pode* fazer diferença. O quicksort, comparado ao merge sort, é um exemplo disso. O quicksort tem uma constante menor do que o merge sort. Assim, como ambos têm tempo de execução $O(n \log n)$, o quicksort acaba sendo mais rápido. Além disso, o quicksort é mais rápido, na prática, pois ele funciona mais vezes no caso médio do que no

pior caso.

E agora você pode estar se perguntando: o que é o caso médio e o que é o pior caso?

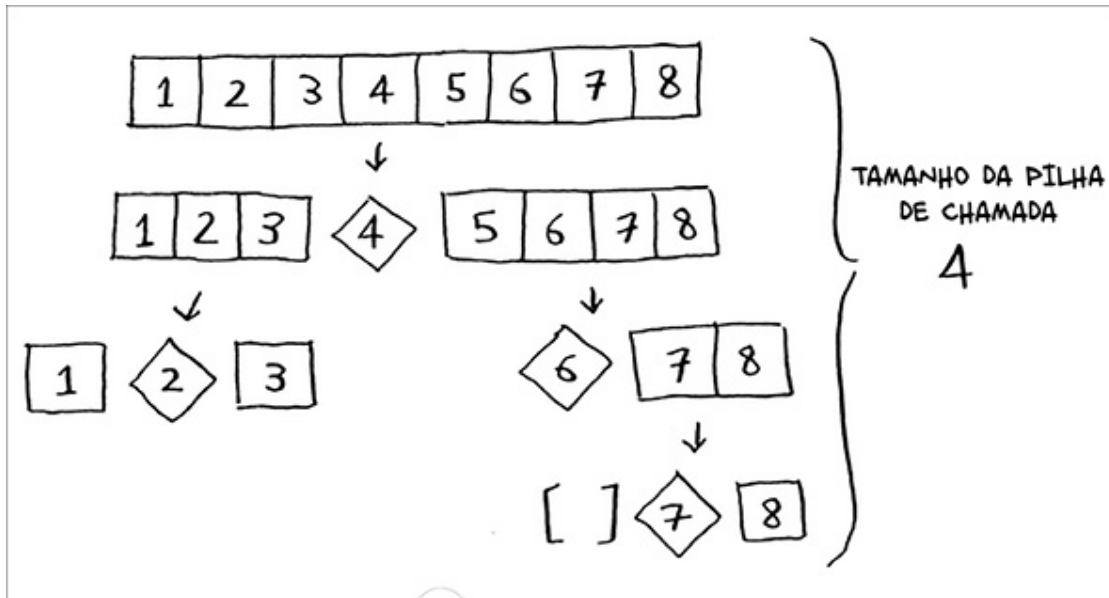
Caso médio versus pior caso

O desempenho do quicksort depende bastante da escolha do pivô. Imagine que você sempre escolha o primeiro elemento como pivô e que você execute o quicksort em um array que *já esteja ordenado*. O quicksort não faz uma checagem para conferir se o array já está ordenado. Logo, ele tentará ordenar o array mesmo assim.



Perceba como você não está dividindo o array em duas metades. Em vez

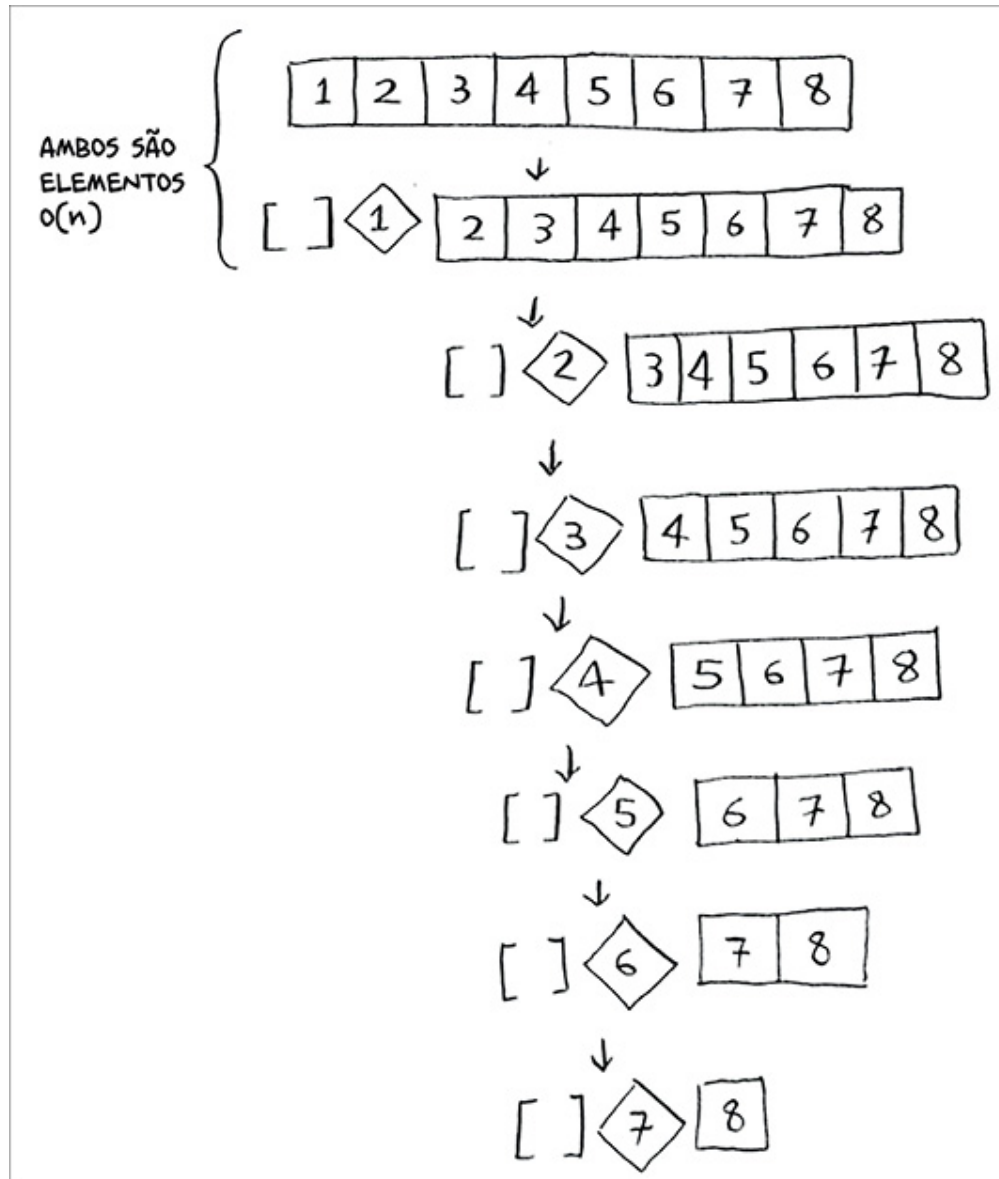
disso, um dos subarrays está sempre vazio, o que faz com que a pilha de chamada seja sempre muito longa. Agora, imagine que você sempre escolha o elemento central do array como pivô. Perceba como a pilha de chamada é menor.



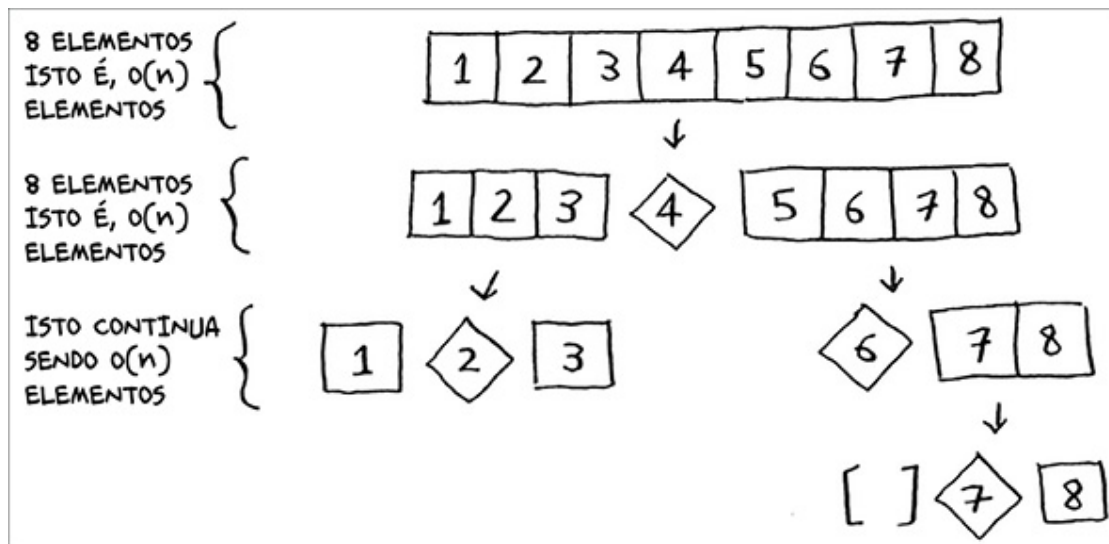
A pilha de chamada é muito menor! Isso acontece porque você divide o array na metade, o que faz com que você não precise fazer tantas execuções recursivas. Assim, você chega ao caso-base e a pilha de chamada é consideravelmente menor.

O primeiro exemplo que você viu representa o pior caso, enquanto o segundo exemplo representa o melhor caso. No pior caso, o tamanho da pilha é $O(n)$. No melhor caso, o tamanho da pilha é $O(\log n)$.

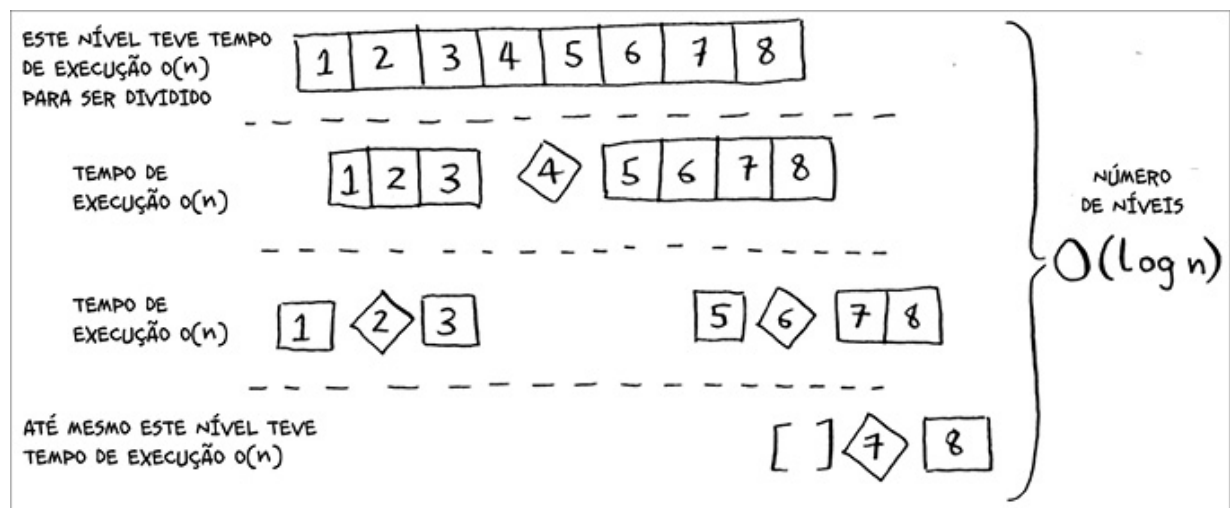
Agora, observe o primeiro nível da pilha. Você escolhe um elemento como o pivô e os demais elementos são divididos em dois subarrays. Assim, você tem de passar por todos os elementos no array. Logo, esta primeira execução tem tempo de execução $O(n)$. Neste nível da pilha você passa por todos os elementos do array. Porém em todos os níveis da pilha de chamada você passará por $O(n)$ elementos.



Mesmo que você particione o array de forma diferente, continuará passando por $O(n)$ elementos a cada execução.



Dessa forma, cada nível tem tempo de execução $O(n)$.

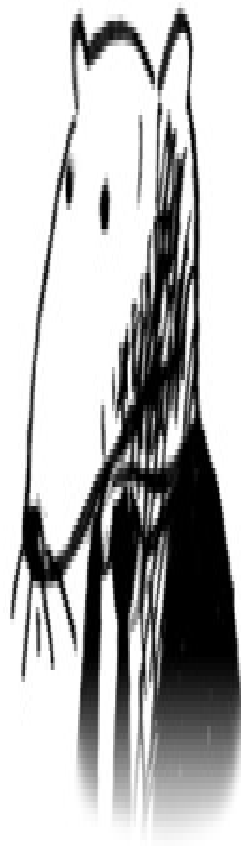


Neste exemplo, existem $O(\log n)$ níveis (a maneira mais técnica de dizer isso é “O peso da pilha de chamada [ou pilha de execução] é $O(\log n)$ ”). Cada nível tem tempo de execução $O(n)$. Além disso, o algoritmo como um todo tem tempo de execução $O(n) * O(\log n) = O(n \log n)$. Este é o melhor caso. No pior caso, existem $O(n)$ níveis. Portanto o algoritmo tem tempo de execução $O(n) * O(n) = O(n^2)$.

Adivinhe? O melhor caso também é o caso médio. Se você sempre escolher um elemento aleatório do array como pivô, o quicksort será completado com tempo de execução médio $O(n \log n)$. O algoritmo quicksort é um dos mais rápidos algoritmos de ordenação que existem, sendo um ótimo exemplo de DC.

8

Algoritmos gulosos



Neste capítulo

- Você aprenderá como lidar com o impossível: problemas que não têm um algoritmo de solução rápida (problemas NP-completo).
- Você aprenderá como identificar esses problemas ao se deparar com eles, de forma que não perca tempo tentando achar um algoritmo rápido para solucioná-los.

- Você conhecerá os algoritmos de aproximação, que podem ser usados para encontrar, de maneira rápida, uma solução aproximada para um problema NP-completo.
- Você conhecerá a estratégia gulosa, uma estratégia muito simples para resolver problemas.

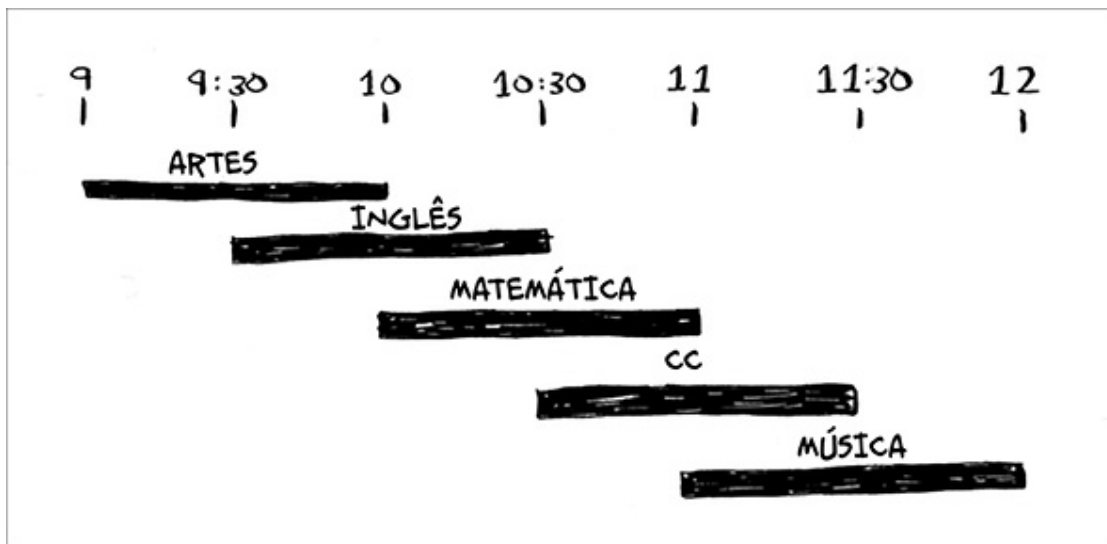
O problema do cronograma da sala de aula



Suponha que você tenha uma sala de aula e queira reservar o máximo de aulas possível nela. Assim, recebe-se uma lista das aulas.

AULA	INÍCIO	FIM
ARTES	9 AM	10 AM
INGLÊS	9:30 AM	10:30 AM
MATEMÁTICA	10 AM	11 AM
CC	10:30 AM	11:30 AM
MÚSICA	11 AM	12 PM

Você não pode reservar *todas* essas aulas na sala porque os horários de algumas delas coincidem.



Soa como um problema difícil, não? Na realidade, o algoritmo é tão simples que pode surpreender. Aqui temos o funcionamento dele:

1. Pegue a aula que termina mais cedo. Esta é a primeira aula que você colocará nessa sala.
2. Agora você precisa pegar uma aula que comece depois da primeira aula. De novo, pegue a aula que termine mais cedo. Esta é a segunda aula que você colocará

Continue fazendo isso e no final você terá a sua resposta! Vamos testar: Artes termina mais cedo, às 10h00, então esta é a aula escolhida.

ARTES	9 AM	10 AM	✓
INGLÊS	4:30 AM	10:30 AM	
MATEMÁTICA	10 AM	11 AM	
CC	10:30 AM	11:30 AM	
MÚSICA	11 AM	12 PM	

Agora você precisa da próxima aula, que começa depois das 10h00 e termina mais cedo que as demais.

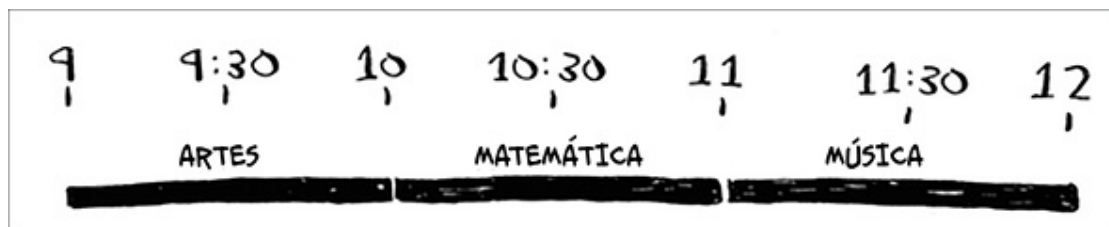
ARTES	9 AM	10 AM	✓
INGLÊS	4:30 AM	10:30 AM	X
MATEMÁTICA	10 AM	11 AM	✓
CC	10:30 AM	11:30 AM	
MÚSICA	11 AM	12 PM	

Inglês não pode ser escolhido porque tem conflito com Artes, mas a aula de Matemática encaixa.

Por fim, Ciências da Computação tem conflito com Matemática, mas a aula de Música encaixa.

ARTES	9 AM	10 AM	✓
INGLÊS	9:30 AM	10:30 AM	X
MATEMÁTICA	10 AM	11 AM	✓
CC	10:30 AM	11:30 AM	X
MÚSICA	11 AM	12 PM	✓

Então, estas são as três aulas que você colocará nessa sala de aula.



Muitas pessoas me dizem que esse algoritmo parece ser fácil. Mas ele é óbvio demais, logo, deve estar errado. No entanto essa é a beleza dos algoritmos gulosos (também chamados de algoritmos gananciosos): eles são fáceis! Um algoritmo guloso é simples: a cada etapa, deve-se escolher o movimento ideal. Nesse caso, cada vez que você escolhe uma aula, deve escolher a que acaba mais cedo. Em termos técnicos: *a cada etapa, escolhe-se a solução ideal*, e no fim você tem uma solução global ideal. Acredite ou não, esse algoritmo simples acha a solução ideal para esse problema!

Obviamente os algoritmos gulosos nem sempre funcionam, mas eles são tão simples de escrever! Vamos olhar outro exemplo.



O problema da mochila

Suponha que você seja um ladrão ganancioso e esteja em uma loja com sua mochila. Na loja existem diversos itens que você pode roubar. Porém você só pode levar aquilo que caiba na sua mochila, que só suporta 16 quilos.



Você está tentando maximizar o valor dos itens que colocará na sua mochila. Para isso, que algoritmo você usa?

1. Pegue o item mais caro que caiba na sua mochila.
2. Pegue o próximo item mais caro que caiba na sua mochila, e assim por diante.

Dessa vez, o algoritmo não funciona! Por exemplo, suponha que existam três itens que você possa roubar.



Sua mochila suporta 16 quilos. O aparelho de som é o item mais caro, então você pode roubá-lo. Mas agora não há espaço para mais nada.



Você roubou R\$ 3.000 em bens, mas espere um pouco! Caso tivesse pegado o notebook e o violão, você poderia ter R\$ 3.500!



Claramente, a estratégia gulosa não oferece a melhor solução aqui, mas fornece um valor bem próximo. No próximo capítulo explicarei como

calcular a solução correta, mas se você é um ladrão em um shopping, não se importa com a melhor solução. “Muito bom” é bom o suficiente.

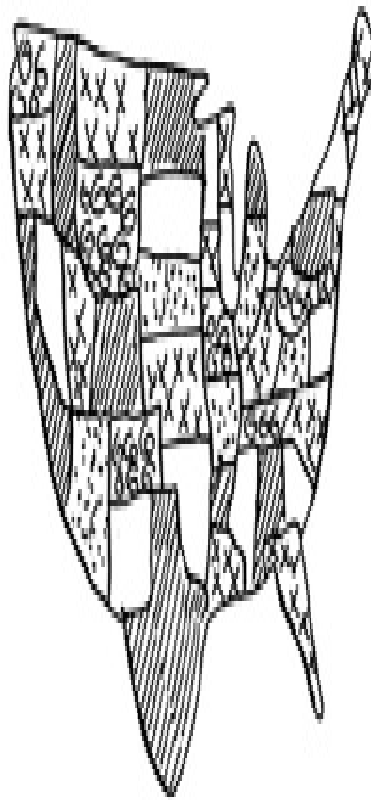
Moral da história para este exemplo: às vezes, o melhor é inimigo do bom. Em alguns casos, tudo o que você precisa é de um algoritmo que resolva o problema de uma maneira muito boa. E é aí que os algoritmos gulosos entram, pois eles são simples de escrever e normalmente chegam bem perto da solução perfeita.

EXERCÍCIOS

8.1 Você trabalha para uma empresa de móveis e tem de enviar os móveis para todo o país. É necessário encher seu caminhão com caixas, e todas as caixas são de tamanhos diferentes. Você está tentando maximizar o espaço que consegue usar em cada caminhão. Como escolheria as caixas para maximizar o espaço? Proponha uma solução gulosa. Ela lhe dará a solução ideal?

8.2 Você está viajando para a Europa e tem sete dias para visitar o maior número de lugares. Para cada lugar você atribui um valor (o quanto deseja ver) e estima quanto tempo demora. Como maximizar o total de pontos (passar por todos os lugares que realmente quer ver) durante sua estadia? Proponha uma solução gulosa. Ela lhe dará a solução ideal?

Vamos analisar um último exemplo. Este é um exemplo em que algoritmos gulosos são absolutamente necessários.

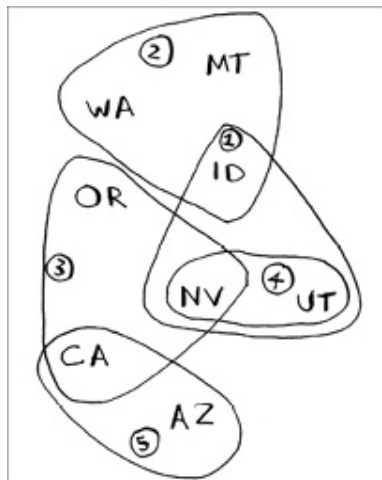


O problema da cobertura de conjuntos

Suponha que você esteja começando um programa de rádio e queira atingir ouvintes em todos os cinquenta estados americanos. É necessário decidir em quais estações transmitir para atingir todos os ouvintes. Porém transmitir em diferentes estações tem um custo, e você está tentando minimizar o número de estações nas quais você transmite para minimizar o custo. Temos uma lista de estações.

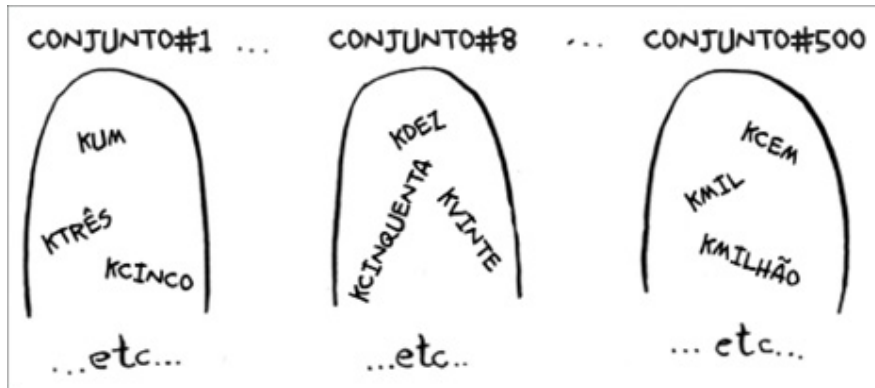
ESTAÇÃO DE RÁDIO	DISPONÍVEL EM
KUM	ID, NV, UT
KDOIS	WA, ID, MT
KTRÊS	OR, NV, CA
KQUATRO	NV, UT
KCINCO	CA, AZ
...etc...	

Cada estação abrange uma região e existe uma sobreposição.



Como descobrir o menor conjunto de estações nas quais você pode transmitir e abranger os cinquenta estados? Soa fácil, não? Acontece que é extremamente difícil. Aqui está uma solução:

1. Liste cada subconjunto possível de estações. Isso é chamado de *conjunto de partes* (também conhecido como conjunto de potência). Neste caso, existem 2^n possíveis conjuntos.



2. Entre eles, escolha o conjunto com o menor número de estações que abranja todos os cinquenta estados.

O problema neste caso é que o tempo para calcular cada possível subconjunto de estações é muito longo, uma vez que o tempo de execução é $O(2^n)$, pois existem 2^n subconjuntos. Seria possível calcular se você tivesse um grupo pequeno de cinco a dez estações, mas, como em todos os exemplos aqui, pense o que aconteceria se você tivesse muitos itens. O tempo com um maior número de estações será longo demais. Para exemplificar, suponha que você consiga calcular dez subconjuntos por segundo.

Não existe um algoritmo que resolva isso rápido o suficiente! O que você pode fazer?

NÚMERO DE ESTAÇÕES	TEMPO NECESSÁRIO
5	3.2 seg
10	102.4 seg
32	13.6 anos
100	4×10^{21} anos

Algoritmos de aproximação

Algoritmos gulosos ao resgate! Aqui temos um algoritmo guloso que chega bem perto da solução:

1. Pegue a estação que abranja o maior número de estados que ainda não foram cobertos. Tudo bem se a estação abranger alguns estados que já

foram cobertos.

2. Repita isso até que todos os estados tenham sido cobertos.

Isto se chama *algoritmo de aproximação*. Quando é necessário muito tempo para calcular a solução exata, um algoritmo de aproximação é uma boa ideia e funciona. Os algoritmos de aproximação são avaliados

- por sua rapidez;
- pela capacidade de chegar à solução ideal.

Os algoritmos gulosos são uma boa escolha porque eles são de fácil compreensão e sua simplicidade também indica que geralmente eles são de rápida execução. Nesse caso, o algoritmo guloso tem tempo de execução $O(n^2)$, em que n é o número de estações de rádio.

Vamos ver como é esse problema em código.

Código para o exemplo

Para esse exemplo, usarei um subconjunto de estados e estações para simplificar.

Primeiro, faça uma lista dos estados que deseja abranger:

```
estados_abranger = set(["mt", "wa", "or", "id", "nv", "ut", "ca",  
"az"]) ❶
```

❶ Você passa um array como entrada e ele é convertido em um conjunto.

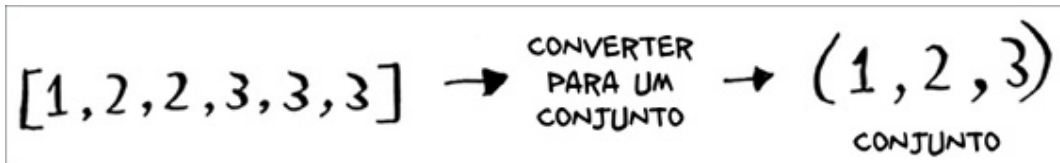
Usei um conjunto para isso, pois um conjunto é como uma lista, com exceção do fato de que cada item só pode aparecer uma vez. *Conjuntos não podem ter elementos duplicados*. Por exemplo, suponha que você tivesse esta lista:

```
>>> arr = [1, 2, 2, 3, 3, 3]
```

E a tivesse convertido para um conjunto:

```
>>> set(arr)  
set([1, 2, 3])
```

Os números 1, 2 e 3 aparecerão apenas uma vez no conjunto.



Você também precisa da lista de estações que podem ser escolhidas. Escolhi usar uma tabela hash para isso:

```
estacoes = {}  
estacoes["kum"] = set(["id", "nv", "ut"])  
estacoes["kdois"] = set(["wa", "id", "mt"])  
estacoes["ktres"] = set(["or", "nv", "ca"])  
estacoes["kquatro"] = set(["nv", "ut"])  
estacoes["kcinco"] = set(["ca", "az"])
```

Em português, vamos chamar essas estações de kum, kdois, ktres, e assim por diante.

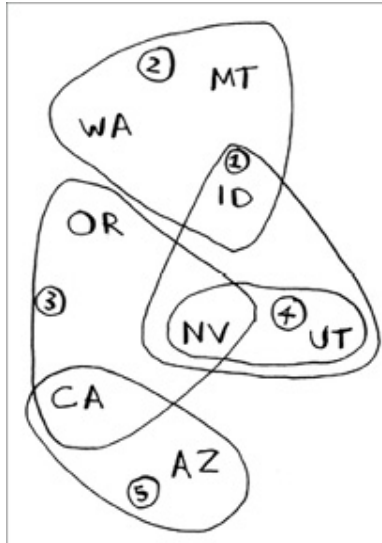
As chaves são os nomes das estações, e os valores são os estados que elas abrangem. Então, neste exemplo, a estação kum abrange Idaho (id), Nevada (nv) e Utah (ut). Todos os valores também são conjuntos, pois fazer com que tudo seja um conjunto tornará sua vida mais fácil, como verá em breve.

Finalmente, você precisa de algo para armazenar o conjunto final de estações. Para isso você usará:

```
estacoes_final = set()
```

Calculando a resposta

Agora, você deve calcular as estações que utilizará. Dê uma olhada na imagem a seguir e veja se consegue prever qual estação deve ser utilizada.



Pode existir mais de uma opção correta, sendo que você deve observar cada estação e escolher uma que cubra o maior número de estados não cobertos. Chamarei isso de `melhor_estacao`:

```
melhor_estacao = None
estados_cobertos = set()
for estacao, estados_por_estacao in estacoes.items():
```

`estados_cobertos` é um conjunto de todos os estados que essa estação abrange que ainda não foram cobertos. O loop `for` lhe permite percorrer todas as estações para ver qual é a melhor estação. Vamos olhar o conteúdo do loop `for`:

```
cobertos = estados_abranger & estados_por_estacao ❶
if len(cobertos) > len(estados_cobertos):
    melhor_estacao = estacao
    estados_cobertos = cobertos
```

❶ Nova sintaxe! Isso é chamado de intersecção.

Tem uma linha engraçada aqui:

```
cobertos = estados_abranger & estados_por_estacao
```

O que está acontecendo?

Conjuntos

Suponha que você tenha um conjunto de frutas e também tenha um conjunto de vegetais.

Quando você tem dois conjuntos, é possível fazer algumas coisas legais com eles.



Vou exemplificar algumas coisas que você pode fazer com conjuntos.



- Uma união significa “combine os dois conjuntos”.
- Uma intersecção significa “encontre os itens que aparecem nos dois conjuntos” (nesse caso, apenas o tomate).
- Uma diferença significa “subtraia os itens de um conjunto dos itens do outro conjunto”.

Por exemplo:

```
>>> frutas = set(["abacate", "tomate", "banana"])
>>> vegetais = set(["beterraba", "cenoura", "tomate"])
```

```
>>> frutas | vegetais ❶
set(["abacate", "beterraba", "cenoura", "tomate", "banana"])
>>> frutas & vegetais ❷
set(["tomate"])
>>> frutas - vegetais ❸
set(["abacate", "banana"])
>>> vegetais - frutas ❹
```

- ❶ Isso é uma união.
- ❷ Isso é uma intersecção.
- ❸ Isso é uma diferença.
- ❹ O que você acha que isso fará?

Relembrando:

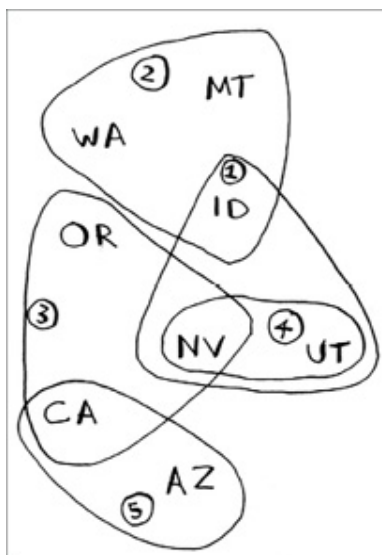
- Conjuntos são como listas, exceto pelo fato de não poderem ter elementos repetidos.
- Você pode fazer algumas operações interessantes com conjuntos como união, intersecção e diferença.

De volta ao código

Vamos voltar ao exemplo original.

Isto é uma intersecção:

```
cobertos = estados_abranger & estados_por_estacao
```



`cobertos` é um conjunto de estados que eram tanto `estados_abranger`

quanto `estados_por_estacao`. Então `cobertos` é o conjunto de estados não cobertos que essa estação abrange! Em seguida, verifique se essa estação abrange mais estados que a atual `melhor_estacao`:

```
if len(cobertos) > len(estados_cobertos):
    melhor_estacao = estacao
    estados_cobertos = cobertos
```

Caso ela abranja, essa estação é a nova `melhor_estacao`. Finalmente, depois que o loop `for` acabar, adicione `melhor_estacao` à lista final de estações:

```
estacoes_finais.add(melhor_estacao)
```

Você também precisará atualizar `estados_abranger`, pois esta estação abrange alguns estados, e esses estados não mais precisam de estações que os abranjam, ou seja, não são mais necessários para o algoritmo:

```
estados_abranger -= estados_cobertos
```

Assim, você fica em um loop até que `estados_abranger` esteja vazio. Aqui está o código completo para o loop:

```
while estados_abranger:
    melhor_estacao = None
    estados_cobertos = set()
    for estacao, estados in estacoes.items():
        cobertos = estados_abranger & estados
        if len(cobertos) > len(estados_cobertos):
            melhor_estacao = estacao
            estados_cobertos = cobertos

    estados_abranger -= estados_cobertos
    estacoes_finais.add(melhor_estacao)
```

Por fim, você pode imprimir `estacoes_finais`, e deverá ver isto:

```
>>> print estacoes_finais
set(['ktwo', 'kthree', 'kone', 'kfive'])
```

É isso que você esperava? Em vez das estações 1, 2, 3 e 5, as estações 2, 3, 4 e 5 poderiam ter sido escolhidas. Agora, vamos comparar o tempo de execução do algoritmo guloso e do algoritmo exato.

	$O(n!)$	$O(n^2)$
NÚMERO DE ESTAÇÕES	ALGORITMO EXATO	ALGORITMO GULOSO
5	3,2 seg	2,5 seg
10	102,4 seg	10 seg
32	13,6 anos	102,4 seg
100	4×10^{21} anos	16,67 min

EXERCÍCIOS

Para cada um desses algoritmos, diga se ele é um algoritmo guloso ou não.

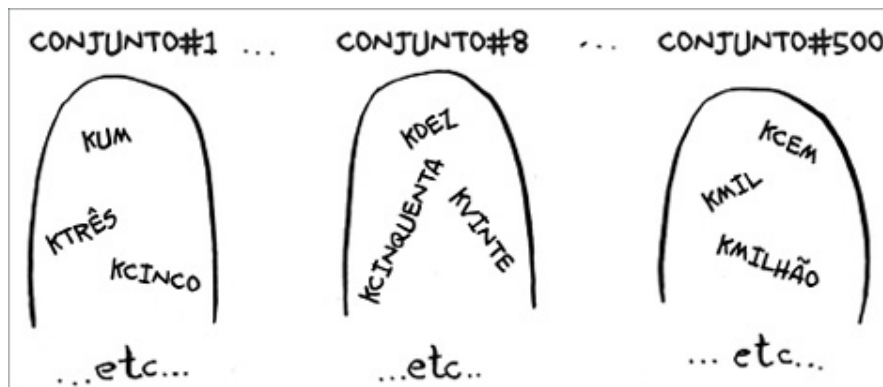
8.3 Quicksort

8.4 Pesquisa em largura

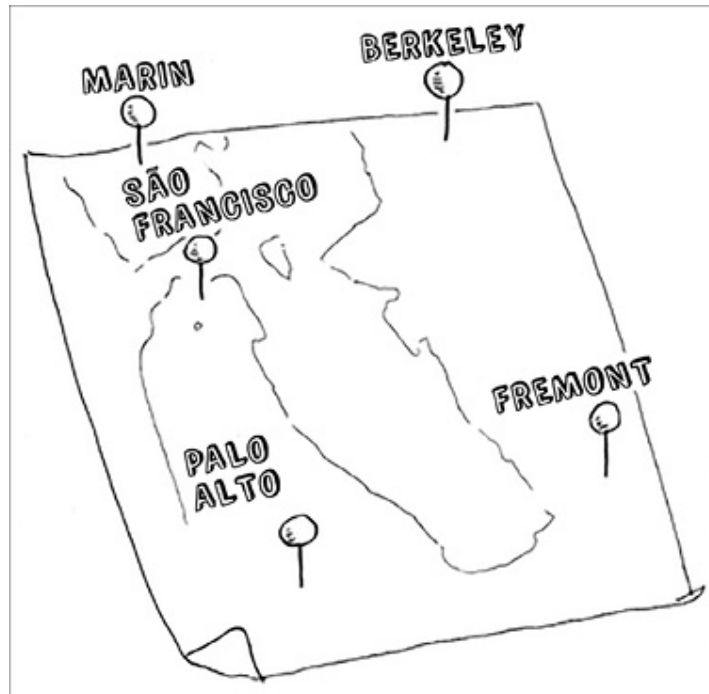
8.5 Algoritmo de Dijkstra

Problemas NP-completos

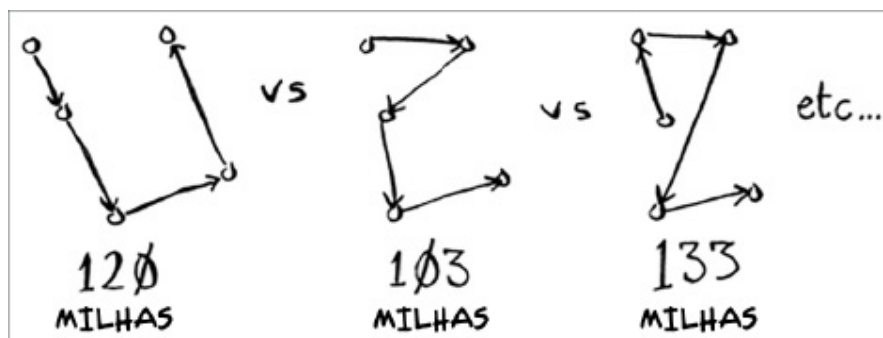
Para resolver o problema de cobertura de conjuntos você deve calcular cada conjunto possível.



Talvez isso o tenha feito se lembrar do problema do caixeiro-viajante do Capítulo 1. Neste problema, o caixeiro-viajante tem de visitar cinco cidades diferentes.



Ele está tentando descobrir a rota mais curta que o levará até as cinco cidades. Para encontrar a rota mais curta, primeiro devem-se calcular todas as rotas possíveis.



Quantas rotas devem-se calcular para cinco cidades?

Caixeiro-viajante, passo a passo

Vamos começar do básico. Suponha que você deseja visitar apenas duas cidades. Há duas rotas que você pode escolher.



Mesma rota ou rota diferente?

Você pode pensar que essa deveria ser a mesma rota. Porque, no fim das contas, $SF > Marin$ acaba tendo a mesma distância que $Marin > SF$, certo? Não necessariamente. Algumas cidades (como San Francisco) têm muitas ruas de sentido único, então você não pode voltar por onde veio. Ou seja, pode ser necessário seguir alguns quilômetros na direção errada para pegar o acesso a uma rodovia. Logo, duas rotas não são necessariamente a mesma coisa.

Você deve estar pensando “No problema do caixeiro-viajante, existe uma cidade específica de onde devo partir?”. Vamos dizer, por exemplo, que sou o caixeiro-viajante e que vivo em San Francisco e preciso ir para quatro cidades. San Francisco seria minha cidade de partida.

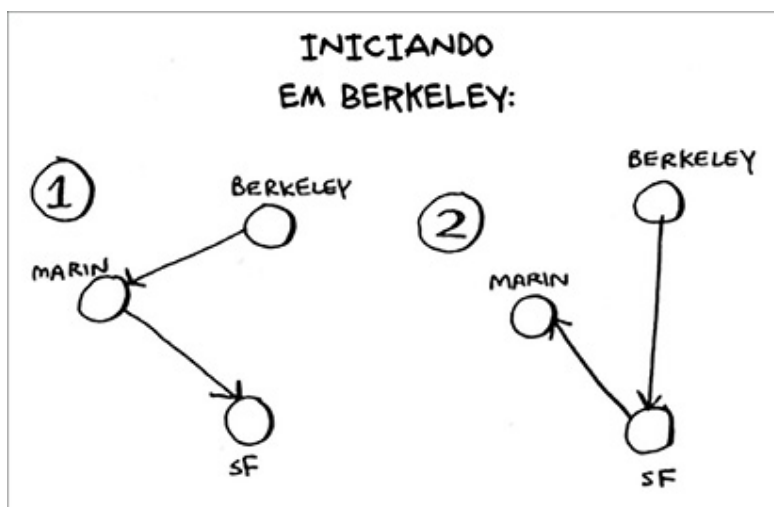
Porém, às vezes, a cidade de partida não está definida. Suponha que você seja o FedEx (serviço postal americano) tentando entregar um pacote em Bay Area (área da Baía de San Francisco). Esse pacote está vindo de Chicago para uma das cinquenta unidades da FedEx em Bay Area. Logo depois, o pacote será transportado em um caminhão que viajará para diferentes locais fazendo as entregas. Quando vindo de Chicago, para qual unidade em San Francisco o pacote deve ser enviado? Aqui o local de partida é desconhecido. Cabe a você calcular o caminho ideal e o local de partida para o caixeiro-viajante.

O tempo de execução das duas versões é o mesmo, mas o exemplo ficará mais fácil se não houver uma cidade de partida, então vou usar esta versão. Duas cidades = duas rotas possíveis.

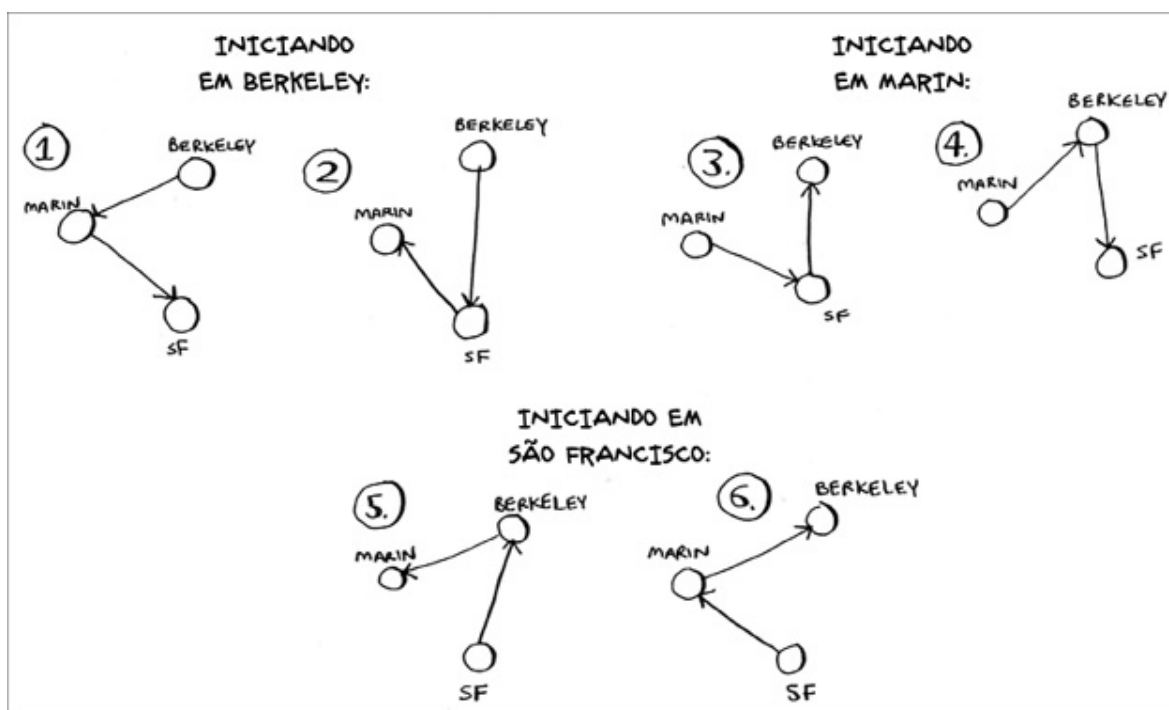
Três cidades

Agora suponha que você tenha adicionado mais uma cidade. Quantas rotas existem?

Se você começar em Berkeley, ainda deverá visitar mais duas cidades.



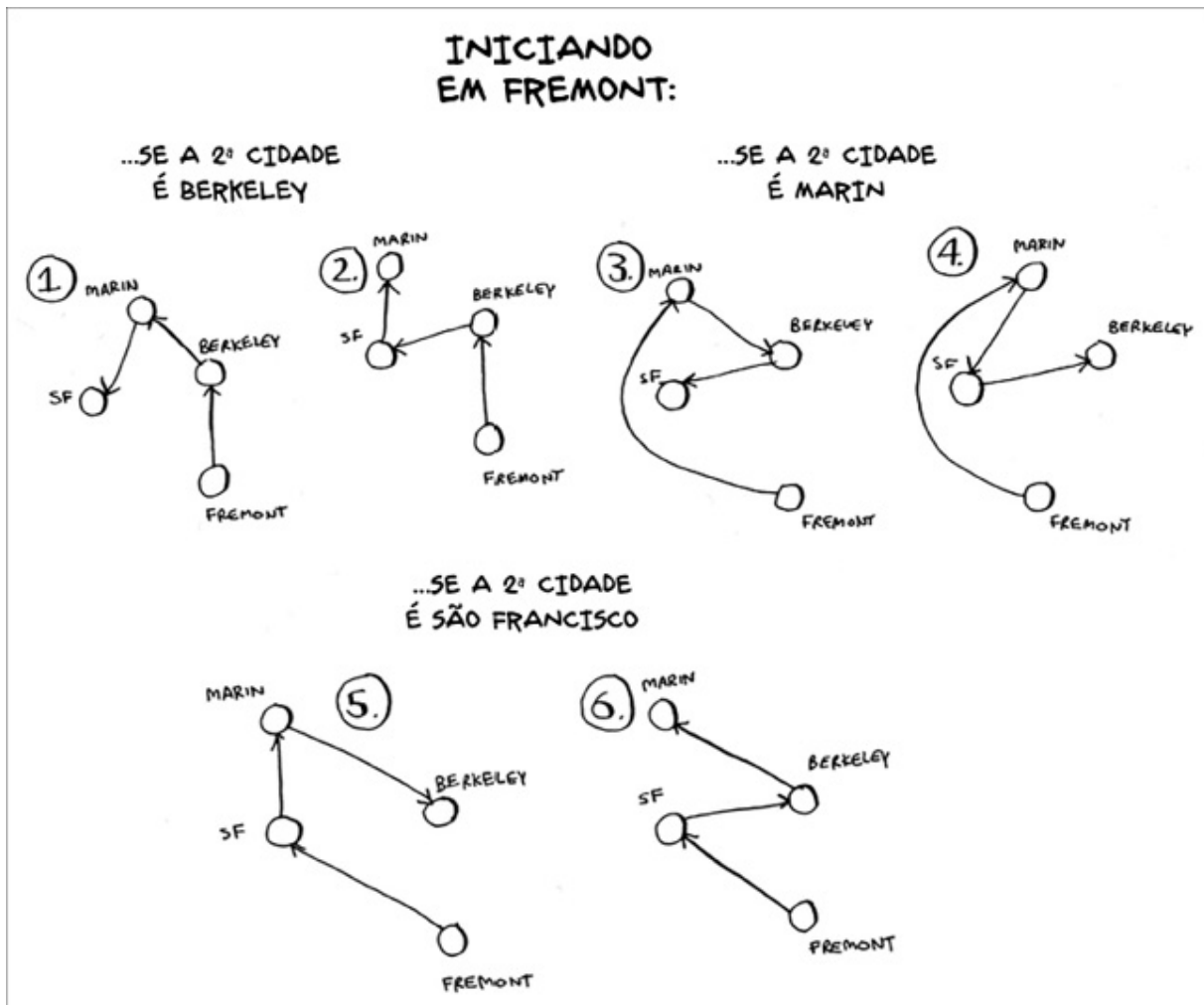
Há um total de seis rotas, duas para cada cidade em que pode começar.



Então três cidades = seis rotas possíveis.

Quatro cidades

Vamos adicionar outra cidade: Fremont. Suponha que você inicie lá.



Há seis rotas possíveis partindo de Fremont, e olhe só! Elas se parecem muito com as seis rotas que você calculou anteriormente, quando tinha apenas três cidades. Com exceção de que agora todas as rotas têm uma cidade adicional: Fremont! Há um padrão aqui, e para visualizá-lo, suponha que existam quatro cidades e que você possa escolher a cidade de partida. Você escolhe Fremont. Há três cidades sobrando, e se há três cidades, existem seis rotas diferentes para trafegar entre elas. Caso você inicie em Fremont, existem seis rotas possíveis. Também pode-se iniciar em uma das outras cidades.

INICIANDO EM MARIN:	INICIANDO EM SÃO FRANCISCO:
= 6 ROTAS POSSÍVEIS	= 6 ROTAS POSSÍVEIS
INICIANDO EM BERKELEY:	
= 6 ROTAS POSSÍVEIS	

Quatro cidades de partida possíveis, com seis rotas possíveis para cada cidade de partida = $4 * 6 = 24$ rotas possíveis.

Percebe o padrão? Cada vez que uma cidade é adicionada, o número de rotas que devem ser calculadas aumenta.

NÚMERO DE CIDADES	
1 →	1 ROTA
2 →	2 CIDADES INICIAIS * 1 ROTA PARA CADA INÍCIO = 2 ROTAS AO TOTAL
3 →	3 CIDADES INICIAIS * 2 ROTAS = 6 ROTAS AO TOTAL
4 →	4 CIDADES INICIAIS * 6 ROTAS = 24 ROTAS AO TOTAL
5 →	5 CIDADES INICIAIS * 24 ROTAS = 120 ROTAS AO TOTAL

Quantas rotas possíveis existem para seis cidades? Se você disse 720, está certo. Além disso, existem 5.040 rotas para sete cidades e 40.320 para oito cidades.

Isso é chamado de *função fatorial* (Você se lembra de ter lido sobre isso no Capítulo 3?). Então $5! = 120$. Suponha que você tenha dez cidades. Quantas rotas possíveis existem? $10! = 3.628.800$. Devem-se calcular perto de 3 milhões de rotas possíveis para dez cidades. Como você pode notar, o número de rotas possíveis cresce rapidamente! É por isso que é impossível calcular a solução “correta” para o problema do caixeiro-viajante caso o número de cidades seja muito elevado.

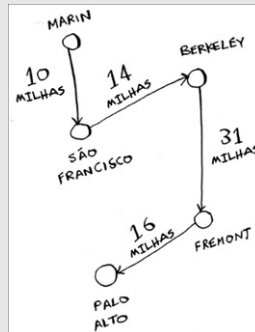
Tanto o problema do caixeiro-viajante quanto o problema da cobertura de

conjuntos têm algo em comum: calcula-se cada solução possível e escolhe-se a menor. Esses dois problemas são *NP-completos*.

Aproximando

Qual é uma aproximação boa para o algoritmo do caixeiro-viajante? Algo simples que encontre um caminho curto. Veja se consegue pensar em uma resposta antes de continuar a leitura.

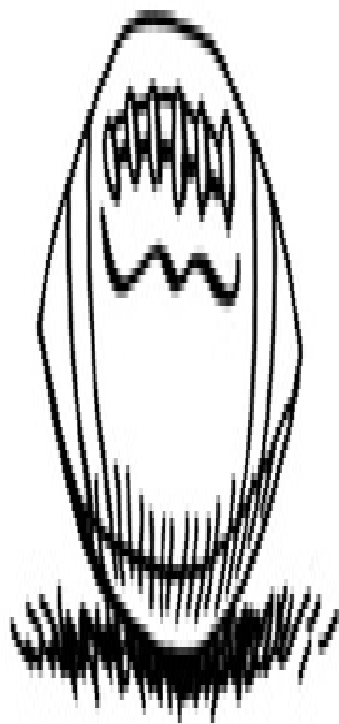
Como eu faria: aleatoriamente, escolheria uma cidade de partida. Em seguida, toda vez que o caixeiro-viajante tivesse de escolher a próxima cidade, ele escolheria a cidade não visitada mais próxima. Suponha que ele tenha começado em Marin.



Distância total: 71 quilômetros. Talvez não seja o menor caminho, mas, ainda assim, é bem curto.

Uma breve explicação sobre NP-completo: alguns problemas são notoriamente difíceis de resolver. O caixeiro-viajante e o problema de cobertura de conjuntos são dois exemplos. Diversas pessoas duvidam da possibilidade de criar um algoritmo que resolva esses problemas de forma rápida.

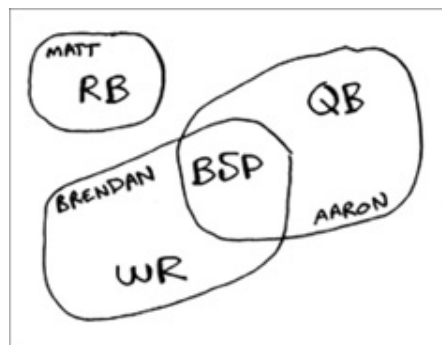
Como faço para saber se um problema é NP-completo?



Jonah está escolhendo jogadores para o seu time de futebol americano de mentira. Ele tem uma lista de características as quais gostaria que seu time tivesse: um bom quarterback, um bom running back, jogadores que joguem bem na chuva e que joguem bem sob pressão, entre outras habilidades. Ele tem uma lista de jogadores, e cada jogador preenche algumas dessas habilidades.

JOGADOR	HABILIDADES
MATT FORTE	RB
BRENDAN MARSHALL	WR / BOM SOB PRESSÃO
AARON RODGERS	QUARTERBACK / BOM SOB PRESSÃO
...	...

Jonah precisa de um time que preencha todas as características desejadas, mas o tamanho do time é limitado. “Espere um segundo”, Jonah pensa. “Este é um problema de cobertura de conjuntos!”

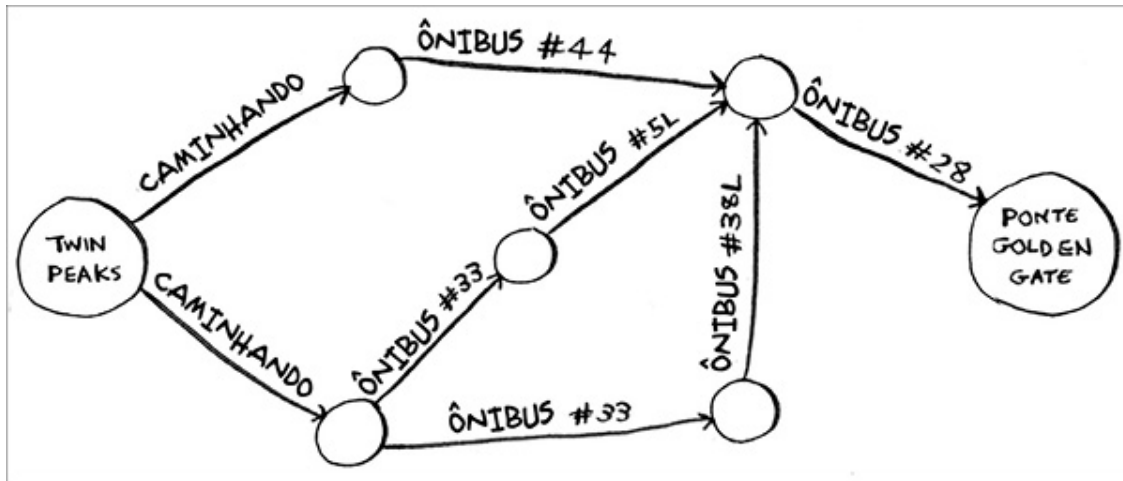


Jonah pode usar o mesmo algoritmo de aproximação para criar seu time:

1. Encontre o jogador que preenche o maior número de habilidades que ainda não foram preenchidas.
2. Repita até que o time tenha preenchido todas as habilidades (ou até que você fique sem espaço no time).

Problemas NP-completos aparecem em todo lugar! É sempre bom saber se o problema que você está tentando resolver é NP-completo, pois nesta situação você pode parar de tentar resolvê-lo perfeitamente e, em vez disso, resolvê-lo usando um algoritmo de aproximação. Porém é difícil perceber se o problema em que você está trabalhando é um problema NP-completo, pois normalmente a diferença entre um problema que é fácil de resolver e um NP-completo é muito pequena. Por exemplo, nos capítulos anteriores falei muito sobre caminhos mínimos. Você sabe como calcular o caminho

mínimo para chegar do ponto A ao ponto B.



Entretanto, se quiser encontrar o caminho mínimo que conecta vários pontos, cairá no problema do caixeiro-viajante, que é um problema NP-completo. A resposta simples é: não há uma maneira fácil de dizer se o problema em que você está trabalhando é NP-completo. Aqui temos alguns indicativos:

- Seu algoritmo roda rápido para alguns itens, mas fica muito lento com o aumento de itens.
- “Todas as combinações de X” geralmente significam um problema NP-completo.
- Você tem de calcular “cada possível versão” de X porque não pode dividir em subproblemas menores? Talvez seja um problema NP-completo.
- Se o seu problema envolve uma sequência (como uma sequência de cidades, como o problema do caixeiro-viajante) e é difícil de resolver, pode ser um NP-completo.
- Se o seu problema envolve um conjunto (como um conjunto de estações de rádio) e é difícil de resolver, ele pode ser um problema NP-completo.
- Você pode reescrever o seu problema como o problema de cobertura mínima de conjuntos ou o problema do caixeiro-viajante? Então seu problema definitivamente é NP-completo.

EXERCÍCIOS

8.6 Um carteiro deve entregar correspondências para vinte casas. Ele deve